Institut Saint-Laurent – Cours du soir

Le Langage C++

Marc Simoens
V4.18

Table des Matières

TABLE DES MATIERES II

TABLE DES FIGURES V

TABLE DES EXERCICES VI

TABLE DES EXEMPLES VII

1. VUE GENERALE DE LA PROGRAMMATION ORIENTEE OBJETS 8

1.1. APPROCHE COURANTE DE LA PROGRAMMATION 8
1.2. L'APPROCHE ORIENTEE OBJET 8
1.3. LES AVANTAGES DE L'OOP 10
1.3.1. La facilité de maintenance 10
1.3.2. La facilité de réutiliser le code 10
1.3.3. L'extensibilité 10
1.3.4. La modularisation 10
1.3.5. La facilité de corriger les erreurs 11

2. LES CONCEPTS CLES DE L'OOP 12

2.1. L'ABSTRACTION 12
2.1.1. Qu'est-ce qu'une abstraction ? 12
2.1.2. Comment implémenter une abstraction ? 12
2.2. L'HERITAGE 13
2.2.1. Les hiérarchies d'objets 13
2.2.1.A. La généralisation 13
2.2.1.B. L'association 13
2.2.1.C. L'agrégation 14
2.2.2. L'héritage 15
2.2.2.A. Définition 15
2.2.2.B. L'héritage multiple 16
2.2.3. Le polymorphisme 17

3. LES DIFFERENCES DU C++ PAR RAPPORT AU C 20

3.1. LES DECLARATIONS DE FONCTION 20
3.2. LES COMMENTAIRES 20
3.3. DE NOUVEAUX MOTS RESERVES 21
3.4. DE NOUVEAUX OPERATEURS 21
3.5. LA STRUCTURE 21
3.6. LE MOT CLE CONST 22
3.7. L'ALLOCATION DYNAMIQUE DE MEMOIRE 24
3.7.1. Les fuites de mémoire 25
3.8. LES REFERENCES 27
3.9. LE CAST 30
3.10. 0 ET NULL 31
3.11. LES POINTEURS GENERIQUES 31

4. LES ENTREES-SORTIES EN C++ 32

4.1. LES OBJETS DE TYPE FICHIER PREDEFINIS 32
4.2. QUELQUES EXEMPLES SIMPLES 35
4.3. DEFINITION DE NOUVEAUX OBJETS FICHIERS 35
4.4. LES FICHIERS EN MODE TEXTE ET EN MODE BINAIRE 38

5. PREMIERS PROGRAMMES 41

5.1. UNE SIMPLE LECTURE EN DOS 41
5.2. CREATION D'UN PROGRAMME WINDOWS AVEC APPWIZARD 41
5.3. AJOUTE DE CLASSES EXISTANTES DANS LE PROJET 42

6. LA CONSTRUCTION D'UNE CLASSE 43

6.1. INTRODUCTION 43

6.2. LES ATTRIBUTS DE CLASSE 43
6.3. LES SPECIFICATEURS D'ACCES 44
6.4. DECLARATION D'UN OBJET 45
6.5. INVOCATION D'UNE FONCTION 46
6.6. DEFINITION DES FONCTIONS MEMBRES 46
6.7. LE POINTEUR THIS 49
6.8. EXERCICE 51

7. CONSTRUCTEURS ET DESTRUCTEUR 52

7.1. LE CONSTRUCTEUR PAR DEFAUT 52
7.1.1. Quelques propriétés 52
7.2. PASSAGE DE PARAMETRES AU CONSTRUCTEUR 53
7.3. LE DESTRUCTEUR D'UNE CLASSE 54
7.4. EXERCICE 58
7.5. LES LISTES D'INITIALISATION 59
7.6. L'INITIALISATION MEMBRE A MEMBRE OU LE CONSTRUCTEUR PAR COPIE 60
7.7. CONSTRUCTEUR DE CONVERSION 62
7.8. CONSTRUCTION DES TABLEAUX D'OBJETS 62
7.9. EXERCICES 63
7.9.1. La classe CPoint 63
7.9.2. La classe Stack 63
7.9.3. La classe CChaine 63
7.9.4. Quels sont les résultats fournits 64
7.9.5. La classe Date 65

8. QUELQUES CARACTERISTIQUES INTERESSANTES 66

8.1. LES ARGUMENTS PAR DEFAUT 66
8.2. LA SURCHARGE DE FONCTIONS 68
8.3. LES FONCTIONS INLINE 70
8.3.1. Description 70
8.3.2. Les fonctions inline augmentent-elles la performance ? 72
8.4. LA NOTION DE FRIEND 73
8.4.1. Exercice 76
8.5. LES CONVERSIONS DE TYPE DANS LES CLASSES 76
8.5.1. Les conversions implicites 76
8.5.2. Les conversions explicites 77
8.5.3. Exercices 83
8.6. LA SURCHARGE DES OPERATEURS 84
8.6.1. Les opérateurs de type binaire 85
8.6.2. Les opérateurs de flux 88
8.6.3. Surcharge des opérateurs ++ et - - 89
8.6.4. Surcharge de l'opérateur d'indexation 93
8.6.5. La surcharge de l'opérateur d'affectation 94
8.6.6. Exercices 95
8.7. ALLOCATION DYNAMIQUE D'OBJETS ET TABLEAU D'OBJETS 97
8.8. LES MEMBRES STATIQUES 99
8.8.1. Les données membres statiques 99
8.8.2. Les fonctions membres statiques 100
8.9. LES ESPACES DE NOMS (NAMESPACE) 102
8.9.1. Description générale 102
8.9.2. Espace de nom anonyme 103
8.9.3. Comment implémenter une fonction définie dans un Namespace ? 103
8.9.4. À quoi sert le namespace std ? 104
8.9.5. Quelques remarques additionnelles 105

9. L'AGREGATION 106

9.1. DEFINITION DE LA RELATION HAS-A 106
9.2. L'ORDRE D'EXECUTION DES CONSTRUCTEURS ET DESTRUCTEURS 106
9.3. EXEMPLE 108
9.4. EXERCICE 111

10. L'HERITAGE 112

10.1. INTRODUCTION 112
10.2. LA SYNTAXE 112
10.3. SUBSTITUTION ET DOMINANCE DES MEMBRES HERITES 115
10.4. L'ORDRE D'INVOCATION DES CONSTRUCTEURS ET DESTRUCTEURS 116
10.5. EXEMPLE 118
10.6. EXERCICE 121
10.7. LE POLYMORPHISME 122
10.7.1. Exercices 128
10.7.2. Les destructeurs virtuels 132
10.8. LES CLASSES DE BASE ABSTRAITES 135
10.9. EXERCICE 137
10.10. L'HERITAGE MULTIPLE 138

11. LES TEMPLATES 140

11.1. LES FONCTIONS TEMPLATE 140
11.2. EXERCICE 143
11.3. LES CLASSES TEMPLATES 145
11.3.1. Spécialisation d'un template de classe 147
11.3.2. Paramètre constant pour une classe 147
11.3.3. Assignation d'objets 149
11.3.4. Classes template et héritage 149
11.3.5. Classes template et friend 150
11.3.6. Restriction des classes Template 150
11.3.7. Exemple : une Classe Liste 150
11.3.7.A. Fichier Liste 151
11.3.7.B. Fichier Cell 153
11.3.7.C. Fichier InterfaceListe 154
11.3.7.D. Fichier Rectangle 154
11.3.7.E. Fichier Rectangle.cpp 154
11.3.7.F. Fichier Liste-Template.cpp 155
11.4. EXERCICE 156

12. LA GESTION DES ERREURS 157

12.1. INTRODUCTION 157
12.2. LES RETURN CODES 157
12.3. LES EXCEPTIONS 158
12.3.1. Le mécanisme en général 158
12.3.2. Un exemple 160
12.3.3. La classe Exception 161
12.3.4. Le Polymorphisme 164
12.3.5. Les classes d'exception standards 165
12.3.6. Exercices 165
12.4. LES ASSERTIONS 170

13. LES COMPOSANTS STANDARDS 171

13.1. INTRODUCTION 171
13.2. EXERCICE 171

14. SOLUTIONS POUR LES EXERCICES DU DERNIER CHAPITRE 175

Table des Figures

FIGURE 1 : UN GRAPHE DE GENERALISATION 13
FIGURE 2 : GRAPHE D'ASSOCIATION 14
FIGURE 3 : GRAPHE D'AGREGATION 14
FIGURE 4 : GRAPHE DE L'HERITAGE 15
FIGURE 5 : HERITAGE ET ATTRIBUTS PROPRES 16
FIGURE 6 : GRAPHE DE L'HERITAGE MULTIPLE 17
FIGURE 7 : OPERATION VIRTUELLE 18
FIGURE 8 : OPERATION VIRTUELLE 18
FIGURE 9 : HIERARCHIE DE CLASSES POUR LES E/S 32
FIGURE 10 : LE PRINCIPE DU TYPE DE DONNEES ABSTRAIT 45
FIGURE 11 : ALLOCATION DYNAMIQUE D'OBJETS 98
FIGURE 12 : TERMINOLOGIE ANGLAISE DE LA RELATION "PARTIE DE " 106
FIGURE 13 : ORDRE D'EXECUTION DES CONSTRUCTEURS DANS UNE AGREGATION 107
FIGURE 14 : L'HERITAGE SIMPLE 112
FIGURE 15 : HERITAGE PUBLIC ET HERITAGE PRIVATE 113
FIGURE 16 : UNE CLASSE DE BASE ABSTRAITE 135

Table des Exercices

EXERCICE PRJ0 1 : LA CLASSE FICHE SALAIRE....51
EXERCICE PRJ0 2 : LA CLASSE FICHE SALAIRE....51
EXERCICE PRJ0 3 : LA CLASSE CSETINT....58
EXERCICE PRJ0 4 : LA CLASSE CCHAINE....63
EXERCICE PRJ0 5 : LA CLASSE CSETINT....63
EXERCICE PRJ0 6 : INSCRIVEZ SUR PAPIER LES RESULTATS FOURNITS PAR LE PROGRAMME SUIVANT : 64
EXERCICE PRJ0 7 : ECRIRE UNE FONCTION FRIEND....76
EXERCICE PRJ08 : LES CONVERSIONS....83
EXERCICE PRJ09 : AJOUTER DES OPERATEURS DE CONVERSION....83
EXERCICE PRJ010 : LA SURCHARGE DES OPERATEURS....95
EXERCICE PRJ011 : LA SURCHARGE DES OPERATEURS....95
EXERCICE PRJ012 : L'HERITAGE....121
EXERCICE PRJ013 :LE POLYMORPHISME....128
EXERCICE PRJ014 : LE POLYMORPHISME....129
EXERCICE PRJ015 : LE POLYMORPHISME....130
EXERCICE PRJ016 : LES FONCTIONS TEMPLATES....143
EXERCICE PRJ017 : LES FONCTIONS TEMPLATES....143
EXERCICE PRJ018 : LES FONCTIONS TEMPLATES....143
EXERCICE PRJ019 : LES FONCTIONS TEMPLATES....143
EXERCICE PRJ020 : LES CLASSES TEMPLATE....156
EXERCICE PRJ021 : LES CLASSES TEMPLATE....156
EXERCICE PRJ022 : LES CLASSES TEMPLATE....156
EXERCICE PRJ023 : LE EXCEPTIONS....165
EXERCICE PRJ024 : LES EXCEPTIONS....166
EXERCICE PRJ025 : LES EXCEPTIONS....167
EXERCICE PRJ026 : LES EXCEPTIONS....168
EXERCICE PRJ027 : LES EXCEPTIONS....170
EXERCICE PRJ028 : CREER UNE CLASSE MANIPULANT DES ENTIERS....171
EXERCICE PRJ029 : AJOUTER UN ITERATEUR A CLASSE PERMETTANT DE MANIPULER DES ENSEMBLES D'ENTIERS....172
EXERCICE PRJ030 : UTILISER LA CLASSE VECTEUR....173
EXERCICE PRJ031 : UTILISER LA CLASSE VECTEUR EN SECURISANT LE [ ]....174
EXERCICE PRJ032 : TABLEAUX DYNAMIQUES A DEUX INDICES....174

Table des Exemples

EXEMPLE 1 : L'UTILISATION DES REFERENCES DANS LA FONCTION SWAP 29
EXEMPLE 2 : UTILISATION OFSTREAM ET FSTREAM 37
EXEMPLE 3 : UTILISATION DE FICHIERS EN MODE TEXTE 40
EXEMPLE 4 : UTILISATION DE FICHIERS EN MODE BINAIRE 40
EXEMPLE 5 : CLASSE CRECTANGLE 43
EXEMPLE 6 : LA CLASSE COMPLEX 44
EXEMPLE 7 : DECLARATION D'UN OBJET 46
EXEMPLE 8 : DEFINITION DE LA FONCTION CRECTANGLE::SURFACE 46
EXEMPLE 9 : PREMIERE UTILISATION D'UN OBJET 47
EXEMPLE 10 : DEFINITION D'UN CONSTRUCTEUR 52
EXEMPLE 11 : DEFINITION DE PLUSIEURS CONSTRUCTEURS DANS UNE CLASSE 53
EXEMPLE 12 : CONSTRUCTEUR ET DESTRUCTEUR POUR LA CLASSE CRECTANGLE 55
EXEMPLE 13 : LA CLASSE CTRACE 57
EXEMPLE 14 : LISTE D'INITIALISATION 59
EXEMPLE 15 : INITIALISATION MEMBRE A MEMBRE 60
EXEMPLE 16 : INITIALISATION MEMBRE A MEMBRE POUR LA CLASSE CCHAINE 60
EXEMPLE 17 : UNE FONCTION RETOURNE UN OBJET COMME VALEUR DE RETOUR 61
EXEMPLE 18 : LE CONSTRUCTEUR X (CONST X&) POUR LA CLASSE CCHAINE 62
EXEMPLE 19 : UTILISATION D'UNE FONCTION AVEC DES PARAMETRES PAR DEFAUT 68
EXEMPLE 20 : SURCHARGE DE FONCTION DANS LA CLASSE CRECTANGLE 70
EXEMPLE 21 : FONCTIONS INLINE IMPLICITES 71
EXEMPLE 22 : FONCTIONS INLINE EXPLICITES 72
EXEMPLE 23 : UNE FONCTION FRIEND 75
EXEMPLE 24 : OPERATEUR DE CONVERSION DANS LA CLASSE CRATIONAL 79
EXEMPLE 25 : CONSTRUCTEUR DE CONVERSION ET OPERATEUR DE CONVERSION 82
EXEMPLE 26 : LA SURCHARGE DES OPERATEURS 86
EXEMPLE 27: L'OPERATEUR D'EXTRACTION POUR LA CLASSE CRATIONAL 88
EXEMPLE 28 : L'OPERATEUR D'INSERTION DE LA CLASSE CRATIONAL 89
EXEMPLE 29 : SURCHARGE DE L'OPERATEUR D'INCREMENTATION PREFIXE 91
EXEMPLE 30 : SURCHARGE DE L'OPERATEUR D'INCREMENTATION POSTFIXE 92
EXEMPLE 31 : SURCHARGE DE L'OPERATEUR D'INDEXATION 93
EXEMPLE 32 : L'OPERATEUR D'AFFECTATION DANS LA CLASSE CRATIONAL 94
EXEMPLE 33 : UTILISATION D'UNE DONNEE STATIQUE 100
EXEMPLE 34 : UTILISATION D'UNE FONCTION MEMBRE STATIQUE 101
EXEMPLE 35 : LA CLASSE CRECTANGLE AVEC UN MEMBER DE TYPE CPOINT 109
EXEMPLE 36 : HERITAGE PUBLIQUE 120
EXEMPLE 37 : LES FONCTIONS POLYMORPHIQUES 127
EXEMPLE 38 : UN DESTRUCTEUR VIRTUEL 134
EXEMPLE 39 : INSTANCIATION D'UNE FONCTION TEMPLATE 141

1. Vue générale de la programmation orientée objets

1.1. Approche courante de la programmation

Les premiers langages de programmation qui ont été développés étaient du type **procédural** et étaient principalement conçus pour accomplir des tâches simples. Comme la capacité des ordinateurs n'a cessé d'augmenter, il est devenu possible de développer des programmes de plus en plus complexes et pour cela les langages procéduraux sont devenus inadéquats : la possibilité de réutiliser du code déjà écrit était pratiquement impossible. Les programmes étaient formés d'un seul bloc, ce qui rendait leur logique difficile à comprendre.

Les informaticiens ont alors élaboré une nouvelle programmation : la **programmation structurée**. Un programme structuré est un programme organisé autour des opérations qu'il doit réaliser et il est décomposé en procédures ou fonctions. On essaie toujours de garder ses fonctions les plus indépendantes les unes des autres : chacune d'elle possédant ses données et sa logique. Le langage C est un exemple de langage structuré.

Malgré cette nouvelle approche, la programmation structurée montre rapidement ses limites lorsqu'elle s'applique à des problèmes qui ne cessent de croître en complexité. Par exemple, lorsque le nombre d'instructions *if…then…else* augmente, la structure du programme devient difficile à maîtriser. Des changements à apporter dans une structure fortement imbriquée deviennent rapidement très difficile à réaliser.

1.2. L'approche orientée objet

OOP = Object-Oriented Programming

L'OOP a été mis au point afin d'offrir une solution aux programmeurs pour les aider à résoudre des problèmes complexes. L'OOP se base sur l'approche orientée objet et considère un système comme étant un ensemble d'objets qui interagissent les uns avec les autres.

La méthode orientée-objet se rapproche beaucoup plus de la pensée humaine que la méthode procédurale. Nous modélisons le monde en termes d'objets et nous leur attribuons de l'intelligence. Les objets reflètent leurs propres capacités.

Par exemple, on demande à un technicien de réparer une machine ; il va utiliser son intelligence pour le faire. Nous lui avons demandé une chose : nous ne lui avons pas donné une suite d'instructions (algorithme) pour le faire : on sait que le technicien existe et qu'il peut offrir des services.

Avec une interface graphique (GUI = Graphical User Interface), les utilisateurs réagissent de façon très intuitive et travaillent avec des objets (fenêtres, scroll bar, macros, …). Ils agissent sur les objets et ceux-ci répondent. L'intelligence que possède chaque objet et qui lui permet de réagir aux demandes de l'utilisateur, est cachée à l'intérieur de l'objet.

Les composants de l'OOP sont :

- Les données
- Les méthodes
- Les objets

- Les membres
- Les classes

Les données

Les données constituent l'information qu'un objet utilise et traite. Par exemple, si on parle de l'objet *CRectangle*, ses données pourraient être constituées de :

- Sa longueur
- Sa largeur
- Sa couleur

Les méthodes

Les méthodes sont les opérations que l'on peut réaliser sur les données. Par exemple, sur le CRectangle, on pourrait réaliser les opérations suivantes :

- Le déplacer
- Le redimensionner
- En calculer sa surface
- Changer sa couleur

Les objets

Un objet est une entité identifiable : il est identifié par ses caractéristiques et par son comportement. Les caractéristiques et le comportement d'un objet le distinguent des autres objets. Le comportement de l'objet est mis en oeuvre par les méthodes tandis que ses caractéristiques forment les données qui lui sont associées. Par exemple, le *CRectangle* est un objet identifiable.

Les membres

Un objet peut contenir des données et des méthodes : on les appellera les membres de l'objet.

Les classes

La classe est un modèle que l'on suit pour la création d'objets. Les objets ayant les mêmes caractéristiques et le même comportement sont groupés dans une même classe. Une classe définit les données utilisées pour représenter les objets et les opérations qui peuvent être réalisées sur les données. Une fois qu'une classe a été créée, on peut déclarer un nombre quelconque d'objets dans cette classe.

Tous les objets d'une classe possèdent les mêmes données et ont les mêmes méthodes. Par analogie, une fois que l'on possède un négatif, on peut en réaliser un nombre quelconque de photos.

Les classes sont toujours définies dans les *headers files*. Elles y sont déclarées complètement avec leurs prototypes et leurs déclarations de données. Si ces déclarations sont bien documentées, il s'agira d'une excellente source de documentation.

1.3. Les avantages de l'OOP

Les principaux avantages de l'OOP par rapport à la programmation structurée sont :

- La facilité de maintenance
- La facilité de réutiliser le code
- L'extensibilité
- La modularisation
- La facilité de corriger les erreurs

1.3.1. La facilité de maintenance

L'arrangement hiérarchique des objets simplifie l'implémentation et la mise à jour des procédures. Le code pour chaque objet est relativement court parce que les méthodes de la plupart des objets sont des extensions de méthodes qui ont déjà été écrites.

L'OOP simplifie le développement des systèmes complexes en les associant au monde réel.

1.3.2. La facilité de réutiliser le code

Il est possible pour les programmeurs de développer des objets simples et bien définis. Ils peuvent alors les réutiliser dans de nouvelles applications avec, très souvent, très peu de modification.

En programmation structurée, on retrouve cette opportunité par l'utilisation de librairies. Cela demande cependant beaucoup plus de discipline et d'organisation de la part des programmeurs.

1.3.3. L'extensibilité

L'OOP permet aux classes existantes d'être facilement étendues par l'apport de nouvelles caractéristiques ou de nouvelles fonctionnalités. Cela peut être réalisé sans pour autant devoir dupliquer le code.

Par exemple, si un programmeur désire créer l'objet *porsche*, il pourrait le créer à partir de l'objet *voiture* déjà existant et y ajouter les caractéristiques et le comportement spécifiques à cette voiture.

1.3.4. La modularisation

La modularisation implique de décomposer un système en modules. On retrouve cette technique dans la programmation structurée : le système est décomposé en modules qui sont développés par des programmeurs différents. Cependant, les problèmes arrivent souvent au moment de l'intégration car les programmeurs n'ont pas fait attention et ont introduit des dépendances entre leurs modules ou n'ont pas respecté les spécifications.

La programmation objet est de par nature modulaire. Chaque classe représente à elle seule un module. Les programmeurs peuvent créer leurs propres classes et objets sans affecter ce qui existe. De plus, la visibilité de chaque classe ou objet peut être réduite au maximum.

1.3.5. La facilité de corriger les erreurs

Les objets contiennent non seulement les données mais aussi toutes les opérations qui vont les manipuler. Cela rend la détection des erreurs sur un objet plus facile puisque l'on sait où il faut chercher l'erreur.

2. Les concepts clés de l'OOP

2.1. L'abstraction

L'**abstraction** signifie ignorer les détails d'un objet qui ne sont pas nécessaires pour l'utilisation de cet objet. L'utilisateur d'un objet sait simplement ce que font les opérations mais il ne sait pas comment elles sont réalisées. L'utilisateur ne connaît rien sur la manière dont l'objet est implémenté.

2.1.1. Qu'est-ce qu'une abstraction ?

Avec une abstraction, les structures de données peuvent être utilisées sans que le programmeur n'ait à se préoccuper des détails exacts de l'implémentation.

En OOP, l'abstraction se réfère à la création d'un type de donnée, une classe, qui possède ses propres méthodes. Une classe est considérée comme une abstraction parce que l'utilisateur de la classe ne connaît pas comment les méthodes sont réalisées. Le programme extérieur passe simplement un message à la classe et le reste est pris en charge par la méthode.

La notion d'abstraction n'est pas propre à l'OOP. Par exemple, quand on utilise des nombres en virgule flottante, on ne se préoccupe pas de la représentation binaire de ce nombre. La connaissance de la représentation interne n'est pas utile pour pouvoir utiliser les nombres en virgule flottante.

La vie de tous les jours est remplie d'abstractions : beaucoup de personnes conduisent leur voiture sans avoir la moindre connaissance en mécanique.

2.1.2. Comment implémenter une abstraction ?

Les données d'un objet vont être rendues cachées aux programmes extérieurs : aucun d'eux ne pourra y accéder directement. La seule manière d'avoir accès aux données de l'objet sera de passer par les opérations définies sur l'objet. Les opérations servent donc d'interface entre l'objet et le monde extérieur.

Par exemple, pour initialiser un objet, on devra utiliser une opération à laquelle on communiquera différentes données qui seront validées par l'opération et qui les mémorisera dans l'objet.

Il est possible de cacher les données mais aussi de cacher certaines opérations plus techniques qui ne sont utilisées que par les seules opérations visibles dans l'interface.

Avantages :

1. Si la représentation interne de l'objet change, seules les méthodes de la classe vont devoir être modifiées.

2. S'il y a une erreur dans la manipulation des données de l'objet, seules les opérations d'accès à l'objet vont devoir être examinées.

2.2. L'héritage

2.2.1. Les hiérarchies d'objets

Avant d'aborder une des notions les plus importantes de l'OOP, l'**héritage**, il est très utile d'expliquer les différentes relations qui peuvent exister entre des objets.

2.2.1.A. La généralisation

Les classes peuvent être organisées selon la relation de **généralisation** ou relation **IS-A** (est un). Cette relation sera à la base de l'héritage. La généralisation est utile pour classifier les objets en fonction de leurs points communs et de leur spécificité. On peut la définir comme suit :

> La généralisation est une fonction, qui, à une classe d'origine, dite **sous-classe**, fait correspondre une classe plus générale, dite **super-classe**.

La fonction inverse de la généralisation, qui fait correspondre à une classe toutes les sous-classes dont elle est la super-classe, est appelée **spécialisation**. Elle permet d'ajouter des propriétés spécifiques à une classe pour obtenir une sous-classe.

Représentation graphique :

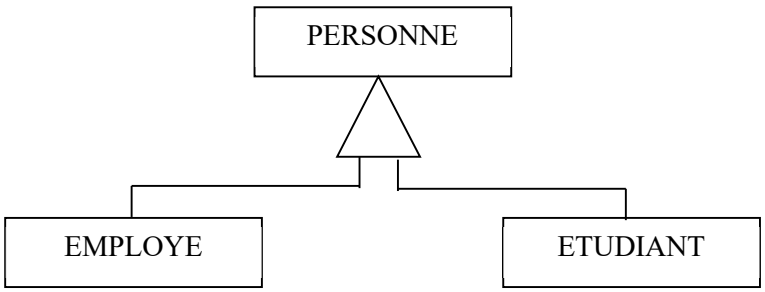


Figure 1 : un graphe de généralisation

Cette relation est transitive : une super-classe d'une classe est aussi super-classe des sous-classes de cette classe.

2.2.1.B. L'association

L'**association** est un concept essentiel pour la modélisation des données. Transposé dans le monde des objets, l'association permet de décrire au niveau des classes les liens connectant les objets.

> L'association est une relation entre plusieurs classes, caractérisée par un verbe, décrivant conceptuellement les liens entre les objets de ces classes.

Les associations binaires, les plus courantes, doivent pouvoir être parcourues dans les deux sens : dans le sens suggéré par le verbe (un employé est **affecté-à** un site) et dans le sens inverse (un site affecte des employés).

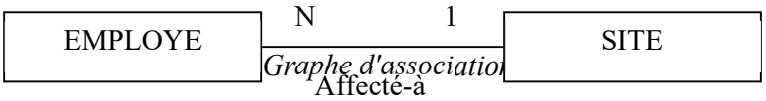
Une caractéristique importante d'une association est sa cardinalité qui permet de contraindre le nombre d'objets maximum connectés à chaque objet. Pour une association binaire entre les classes C1 et C2, les cardinalités peuvent être les suivantes :

- **1-1** : chaque objet de C1 est connecté à au plus un objet de C2, et chaque objet de C2 est connecté à au plus un objet de C1.

- **1-N** : chaque objet de C1 est connecté à zéro, un ou plusieurs objets de C2, et chaque objet de C2 est connecté à au plus un objet de C1.

- **N-M** : chaque objet de C1 est connecté à plusieurs objets de C2, et chaque objet de C2 est connecté à plusieurs objets de C1.

L'association binaire est représentée graphiquement comme suit :



Graphe d'association

2.2.1.C. L'agrégation

Un cas particulier d'association est l'**agrégation** qui représente la relation **partie-de** (has-a). Cette association est utile pour modéliser les objets complexes, typiquement composés d'autres objets.

> L'agrégation est une relation entre deux classes spécifiant que des objets d'une classe (la classe cible) sont les composants de l'autre classe (la classe source).

Une propriété importante de l'agrégation est la transitivité. Si C1 est partie de C2, et C2 partie de C3, alors C1 est partie de C3.

Représentation graphique de l'agrégation :

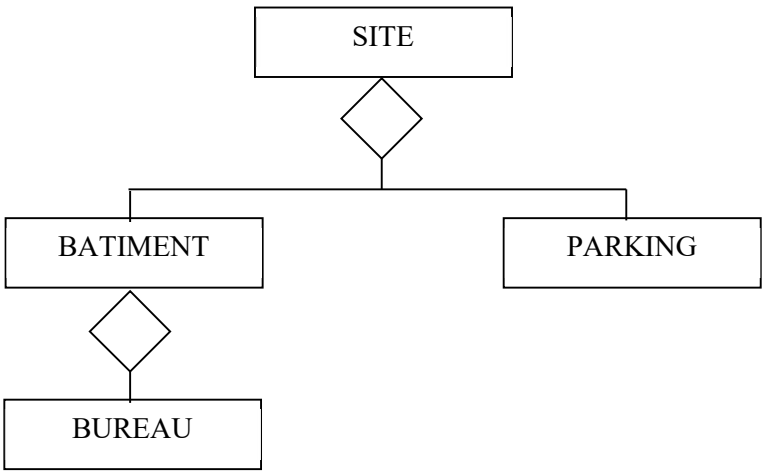


Figure 3 : Graphe d'agrégation

2.2.2. L'héritage

L'héritage est un des apports essentiels des langages à objets. Il peut être simple ou multiple et permet de supporter le polymorphisme des opérations. L'héritage n'est que la mise en œuvre d'une relation de généralisation entre deux classes d'objets.

L'avantage principal de l'héritage est qu'il va favoriser la réutilisation du code. En effet, les attributs et les opérations d'une classe sont réutilisables directement par les sous-classes. Avec l'héritage multiple, la redondance de code peut être grandement réduite.

2.2.2.A. Définition

L'organisation des classes selon la relation de généralisation permet de regrouper les propriétés communes au sein d'une même classe sans duplication dans d'autres classes. Afin d'exploiter le graphe des classes, le mécanisme d'héritage permet à une sous-classe d'hériter des propriétés de toutes ses super-classes :

> L'**Héritage** est un mécanisme de transmission des propriétés d'une classe vers une sous-classe.

Si la classe C2 hérite de la classe C1 (C2 est une sous-classe de C1), alors C2 possède toutes les propriétés de C1. Par exemple, un *employé* peut être classé dans l'une des catégories suivantes : *secrétaire, ingénieur et chercheur*. La classe *secrétaire* hérite donc de toutes les propriétés d'un *employé* comme le **nom** et l'**affectation**. Un objet de la classe *secrétaire* est un aussi un objet de la classe *employé*.

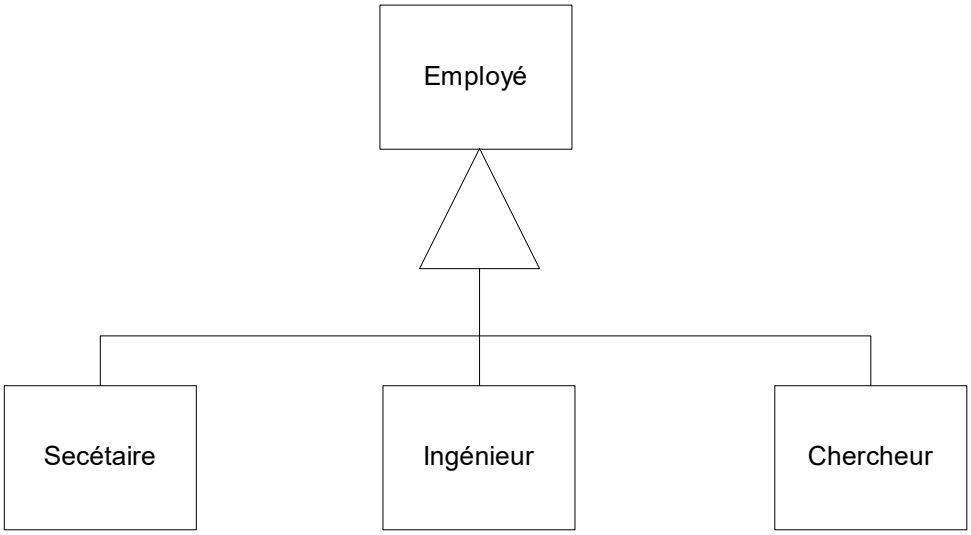


Figure 4 : Graphe de l'Héritage

Une instance de C2 peut aussi posséder des propriétés propres qui reflètent naturellement sa spécificité. La figure suivante montre la spécialisation de la classe *employé* en *ingénieur* et *chercheur*. Les propriétés **affectation** et **prime** sont définies dans employé et héritées par *ingénieur* et *chercheur*. En d'autres termes, l'opération qui donne l'**affectation** ou qui calcule la **prime** d'un employé est la même, qu'il soit ingénieur ou chercheur.

Cependant, les propriétés **projet** et **brevets** sont propres aux classes *ingénieur* et *chercheur* respectivement.

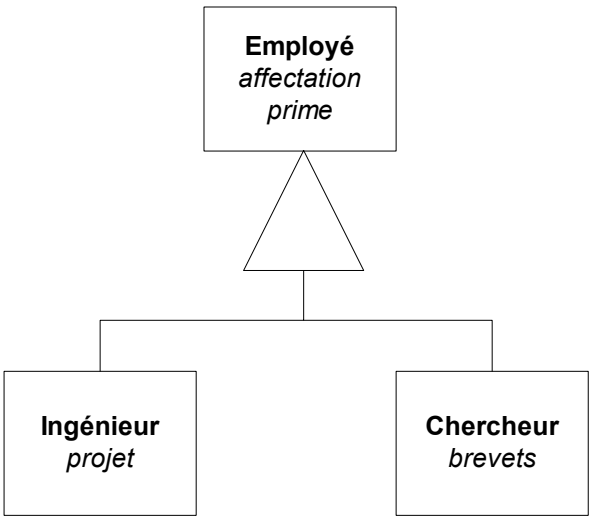


Figure 5 : Héritage et attributs propres

Synonymes possibles :

- À la place de **super-classe**, on pourra aussi parler de **classe de base**
- À la place de **sous-classe**, on pourra aussi parler de **classe dérivée**.

2.2.2.B. L'héritage multiple

Lorsque toute classe ne peut posséder qu'une super-classe directe, le graphe se réduit à un arbre et l'héritage est dit **simple**. Cependant, lorsque des classes indépendantes ont les mêmes propriétés, celles-ci doivent être dupliquées. Par exemple, supposons qu'un étudiant puisse être employé comme stagiaire et, dans ce cas, considéré comme employé chercheur. Avec l'héritage simple, pour représenter la propriété *durée de stage*, il faut l'ajouter à la fois dans les classes *chercheur* et *étudiant* ou créer deux sous-classes de ces classes.

L'**Héritage multiple** résout ce problème en permettant à une classe de posséder plusieurs super-classes directes et d'hériter des propriétés de toutes ses super-classes :

> L'**Héritage multiple** est un mécanisme par lequel une sous-classe hérite des propriétés de plus d'une super-classe.

La Figure 6 illustre la classe stagiaire qui modélise bien le fait qu'un stagiaire est à la fois un chercheur et un étudiant.

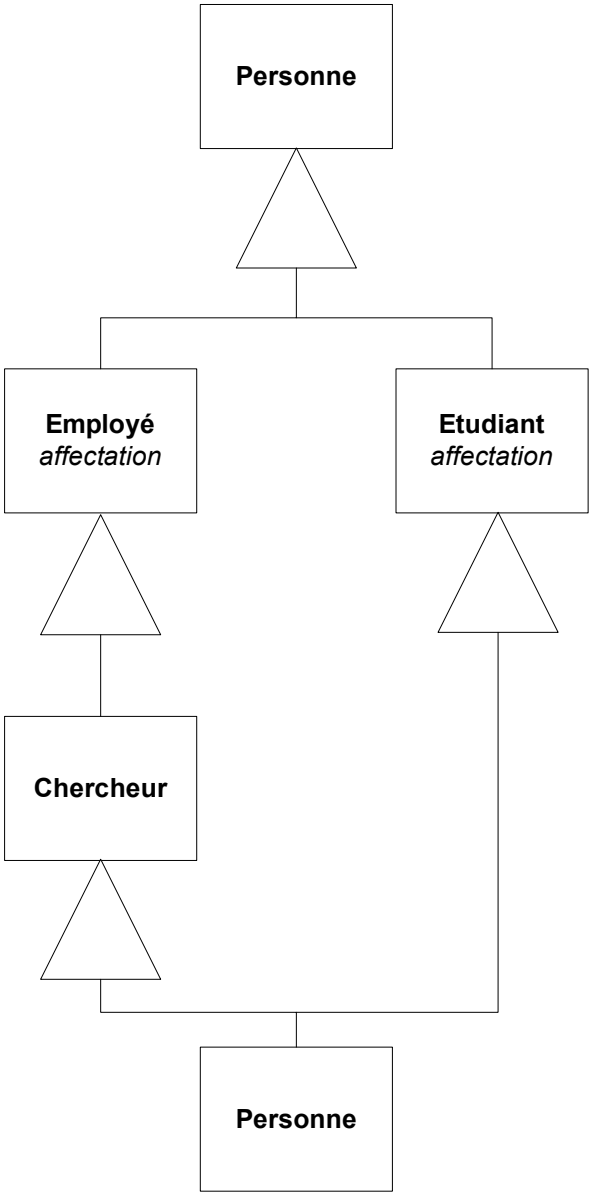


Figure 6 : Graphe de l'Héritage multiple

2.2.3. Le polymorphisme

Il est souvent nécessaire de redéfinir une opération héritée pour pouvoir lui donner une implémentation différente (un code différent) qui tienne compte des propriétés locales (non héritées) de la classe. L'opération redéfinie a la même signature que l'opération héritée ; on dira que l'opération ainsi redéfinie **substitue** l'opération définie dans la classe de base.

> La **substitution** est une redéfinition d'une opération héritée avec un code différent mais avec une interface identique. Au contraire de la *surcharge*, abordée plus loin, qui consiste à redéfinir une même opération mais avec des paramètres différents.

Une opération redéfinie portera le nom d'opération **virtuelle.**

Le **polymorphisme** est la faculté d'une opération à pouvoir s'appliquer à des objets de classes différentes.

La Figure 7 illustre une opération virtuelle.

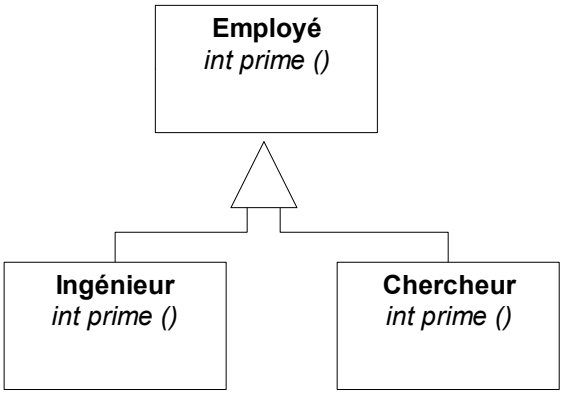


Figure 7 : Opération virtuelle

L'opération *prime* définie dans la classe *Employé* a été substituée dans les classes *Ingénieur* et *Chercheur* pour prendre en compte les attributs propres (par exemple, importance et degré de succès du projet) dans le code de l'opération.

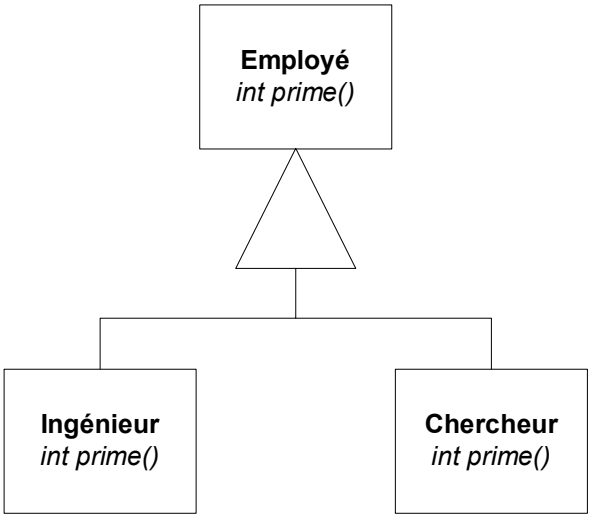


Figure 8 : Opération virtuelle

Le calcul de la surface du cube va faire intervenir les données *longueur* et *surface* définies au niveau du CRectangle mais fera aussi intervenir la notion de *hauteur*. L'opération surface est donc une opération **virtuelle**.

Grâce à la **liaison dynamique**, la sélection du corps de l'opération peut être faite par le système à l'exécution.

> La **liaison dynamique** est un mécanisme de sélection d'une opération à l'exécution en fonction de la classe d'appartenance de l'objet receveur.

Quand une opération est associée à l'objet à la phase de compilation, alors on parlera d'édition des liens **statique**. Quand une opération est associée à l'objet sur lequel elle porte au moment de l'exécution, on parlera d'édition des liens **dynamique.**

3. Les différences du C++ par rapport au C

Comme langage de programmation orienté objet, nous utiliserons dans ce cours le langage C++. Ce langage a été normalisé en juillet 1998.

Dans ce chapitre, on pourra trouver la liste des quelques différences existantes entre le langage C++ et le langage C.

3.1. Les déclarations de fonction

Tandis que le C ne définit pas un format spécifique pour la fonction *main ()*, dans le C++, le *main ()*, possède une valeur de retour. C++ définit explicitement la fonction *main ()* comme devant correspondre à un des deux prototypes suivants :

- `int main () ;`
- `int main (int argc , char **argv) ;`

En C++, toute fonction utilisée dans un fichier source doit obligatoirement avoir fait l'objet :

- soit d'une déclaration sous la forme d'un prototype
- soit d'une définition préalable au sein du même fichier source

En C, une fonction pouvait ne pas être déclarée (auquel cas, on considérait que la valeur de retour était par défaut *int*).

3.2. Les commentaires

Le C++ accepte deux types de commentaire. En plus du commentaire classique déjà présent dans le C délimité par /* …. */, C++ propose le commentaire introduit par //. Tous les mots de la ligne qui suivent ce double slashs sont considérés comme du commentaire.

Par exemple :

```
int  main ()
        {

        ….. // voici une ligne de commentaire

        }
```

Les programmes C++ contiennent en général un mélange des deux types de commentaire : le commentaire /* … */ est utilisé pour les blocs de commentaire que l'on place avant un bloc de traitement spécifique. Tandis que le // est utilisé pour les commentaires qui se placent sur une ligne.

3.3. De nouveaux mots réservés

Voici une liste de nouveaux mots réservés spécifiques au langage C++ :

class	inline	private	template
delete	new	protected	this
friend	operator	public	virtual

3.4. De nouveaux opérateurs

Le C++ définit l'opérateur **::** qui est un opérateur de portée. Il est utilisé pour accéder à des éléments cachés dans la portée courante.

Par exemple

```
int dA ;

int main ()
      {
      int dA = 3 ;

      printf ("local dA = %d\n" , dA);

      printf ("global dA = %d", ::dA);

      }
```

l'output du programme fournira :

```
local dA = 3
global dA = 1
```

L'opérateur de portée permet de retrouver la valeur globale alors que celle-ci est cachée dans le programme par la portée de la variable locale.

L'opérateur de portée est aussi utilisé pour définir une méthode qui appartient à une classe donnée. Par exemple :

```
Animal::Nombrepatte ()
```

Cette déclaration signifie que la méthode *NombrePatte* appartient à la classe *Animal*.

3.5. La structure

En C++, le type **struct** diffère de celui qui existe en C.
Prenons pour exemple le morceau de code suivant en C :

```
struct eg  {
        int dA;
        float fB;
        };

struct eg stX;
```

La dernière déclaration déclare la variable **stX** comme étant du type **struct eg**.

En C++, on écrirait ce qui suit :

```
struct eg  {
        int dA;
        float fB;
        };

eg stX;
```

Ici, le mot clé **struct** n'est plus nécessaire pour la déclaration de la variable **stX**.

En C++, on peut aussi associer des fonctions membres à une structure. Dans ce cas, toutefois, il n'est pas possible de réaliser des sections : tous les membres seront obligatoirement de type *public*.

3.6. Le mot clé const

Une variable déclarée de type *const* est une variable dont le contenu ne peut pas être modifié. Le mot clé *const* ne peut être utilisé que dans une déclaration et c'est une erreur de ne pas initialiser une telle variable. Par exemple :

```
const int dY;          // provoquera une erreur car la variable n'est pas initialisée
```

L'emploi du mot-clé *const* est une bien meilleure technique que l'emploi de la macro *#define*. En effet, avec une variable de type *const*, le compilateur va pouvoir réaliser des contrôles de cohérence au niveau du type et il pourra réaliser de l'optimalisation du code. Il s'agit de deux choses qui ne peuvent pas être réalisées avec le *#define.*

Plusieurs combinaisons sont possibles :

- Variable *const*
- Pointeur *const*
- Pointeur *const* vers une variable non *const*
- Pointeur *const* vers une variable *const*

Voici quelques exemples :

```
int dVal;

int *pVal;                          // variable et pointeur non const

int * const pcVal = &dVal;          // pointeur const vers une variable de type int

const int dcVal = 7;                // variable int const

const int *cpVal;                   // pointeur vers un int const

const int * const cpcVal = &dcVal;  // pointeur const vers une variable int const
```

On peut utiliser une variable de type const pour déclarer un tableau :

```
const int dcTaille = 20;

char sMsg [dcTaille];
```

Le C++ considère la définition d'une variable globale de type **const** comme étant **static**. En C, au moyen d'une clause *extern*, on pouvait utiliser une variable globale de type *const* dans d'autres fichiers sources. Les deux déclarations suivantes sont donc équivalentes :

```
1. const int dcNombre = 50;     // variable globale

2. static const int dcNombre = 50;
```

Cette possibilité permet d'introduire de telle déclaration dans les fichiers entête () et de les inclure dans plusieurs fichiers sources sans qu'il n'y ait jamais de double déclaration. N'oublions pas que les fichiers sont compilés séparément et qu'une variable static n'est connue que du compilateur et pas de l'éditeur des liens. Si la variable n'était pas considérée comme étant du type const, alors l'éditeur des liens détecterait une double déclaration de la variable à partir du moment où le fichier d'entête serait inclus dans deux fichiers sources.

En C++, on peut aussi utiliser des fonctions membres qui sont de type *const*. Cela indique des fonctions qui ne peuvent pas modifier les variables membres de leur classe. La syntaxe est la suivante :

```
type  NomFonction (paramètres) const
```

En C++, un identifiant global déclaré avec le qualificatif *const* peut être utilisé dans une expression constante alors qu'il ne pouvait pas l'être en C ; ce point permet notamment, d'utiliser de tels symboles pour définir la taille d'un tableau (en C, il fallait obligatoirement avoir recours à une définition d'identifiants par la macro *#define*).

Attention. On ne peut pas invoquer des fonctions non const sur des objets const ! En effet, la fonction pourrait être amenée à modifier l'objet.

3.7. L'allocation dynamique de mémoire

En C, l'allocation dynamique de mémoire est réalisée au moyen de fonctions telles que *malloc()* et *free()*. Quand à lui, le C++ définit une nouvelle méthode d'allocation basée sur l'emploi d'opérateurs en lieu et place de fonctions.

Les opérateurs **new** et **delete** exécutent le même travail que *malloc()* et *free()*. L'opérateur *delete* présente malgré tout un avantage sur la fonction *free()* : il ne génère pas d'erreur si on lui passe la valeur zéro comme opérande.

L'emploi de l'opérateur **new** pour allouer dynamiquement un objet permettra d'invoquer le constructeur par défaut de cet objet (les notions d'objet et de constructeur seront abordées par la suite). L'emploi de l'opérateur **delete** pour détruire un objet permettra d'invoquer automatiquement le destructeur de l'objet.

Par exemple :

```
void f ()
     {
     int *pPtr;

     pPtr = malloc (sizeof(int));

     free (pPtr);
     }
```

est équivalent en C++ à :

```
void f ()
     {
     int *pPtr;

     pPtr = new int;

     delete pPtr;

     }
```

Quelques exemples

- char *pPtr = **new** char [5]; // équivalent à :
  // malloc (5 * sizeof(char));
- CPoint *pPtr = **new** CPoint; // utilisation du constructeur par
  // défaut de la classe CPoint
- CPoint *pPtr = **new** CPoint (1.0 , 2.6 , 3.9); // utilisation du constructeur
  // acceptant trois paramètres de type
  // float
- CPoint *pPtr = **new** CPoint [50]; // il retourne zéro s'il ne peut pas
  // trouver la place nécessaire

En définissant des opérateurs plutôt que des fonctions pour réaliser l'allocation de mémoire, le C++ réduit l'overhead produit par un appel de fonction et offre par conséquent une gestion de la mémoire plus rapide.

La suppression de l'espace alloué se fera au moyen de l'opérateur **delete**. Voici, pour les exemples repris ci-dessus comment il faudrait s'y prendre :

- **delete** [ ] pPtr; // lorsqu'il s'agit de l'adresse d'un tableau, l'emploi des [ ] est
  // obligatoire

- **delete** pPtr; // lorsqu'il s'agit de l'adresse d'une donnée simple

Attention

Il n'est pas possible de mélanger l'emploi des fonctions d'allocation avec les opérateurs d'allocation. On ne peut pas allouer une zone mémoire au moyen de la fonction *malloc()* et libérer cet espace au moyen de l'opérateur *delete*.

En outre, si on souhaite redimensionner l'espace alloué, alors, seules les fonctions d'allocation pourront être utilisées pour résoudre ce problème.

[extract from MSDN] : When **new** is used to allocate a multidimensional array of objects, it yields a pointer to the first element of the array, and the resultant type preserves the size of all but the leftmost array dimension.

For example:

```
new float[10][25][10]
```

yields type float (*)[25][10]. Therefore, the following code will not work because it attempts to assign a pointer to an array of float with the dimensions [25][10] to a pointer to type float:

```
float *pF;
pF = new float[10][25][10];
```

The correct expression is:

```
float (*pTab)[25][10];
pTab = new float[10][25][10];
```

pTab pointe vers un tableau à deux dimensions de taille [25][10]. Donc, nous avons alloué un tableau de 10 éléments. Chaque élément étant lui-même un tableau de type float à deux dimensions : [25][10].

3.7.1. Les fuites de mémoire

Une fuite de mémoire se produit quand le programme alloue de la mémoire sur le segment et qu'il exécute une action qui fait perdre ou qui modifie le pointeur vers la mémoire allouée, perdant ainsi la connexion avec le bloc de mémoire.

Les causes courantes des fuites de mémoire sont notamment les suivantes :

- **Appel de *new* sans jamais appeler *delete***. Si *delete* n'est jamais appelé, la variable sort simplement de la portée et le pointeur est perdu.

- **Versions non concordantes de *new* et *delete***. Les tableaux créés avec *new* nécessitent l'opérateur *delete []*, pas seulement *delete*.

- **Oubli de déférencer proprement un pointeur**. Le fait de modifier l'adresse d'un pointeur plutôt que de modifier les données vers lesquelles il pointe va faire perdre l'accès aux données.

- **Survenance d'une exception avant d'atteindre le *delete***. Les exceptions avant un *delete* créent une fuite de mémoire, même si l'excpeiotn est gérée automatiquement.

3.8. Les références

Pour le passage des arguments à une fonction, le C++ offre une alternative au passage par valeur : le passage par **référence**.

A partir d'un exemple, voyons les différences entre le passage par valeur et le passage par référence :

```
void swap (int *pA , int *pB)
     {
     int dTemp ;

     dTemp = *pA ;
     *pA = *pB ;
     *pB = dTemp ;
     }
```

L'appel à cette fonction va se réaliser comme suit : swap (**&dVal1 , &dVal2**)

L'inconvénient de cette méthode est qu'il faut toujours utiliser l'opérateur * à chaque fois que l'on veut accéder à une valeur pointée. Pour éviter cet inconvénient, le C++ offre le mécanisme de **référence**.

> Une **référence** peut se définir comme étant un synonyme que l'on donne à une variable précédemment définie. La définition d'une référence se réalise au moyen de l'opérateur **&**.

Par exemple :

```
int dV1 ;

int& dV2 = dV1 ; // dV2 devient un synonyme, un alias de la
                 // variable dV1
```

Les identifiants **dV1** et **dV2** référencent la même valeur, au même emplacement mémoire. On peut utiliser indifféremment l'une comme l'autre. Dans ce contexte, **&** n'est pas l'opérateur *calcul d'adresse*. Il signifie ici *référence vers un int* un peu comme **int *** signifie *pointeur vers un int*.

La référence ne peut être initialisée que lors de sa déclaration. Une fois que l'association entre une référence et une variable a été créée, il n'est plus possible de la changer.

La principale utilisation des références est dans le passage d'arguments à une fonction. En utilisant une référence comme argument, la fonction travaille avec la donnée originale à la place d'une copie. Cette méthode de passage de paramètre est connue sous le nom de **passage par référence.**

En reprenant l'exemple ci-dessus, on aurait en C++ ce qui suit :

```
void swap (int& dA , int& dB)
     {
     int dTemp ;

     dTemp = dA ;
     dA = dB ;
```

```
        dB = dTemp ;
        }
```

L'appel de la fonction se faisant comme suit : `swap (oR1 , oR2) ;`

La référence est une alternative intéressante à l'utilisation de pointeurs pour manipuler de grandes structures de données. Mais attention. Si une référence ressemble à un pointeur, elle n'en est pas un ! La référence ne peut pas s'utiliser pour réaliser une allocation dynamique de mémoire !

Il est également possible de communiquer une référence comme valeur de retour d'une fonction. Mais attention à ne pas retourner la référence d'une variable locale à la fonction !

```
type& NonFonction (paramètre) ;
```

En C++, il sera très fréquent de trouver des prototypes de fonction qui ressemblent à ceci :

```
TypeRetour f (const type& paramètre) ;
                    // le paramètre ne pourra pas être
modifié
                    // par la fonction
```

ou

```
TypeRetour f (type& paramètre) ;
                    // la fonction pourra modifier son
                    // paramètre
```

```
// Functions that take reference arguments have access
// to the callers data.
#include <iostream>
                                        // CHANGE #1
                                        // function prototype
void swap(int&, int&);                  // reference to integer

void main()                             // Identical to swap.cpp
{                                       // two local variables
    int nX(5), nY(10);                  // Note: equivalent to:
                                        // int nX=5, nY=10;
    cout << "X is " << nX
         << " and Y is " << nY << endl;
    swap (nX, nY);                      // function call
    cout << "X is " << nX
         << " and Y is " << nY << endl;
}
                                        // CHANGE #2 Now takes
                                        // references as arguments
void swap(int &nA, int &nB)
{
    int nTemp;
                                        // same as original SWAP!
    nTemp = nA;
    nA = nB;
    nB = nTemp;
}
```

Exemple 1 : L'utilisation des références dans la fonction Swap

Il est possible d'appliquer l'opérateur de calcul d'adresse à une référence ; cela revient à calculer l'adresse de la variable référencée. Voici un exemple :

```
void AFXAPI DDX_FieldCBString(CDataExchange* pDX, int nIDC, CString& value,
       CRecordset* pRecordset)
{
       ASSERT_VALID(pRecordset);

       HWND hWndCtrl = pDX->PrepareCtrl(nIDC);
       if (pDX->m_bSaveAndValidate)
       {
              // just get current edit item text (or drop list static)
              int nLen = ::GetWindowTextLength(hWndCtrl);
              if (nLen != -1)
              {
                     CString strTemp;
                     ::GetWindowText(hWndCtrl, strTemp.GetBuffer(nLen),
                                   nLen+1);
                     strTemp.ReleaseBuffer();
                     nLen = strTemp.GetLength();
                     if (nLen > value.GetAllocLength())
                            AfxFailMaxChars(pDX, value.GetAllocLength());
                     // get known length
                     ::GetWindowText(hWndCtrl, value.GetBuffer(0), nLen+1);
              }
              else
              {
                     // for drop lists GetWindowTextLength does not work -
                     // assume
                     //  preallocated length
                     ::GetWindowText(hWndCtrl, value.GetBuffer(0),
                                   value.GetAllocLength()+1);
              }
              value.ReleaseBuffer();
              if (value.GetLength() == 0)
              {
                     if (pRecordset->IsFieldNullable(&value))
                            pRecordset->SetFieldNull(&value, TRUE);
              }
              else
              {
                     pRecordset->SetFieldNull(&value, FALSE);
              }
       }
       else
       {
              if (!pRecordset->IsOpen() || pRecordset->IsFieldNull(&value))
              {
                     SendMessage(hWndCtrl, CB_SETCURSEL, (WPARAM)-1, 0L);
              }
```

3.9. Le cast

Le C++ présente deux manières différentes de coder l'opérateur *cast* :

- (type) expression
- type (expression)

3.10. 0 et NULL

0 est une valeur spéciale en C++ : elle ne possède pas de type. N'importe quelle variable peut être confrontée à la valeur **0**.

On peut obtenir ce qui suit :

```
char *pChar = 0 ;       // (void *) 0

float fFloat = 0 ;      // 0.0

int dInt = 0 ;          // 0

char cChar = 0 ;        // '\0'
```

3.11. Les pointeurs génériques

En C++, un pointeur générique (*void* *) ne peut pas être converti implicitement lors d'une affectation en un pointeur d'un autre type ; la chose était permise en C. Bien entendu, en C++, il reste possible de faire appel à l'opérateur *cast*.

4. Les entrées-sorties en C++

4.1. Les objets de type fichier prédéfinis

Les E/S vont se gérer en C++ au moyen de nouvelles classes, prédéfinies, et de deux nouveaux opérateurs :

- L'opérateur d'extraction **>>**, utilisé pour la lecture
- L'opérateur d'Insertion **<<,** utilisé pour l'écriture

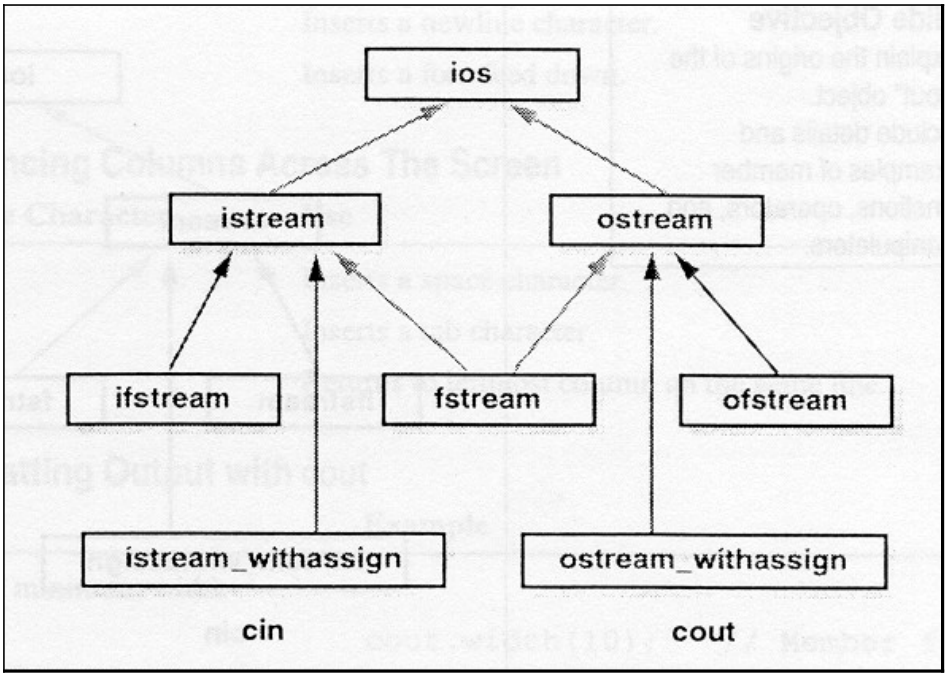


Figure 9 : Hiérarchie de classes pour les E/S

Cette hiérarchie de classes est définie dans le fichier *iostream* qui contient en outre la définition de quatre objets globaux :

- L'objet **cin** qui permet de lire des données à partir de l'entrée standard, le clavier, au moyen de l'opérateur d'extraction >>.

- L'objet **cout** qui permet d'écrire des données sur la sortie standard, l'écran, au moyen de l'opérateur d'insertion <<.

- L'objet **cerr** qui correspond au fichier standard pour les erreurs qui est aussi associé au terminal. Par défaut, il n'y a pas de buffer pour ce fichier. L'information est de suite envoyée dans le fichier, donc sur l'écran.

- L'objet **clog** utilisé pour les sorties avec buffer : au contraire de **cerr** qui travaille sans buffer.

L'opérateur d'extraction >> fait correspondre les données du fichier avec les variables fournies et retourne une référence vers le fichier. Cette référence est indispensable pour pouvoir coder plusieurs opérateurs les uns à la suite des autres :

```
cin >> dA >> dB >> dC ;
```

La première valeur introduite sera assignée à la variable *dA*, la deuxième à *dB* et ainsi de suite.

Un bit d'erreur est positionné lorsqu'une erreur est détectée à la lecture des données. Par exemple, si avec l'instruction de lecture précédente, on essaie d'introduire des données non numériques. Le bit d'erreur peut être remis à zéro au moyen de l'opération **cin.clear**.

Attention : le >> va commencer par sauter les blancs et ensuite, va lire jusqu'à rencontrer le blanc suivant.

Avec les objets destinés à l'écriture, voici une liste de fonctions, de manipulateurs ou encore de caractères de contrôle :

Manipulateur	Description
endl	Insère un caractère de nouvelle ligne et vide le buffer
ends	Insère un caractère null de terminaison
flush	Vide le buffer

Fonctions membres	Description
put	Insère un caractère
write	Insère un nombre spécifié de caractères d'un buffer vers le fichier
tellp	Donne la position courante d'écriture du fichier
seekp	Change la position courante d'écriture du fichier

Avancement de lignes sur l'écran	Description
'\n'	Insère un caractère de nouvelle ligne
'\f'	Insère un saut d'écran (formfeed down)

Avancement de colonnes sur l'écran	Description
' '	Insère un caractère blanc
'\t'	Insère un caractère de tabulation
'\r'	Retourne à la colonne la plus à gauche de la même ligne

Suppression des blancs à la lecture	Description

Skipws **Noskipws**	Ne lit pas les caractères blancs, les carriage returns, … (valeur par défaut) Ne saute pas les carcatères blancs qui se trouvent en début

4.2. Quelques exemples simples

Voici quelques exemples simples qui montrent la différence entre le C et le C++ :

Style C	Style C++
`puts ("Hello");`	`cout << "Hello" << endl;`
`printf ("%d\n" , dQuant);`	`cout << dQuant << endl;`
`gets (sNom);`	`cin >> sNom;`
`gets (sNom);` `scanf ("%d", dAge);`	`cin >> sNom >> dAge;`

4.3. Définition de nouveaux objets fichiers

Le programmeur peut définir ses propres fichiers objets ; ceux-ci seront déclarés dans une des classes prédéfinies et héritera par conséquent des fonctions membres et de tous les opérateurs qui y sont définis.

La classe **fstream** est spécialement prévue pour les fichiers utilisés en input et en output.

- La fonction membre **is_open ()** vérifie si oui ou non le fichier est ouvert.
- L'opérateur **NOT (!)**, est un opérateur surchargé dans la classe **ios**. Cet opérateur retourne une valeur différente de zéro si une erreur sur le fichier s'est produite.

On peut consulter l'exemple (*voir Exemple 2 : Utilisation ofstream et fstream, page 37*) pour trouver la déclaration de nouveaux objets fichiers.

```
#include <fstream>     // For file stream support
#include <ctype>       // For character macro
#include <iostream>

void FileCreate();       // prototype file functions
void FileUpdate();
void FileWrite();

int main(void)
{
    FileCreate();        // Create ofstream & output strings.
    FileUpdate();        // Read and append to an fstream.
    FileWrite();         // Get from cin, put to an ofstream.
    return 0;
}

// Creates a file, test.txt, and outputs two lines.
// Create disk file: test.txt.  Note: the 2nd arg
// to the c'tor is the default: ios::out | ios::app

void FileCreate()
{
    ofstream outfile("test.txt");
    if (!outfile)                 // check for successful open
        cerr << "Cannot open 'test.txt' for output.\n";
        else outfile      << "This is test data.\n"
                          << "File will be closed at
                          destruction.\n";
    cout << "File created.\n";
}

// Read an input file, test.txt, getting a character
// at a time, appends the files content as all capital
// letters at the end of the original file.
// fstream inherits input & output

void FileUpdate()
{
    int iCount = 0;                // ::in input mode
    char data[100];                // ::app append additions
    fstream iofile("test.txt", ios::in | ios::app);

    if (!iofile)                   // error handling
        cerr << "Trouble opening file 'test.txt'.  "
                "Function 'FileUpdate()' failed.\n";

    while (!iofile.eof())          // While data exists, load
        iofile.get(data[iCount++]); // data by char

    iofile.clear();                // Clear eof error state

    iCount--;                      // Adjust for 'off by one'
    for (int j = 0; j < iCount; j++)
    {                             // Put uppercase char to file
       data[j] = (char) toupper(data[j]);
       iofile.put(data[j]);
       cout.put(data[j]);          // Echo characters to cout
    }
    cout << "File appended.\n";
}

// Take user keyboard input and write characters
// to a file "test.out".

void FileWrite()
{
    char ch;
```

```
    ofstream outfile("test.out", ios::out);
    if (!outfile)                  // Detect error opening file
        return;
    cout << "Enter characters. "
         << "(Use Ctrl-Z and ENTER to Quit)\n";
    while (cin.get(ch))            // While data exists,
        outfile.put(ch);           // put char to file.
    cin.clear();                   // Clear eof error for cin
}


// What would happen without the line above?
// What if main() was changed to "cin" more data?
```

Exemple 2 : Utilisation ofstream et fstream

4.4. Les fichiers en mode texte et en mode binaire

Par défaut, un fichier sera toujours ouvert en mode texte. L'ouverture d'un fichier peut se réaliser soit au moment de la construction de l'objet fichier, soit explicitement au moyen de la fonction *open*.

Voici les différents modes d'ouverture d'un fichier :

Option	Description
`ios : :app`	Insertion à la fin du fichier
`ios : :ate`	(pas clair : start at the end but allow new bytes to be written at eof or anywhere under program control).
`ios : :in`	Ouverture en input
`ios : :out`	Ouverture en output
`ios : :trunc`	(pas clair : discard contents if file already exists. Caution : truncate is implied with ios : :out when ios : :ate, ios : :app and ios : :in are not specified)
`ios : :nocreate`	L'opération d'ouverture échoue si le fichier n'existe pas
`ios : :noreplace`	(pas clair : fail the open if the file does not exist)
`ios : :binary`	Ouverture en mode binaire

On pourra trouver aux pages suivantes deux exemples de programmes utilisant des fichiers utilisés en mode texte et en mode binaire.

```
// The application reads text files by char, word, and
// line.  It duplicates the input file, creating a
// line-numbered file with the extension ".out".
#include <iostream>
#include <fstream>
#include <iomanip>
#include <stdlib>      // for exit()

#define SIZE 256

void main()
{
    int nCntChars, nCntWords, nCntLines;
    char data[SIZE], ch;

    // Create stream objects using constructors:

    ifstream infile("test.txt", ios::in);
    ofstream outfile("test.out", ios::out);
    if (!infile || !outfile)
    {
        cerr << "Error opening file(s)";
        exit(1);
    }

    /*********** 'char' pass thru input file **********/

    for (nCntChars = 0; infile.get(ch); ++nCntChars);

    cout << "Input file contained " << nCntChars
         << " characters, ";
                              // reset infile for 'word' pass
    infile.clear();               // reset eof state
    infile.seekg(0L, ios::beg);    // seek to 0-byte

    /*********** 'word' pass thru input file **********/

    nCntWords = 0;
    while (infile >> data)
        ++nCntWords;

    cout << nCntWords << " words, ";
                              // reset infile for 'line' pass
    infile.clear();           // reset eof
    infile.seekg(0L);         // seek (default ios::beg)

    /*********** 'line' pass thru input file **********/

    for (nCntLines = 1; infile.getline(data, SIZE);
        ++nCntLines)
    {
        outfile.width(3);             // set width for line #
        outfile << nCntLines << ". ";  // insert line #
        outfile << data << endl;       // insert line to file
    }
    cout << --nCntLines << " lines.\n";
    cout << "File-to-file number copy complete.\n\n";
    infile.close();   // close files (disconnect stream)
    outfile.close();  // or the d'tor will (good style!)

    cout << "***      Brain Teaser      ***\n";
    cout << "    get.(c) reports " << nCntChars
         << " chars.\n";
    cout << "getline.(*) reports " << nCntLines
         << " lines.\n";
    cout << "But, dir cmd shows: "
         << nCntChars + nCntLines << " size.\n";
    cout << "*** Q: Why the difference? ***\n";
```

```
}
```

Exemple 3 : Utilisation de fichiers en mode texte

```
// Uses block read and write operations to copy a
// file using binary mode.

#include <iostream>
#include <fstream>
#include <iomanip>
#include <stdlib>      // for exit()

#define SIZE 256

void main()
{
    char data[SIZE];     // set i/o buffer size
    int nCount = SIZE;   // bytes read, used for eof test
                         // Create stream objects:
    ifstream infile;     // - input file stream
    ofstream outfile;    // - output file stream
                  // fstream flags:
                  // ios::in  open for input (default)
                  // ios::nocreate if does not exist
                  // ios::binary mode (default is text)

    infile.open("test.txt", ios::in | ios::binary);
    outfile.open("test.out", ios::out | ios::binary);
                        // error checking
    if (!infile || !outfile)
    {
        cerr << "Error opening file(s)";
        exit(1);
    }

    // Loop reads up to 256 characters into data buffer.
    // Each read is size-verified to determine whether
    // end of file was reached.

    while (nCount == SIZE)
    {
        infile.read(data, SIZE);     // read up to SIZE
        nCount = infile.gcount();    // adj. actual size
        outfile.write(data, nCount); // update outfile
        if (outfile.rdstate() != ios::goodbit)
            cout << "Error writing file.\n";
    }

    cout << "File-to-file copy complete.\n";
    infile.close();  // close files (disconnect stream)
    outfile.close(); // or the D'tor will  (good style)
}
```

Exemple 4 : Utilisation de fichiers en mode binaire

5. Premiers programmes

5.1. Une simple lecture en DOS

Ce programme ne réalise qu'une seule opération : lecture et affichage de votre nom.

```
#include <iostream>
#include <string>

int main ()
{
	string Nom;

	cout << "Comment vous appelez-vous ?";
	cin >> Nom;
	cout << "Bonjour" << Nom << "!" << endl;

	return 0;
}
```

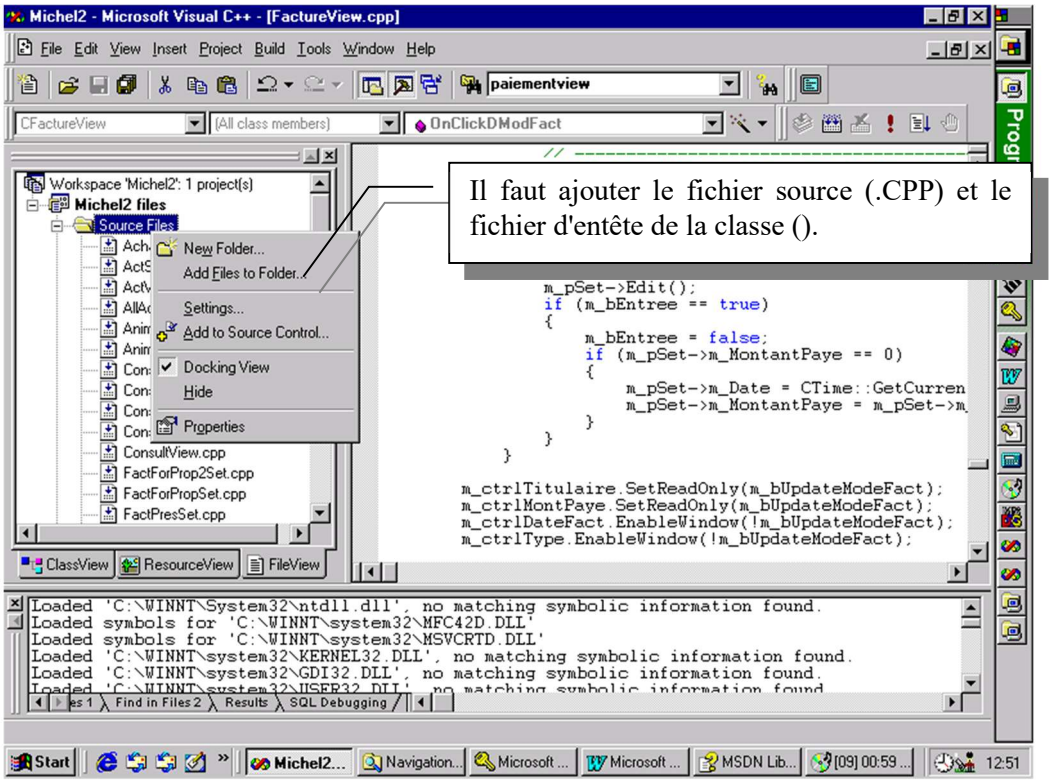
5.2. Création d'un programme Windows avec AppWizard

AppWizard est un outil générant un projet MFC en fonction des options que l'on sélectionne. *AppWizard* crée les fichiers sources servant de base pour le programme Windows. Il peut s'agir d'une application monodocument, multidocuments ou reposant sur des boîtes de dialogue.

5.3. Ajoute de classes existantes dans le projet

Voici la méthode la plus corretce pour ajouter une classe existante à l'intérieur d'un projet :

1. il faut ajouter le fichier Source de la classe et le fichier Hearder de la classe :



2. Cette classe qui vient d'être ajoutée peut être utilisée de deux manières :

   a) comme classe de base d'une nouvelle classe
   b) comme type pour la déclaration d'une nouvelle donnée membre

3. Si cette opération est réalisée avec le Wizzard, alors, ce dernier ajoutera automatiquement l'instruction *#Include* avec comme complément le path complet pour trouver le fichier.

6. La construction d'une classe

6.1. Introduction

Le C++ permet au programmeur de définir ses propres types de données. En C++, un type définit par un programmeur est connu sous le nom de **classe**. Une classe peut introduire un nouveau type de donnée ou permet d'ajouter de nouvelles fonctionnalités à un type déjà existant. La définition des classes se trouvera dans les **header files**.

Tout en donnant un exemple, voici la syntaxe de déclaration d'une classe :

```
class CRectangle
      {
      private                    :

      int m_dhauteur;
      int m_dLargeur;

      public :
      void Initlargeur (int);
      long Surface (void);
      };
```

Exemple 5 : Classe CRectangle

A partir de cet exemple, on peut essentiellement remarquer les aspects suivants :

- Le mot réservé class suivi par le nom de la classe. Le nom de classe dans l'environnement Microsoft est toujours préfixé par la lettre C majuscule.
-
- Les membres de la classe sont composés à la fois par des données et par des descriptions de fonctions. Techniquement, il est possible de trouver dans la définition de la classe, le contenu de chaque fonction. Cependant, par convention, on ne le fera pas ; le code des fonctions se trouvera en général en dehors de la définition de la classe (excepté pour certaines fonctions inline).
-
- Les notions de private et de public
-
- La syntaxe utilisée pour la définition de la classe est proche de celle utilisée en C pour la définition d'une structure.

6.2. Les attributs de classe

Comme on vient de le voir, une classe possède les attributs suivants :

- Un ensemble de **données membres** : il peut y en avoir un nombre quelconque, y compris zéro. Chaque donnée membre sera préfixée par **m_**.

- Un ensemble de fonctions membres qui représentent le comportement de la classe. Il s'agit des opérations que l'on pourra appliquer sur chacun des objets qui

appartiendront à la classe. Chaque classe peut contenir un nombre quelconque de fonctions membres, y compris zéro.

Considérons l'exemple de la classe **Complex** qui contient les membres suivants :

- Deux données membres : **lfReel , lfImag** qui représentent les parties réelles et imaginaires du nombre complexe.

- Quatre fonctions membres qui vont permettre de manipuler les objets de la classe *Complex* :

  - *Set* : initialise la valeur d'un nombre complexe
  - *Print* : imprime un nombre complexe
  - *Add* : additionne deux nombres complexes
  - *Sub* : soustrait deux nombres complexes

Voici la description de la classe *Complex* :

```
class CComplex
      {
      private :
      double m_lfReel , m_lfImag;

      public :
      CComplex Set (double r , double i);
      void Print (void);
      CComplex Add (CComplex c);
      CComplex Sub (CComplex c);
      };
```

Exemple 6 : La classe Complex

6.3. Les Spécificateurs d'accès

Un **spécificateur d'accès** contrôle l'accès que l'on peut faire aux différents membres d'une classe. Il y a trois types de spécificateurs d'accès :

- public
- private
- protected

Les membres se trouvant dans la section **public** d'une classe peuvent être accédés par toutes les autres classes du programme. Concrètement, toutes les fonctions membres de toutes les classes peuvent accéder aux membres **public**. Y compris bien sûr le programme principal.

Les membres se trouvant dans la section **private** de la classe peuvent seulement être accédés par la classe elle-même. Cela signifie que les membres **private** ne peuvent être accédés que par les fonctions membres de la classe et par personne d'autre.

Les membres se trouvant dans la section **protected** de la classe peuvent seulement être accédés par la classe et par ses classes dérivées. Concrètement, les membres **protected** ne sont accessibles que par les fonctions membres de la classe et par les fonctions membres de toutes les classes dérivées.

Etant donné ces différents modes d'accès, on établit la règle générale suivante :

- de toujours déclarer les fonctions membres dans la section **public**
- de toujours déclarer les données membres dans la section **private**
- de fournir des fonctions membres qui permettent de modifier ou de retrouver les données membres de la classe.

En respectant cette règle, on appellera une telle classe un **type de donnée abstrait.**

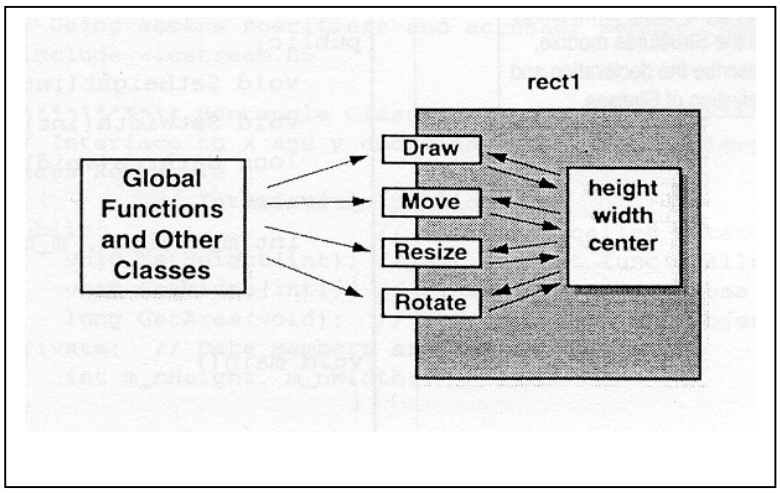


Figure 10 : Le principe du type de données abstrait

6.4. Déclaration d'un objet

Maintenant que la classe est définie, il va être possible de déclarer des objets. Un objet est un peu comme une variable que l'on déclare. La variable reçoit un type et en hérite toutes les propriétés. Si on déclare une variable de type *float*, cette variable hérite de toute une série de caractéristiques : place mémoire, codage binaire, opérations autorisées, limite maximale et minimale, …

Au niveau de l'objet, cela va être le même principe : on va déclarer un objet comme étant d'une classe donnée. Cela signifie que :

- de la place mémoire va être allouée pour toutes les données membre de la classe.
- les fonctions membres de la classe vont pouvoir être utilisées sur l'objet

Ainsi, si dans le programme principal, on désire déclarer un objet de la classe *CRectangle*, on écrira :

```
void main ()
      {
      CRectangle oRect;

      …
      }
```

Exemple 7 : Déclaration d'un objet

Pour les variables objet, on utilisera le préfixe **o**.

6.5. Invocation d'une fonction

Lorsqu'un objet a été déclaré, on peut lui appliquer toutes les opérations définies dans la classe. Pour ce faire, on va utiliser l'opérateur **point** :

```
cout << "Surface du CRectangle = " << oRect . Surface
( ) << endl;
```

Dans l'instruction ci-dessus, on dira que l'on invoque la fonction membre *Surface* sur l'objet *oRect*.

6.6. Définition des fonctions membres

Dans la classe, on l'a vu, on y trouve la définition de toutes les données membres. Cependant, en général, le corps des fonctions ne s'y trouve pas. La définition de la classe va intégrer un header file, tandis que l'on trouvera la définition des fonctions dans un fichier source portant le même nom que le header file.

Pour définir la fonction, on va utiliser l'opérateur de portée **::** de la manière suivante :

```
long CRectangle::Surface (void)
      {
      return (long) m_dHauteur * m_dLargeur;
      }
```

Exemple 8 : Définition de la fonction CRectangle::Surface

Cette syntaxe est tout à fait classique (si on se réfèrre à la syntaxe du C)… Seule exception, le nom de la fonction a été préfixé du nom de la classe à laquelle la fonction appartient et suivi par l'opérateur de portée **::**.

A l'intérieur de la fonction, on fait référence aux données membres *m_dHauteur* et *m_dLargeur*. Il s'agira des données appartenant à l'objet sur lequel on a invoqué la fonction *Surface.* Plus loin, ce mécanisme va s'éclaircir lorsque l'on abordera le pointeur **this.**

En attendant, avant de passer à la notion suivante, qui est celle du constructeur, voyons un exemple plus complet manipulant un objet de la classe *CRectangle.*

```
// Demonstration of accessor/manipulator pairs.
// Note: Many commercial class packages refer to these
// as functions that access object attributes.
#include <iostream>

/************ CRectangle Class Declaration *************/

class CRectangle
{
public:
    void SetHeight(int);       // Set member functions:
    void SetWidth(int);        // take an arg as a new value
    int  GetHeight(void);      // Get member functions:
    int  GetWidth(void);       // take no args, return a value
private:
    int m_dHeight, m_dWidth;
};

/************* CRectangle Member Functions *************/

void CRectangle::SetHeight(int dH)
{
    m_dHeight = dH;
}

void CRectangle::SetWidth(int dW)
{
    m_dWidth = dW;
}

int CRectangle::GetHeight(void)
{
    return m_dHeight;
}

int CRectangle::GetWidth(void)
{
    return m_dWidth;
}

/**************** Small Test Program ******************/

int main()
{
    CRectangle oR1;                    // Declare a CRectangle object oR1
    oR1.SetHeight(15);                 // Set the height attribute.
    oR1.SetWidth(10);                  // Set the width attribute.
    cout << "width " << oR1.m_dWidth;  // Illegal access!
    cout << "The area of CRectangle oR1 is "
         << (long)oR1.GetHeight() * oR1.GetWidth()
         << endl;
    return 0;
}
```

Exemple 9 : Première utilisation d'un objet

Dans l'exemple précédent, on peut remarquer qu'il n'est pas question pour le programme principal d'avoir accès directement aux données membres de la classe *CRectangle*. Dans cet exemple simple, cela peut apparaître comme une certaine lourdeur.

Pour des objets plus complexes, ce mécanisme va permettre de cacher complètement la représentation interne de l'objet. L'utilisateur de l'objet connaît les opérations qu'il peut réaliser dessus mais il ne sait pas du tout comment fonctionne l'objet ; il ne connaît pas la représentation

interne de l'objet. Comme la représentation interne de l'objet est cachée, cela va permettre au concepteur de la classe d'y apporter toutes les modifications qu'il désire sans que quiconque ne soit au courant.

6.7. Le pointeur This

Chaque objet d'une classe possède sa propre copie des données membres de la classe. Cependant, en ce qui concerne les fonctions membres, celles-ci n'existent qu'une seule fois ; tous les objets utilisent donc la même version de chaque fonction membre.

En sachant cela, la question qui vient directement à l'esprit est : comment le compilateur fait-il pour mettre en liaison le code de la fonction et les données de l'objet qui a invoqué la fonction?

Vous l'aurez deviné … la réponse est : grâce au pointeur **this**. Il s'agit d'un argument caché qui pointe vers l'objet qui invoque une fonction membre.

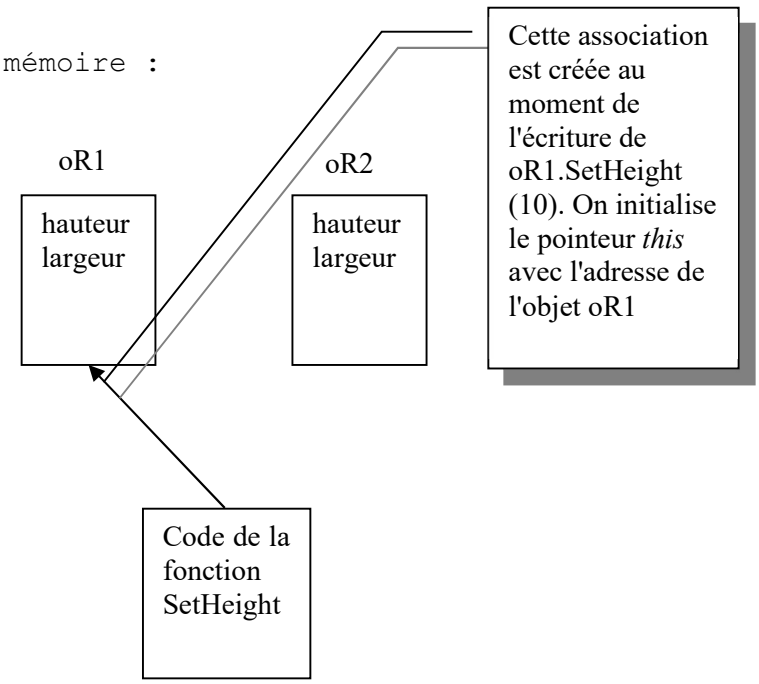
Si on reprend les définitions suivantes :

```
int main ()
     {
     CRectangle oR1 , oOR2;

     oR1.SetHeight(10);

     }
```

On dispose de ceci en mémoire :



Le compilateur interprète comme suit le code de chaque fonction membre :

```
void CRectangle::SetHeight (int dH)
     {
     this->m_dHeight = dH;
     }
```

En écrivant le code d'une fonction membre, le programmeur peut, quand il le veut, faire référence au pointeur *this.* L'exemple d'utilisation le plus connu est celui d'une fonction membre qui doit retourner comme valeur l'adresse de l'objet sur lequel la fonction a porté. Vous pouvez vous référer à *l' Exemple 17 : Une fonction retourne un objet comme valeur de retour, page 61*.

Dans cet exemple, la fonction *substitute* doit retourner un objet de la classe *String* ; cet objet est celui sur lequel la fonction a été exécutée. Pour y arriver, on trouvera dans la fonction l'instruction suivante :

```
return (*this);
```

6.8. Exercice

Exercice PRJ0 1 : La classe Fiche Salaire

Ecrire une classe *FicheSalaire* qui contient les données suivantes :

- le coût du salaire horaire
- le nombre d'heures prestées

Et qui contient les fonctions suivantes :

- l'initialisation du salaire horaire
- l'initialisation du nombre d'heures prestées
- le calcul du salaire.

Exercice PRJ0 2 : La classe Fiche Salaire

Reprendre l'énoncé de l'exercice précédent mais en ajoutant :

1. la fonction membre Affichage de la classe FicheSalaire
2. la fonction non-membre Affichage de la classe FicheSalaire

7. Constructeurs et Destructeur

7.1. Le constructeur par défaut

Le **constructeur** d'une classe est une fonction membre de cette classe qui porte le même nom que celui de la classe. Il s'agit d'une fonction qui va être appelée automatiquement lorsqu'un objet de la classe sera créé. Sa fonction essentielle va être d'initialiser l'objet qui est en train d'être créé.

Chaque classe a au moins un constructeur et le programmeur peut ne pas le déclarer ; dans ce cas, le compilateur en créera un par défaut. Le **constructeur par défaut** n'accepte aucun argument. L'initialisation des variables se fait comme lors de la déclaration d'une variable simple que l'on n'initialise pas. Les variables globales sont initialisées à zéro tandis que les variables locales ne le sont pas. Le compilateur considère l'existence du constructeur par défaut même lorsqu'il ne se trouve pas dans la liste des fonctions de la classe. Sauf, lorsqu'il existe d'autres constructeurs dans la classe : dans ce cas, le constructeur par défaut doit être mentionné de manière explicite.

Un constructeur ne retourne aucune valeur. Lors de la définition du constructeur, il ne faut indiquer aucun type de retour ; même pas *void*.

```
class CRectangle
        {
        public:
        CRectangle ();
        …
        }

CRectangle :: CRectangle ()
        {
        cout << "exécution du constructeur";
        m_dLargeur = 0;
        m_dHauteur = 0;
        }
```

Exemple 10 : Définition d'un constructeur

Le terme **constructeur**, en Anglais, est souvent abréggé comme suit : ***c'tor***.

7.1.1. Quelques propriétés

Voici quelques propriétés des constructeurs :

- une classe peut avoir plusieurs constructeurs
- le constructeur est invoqué dès qu'un objet de la classe est créé
- le constructeur n'a pas de type de retour
- si une classe dérivée n'a pas d'autres données membres que celles définies dans la classe de base, alors le constructeur de la classe dérivée peut avoir un corps vide mais il doit être défini.

- le compilateur génère automatiquement un constructeur par défaut s'il n'en trouve pas dans la classe.

- le constructeur hérite du niveau d'accessibilité de la section où il est définit : *public, private, protected.*

7.2. Passage de paramètres au constructeur

Une classe pourra contenir plusieurs constructeurs lorsque l'on désire donner à chacun d'eux des listes de paramètres différentes. Considérons l'exemple ci-dessous.

```
class CComplex
       {
       public:
       CComplex ();
       CComplex (double , double);
       …
       }

CComplex::CComplex ()
       {
       m_lfReel = 0;
       m_lfImag = 0;
       }

CComplex::CComplex (double lfR , double lfI)
       {
       m_lfReel = lfR;
       m_lfImag = lfI;
       }
```

Exemple 11 : Définition de plusieurs constructeurs dans une classe

Voyons maintenant comment l'utilisateur peut invoquer un de ces constructeurs :

- par un appel explicite : CComplex oC1 = **CComplex (1.0 , 2.0)**;
- par un appel implicite : CComplex oC1 **(1.0 , 2.0)**;
  CComplex oC1;

7.3. Le destructeur d'une classe

Le **destructeur** d'une classe est une autre fonction spéciale pouvant exister à l'intérieur d'une classe. A l'inverse du *constructeur*, il s'agit d'une fonction qui va être invoquée au moment où l'objet est supprimé. Grâce à cette fonction membre, il va ainsi être possible de supprimer proprement un objet ; l'exemple le plus simple à comprendre est celui d'un objet pour lequel on a du allouer de l'espace mémoire dynamiquement.

A l'inverse du *constructeur*, il ne peut exister au plus qu'un seul *destructeur* par classe. Ce *destructeur* :

- n'accepte aucun paramètre
- ne retourne aucune valeur

Le *destructeur* d'une classe est une fonction membre dont le nom est identique au nom de la classe mais précédé du caractère *tilde* ~ .

Comme exemple, on peut se référer à l'exemple suivant qui montre quelques fonctions membres simples de la classe *CCRectangle* et notamment, un *constructeur* et un *destructeur*.

```
// Includes default constructor and destructor
#include <iostream>

/************ CRectangle Class Declaration *************/
// Interface to x and y coordinates not yet implemented.
class CCRectangle
{
public:                        // Construction section:
    CRectangle();               // constructor (no return value)
    ~CRectangle();              // destructor (no args, no ret)
    void SetHeight(int);       // Attributes section:
    void SetWidth(int);
    long GetArea(void);
private:                       // Implementation section:
    int m_dHeight, m_dWidth;
};

/************ CRectangle Member Functions **************/
CCRectangle::CCRectangle()    // Definition of constructor
{                             //   name matches class name
    cout << "CRectangle c'tor.\n";
    m_dHeight= 0;            // Member functions have full
    m_dWidth = 0;             // access to data members.
}                             // C'tors never return a value!

CCRectangle::~CCRectangle()   // Definition of destructor
{                             //   ~ and class name
    cout << "CRectangle d'tor.\n";
}

void CCRectangle::SetHeight(int dH)
{
    m_dHeight = dH;
}

void CCRectangle::SetWidth(int dW)
{
    m_dWidth = dW;
}

long CCRectangle::GetArea(void)
{
    return (long)m_dWidth * m_dHeight;
}

/***************** Small Test Program ******************/
int main()
{                             // When instantiating an object
    CCRectangle oR1;             // the constructor for that
                              // class is called (no ()'s).
    /*                Why no ()'s? To the compiler, this
    CRectangle oR2();  looks like a function prototype! */

    cout << "The initial area of CRectangle oR1 is "
         << oR1.GetArea() << endl;
    oR1.SetHeight(15);     // Set attributes for oR1
    oR1.SetWidth(10);
    cout << "The area of CRectangle oR1 is "
         << oR1.GetArea() << endl;
    return 0;             // Note: A call to the d'tor
}                         // is not coded!
```

Exemple 12 : Constructeur et destructeur pour la classe CRectangle

Fichier "Trace.h"

```
class CTrace
{
        private:

        char * m_pFct;
        static short m_dLevel;

        public:

        CTrace ();
        CTrace (char *);
        ~CTrace ();
        static short GetLevel ();
};
```

Fichier "Trace.cpp"

```
short CTrace::m_dLevel = 0; // Obligatoirement dans le .CPP afin d'éviter la
                            // double inclusion dans les 2 fichiers .CPP.
                            // Le mécanisme de protection d'inclusion marche
                            //  uniquement  pour  la  compilation  d'un  fichier
                            // source !
Use namespace std ;

CTrace::CTrace (char *pNomFonction)
{
        m_pFct = (char *) malloc (sizeof(char) * (strlen(pNomFonction) + 1);
        if (m_pFct == NULL)
              assert(0);

        strcpy(m_pFct, pNomFonction);
        m_dLevel++;

        for (int dI = 0 ; dI < m_dLevel ; dI++)
              cerr << "-";
              ou cout << "-";
              ou std::cout << "-";

        cerr << "Entering " << m_pFct << endl;
}

CTrace::CTrace()
{}

short CTrace::GetLevel ()
{
        return(m_dLevel);
}

CTrace::~CTrace ()
{

        for (int dI = 0 ; dI < m_dLevel ; dI++)
              cerr << "-";

        cerr << "Exiting " << m_pFct << " [" << m_dLevel  << "]" << endl;
        m_dLevel--;
        free (m_pFct);
}
```

Fichier "Main.cpp"

```
void F1 ()
{
        CTrace oF1 ("F1");
        printf ("[%d]", CTrace::GetLevel()) ;
}


Void F2 ()
{
        CTrace oF2 ("F2");
        printf ("[%d]", CTrace::GetLevel()) ;
        F1 ();
}

Void F3 ()
{
        CTrace oF3 ("F3");
        printf ("[%d]", CTrace::GetLevel()) ;
        F2 ();
}

int main ()
{

        CTrace oM ("main");
        printf ("[%d]", CTrace::GetLevel()) ;
        F3 ();
}

-   - Entering main [1]
-   - - Entering F3 [2]
-   - - - Entering F2 [3]
-   - - - - Entering F1 [4]
-   - - - - Exiting F1 [4]
-   - - - Exiting F2 [3]
-   - - Exiting F3 [2]
-   - Exiting main [1]
```

Exemple 13 : La classe CTrace

7.4. Exercice

Exercice PRJ0 3 : La classe CSetInt

Réaliser une classe nommée *CSetInt* permettant de manipuler des ensembles de nombres entiers. On devra pouvoir réaliser sur un tel ensemble les opérations classiques suivantes :

- Lui ajouter un nouvel entier ; si plus de place, la fonction renvoit une valeur dans le code de retour. Si l'entier existe déjà dans l'objet, alors il ne devra plus y être ajouté.
- Connaître son cardinal (nombre d'éléments)
- Savoir si un entier donné lui appartient
- Comptabiliser le nombre d'objets qui existent et pouvoir afficher ce total à tout moment

Ici on conservera les différents éléments de l'ensemble dans un tableau alloué dynamiquement par le constructeur. Un argument (auquel on pourra prévoir une valeur par défaut) lui précisera le nombre maximal d'éléments de l'ensemble.

- Constructeur par défaut
- Constructeur acceptant un seul paramètre spécifiant la taille de l'ensemble
- Le destructeur
- Fonction permettant d'afficher le nombre d'objets existants

7.5. Les listes d'initialisation

Il arrive très souvent que le *constructeur* d'une classe se limite à initialiser les données membres de la classe avec la valeur des paramètres reçus. Aussi, afin d'éviter de devoir écrire le corps d'un tel constructeur, on a la possibilité d'utiliser une **liste d'initialisation**. Voici la syntaxe a utiliser :

```
classe::classe (liste paramètres) : member1(expression), …,
                membern(expression)
```

```
class CRectangle
       {
       CRectangle ();
       CRectangle (int , int);
       …
       }

CRectangle::CRectangle () : m_dLargeur (0) , m_dHauteur(0)
       {
       }

CRectangle::CRectangle (dL , dH) : m_dLargeur (dL) , m_dHauteur (dH)
       {
       }
```

Ces initialisations sont exécutées **avant** l'exécution du constructeur

Exemple 14 : Liste d'initialisation

Le corps d'un constructeur d'une classe dérivée est exécuté **après** l'exécution du constructeur des classes de base.

Ce mécanisme est indispensable pour une donnée membre définie comme **const**.

7.6. L'initialisation membre à membre ou le constructeur par copie

Plus haut dans ce chapitre, nous avons déjà évoqué le constructeur par défaut que le compilateur invoque lorsque le concepteur de la classe n'en n'a pas prévu.

Il existe un autre constructeur par défaut que le compilateur va invoquer si celui-ci n'en trouve pas un du même type dans la définition de la classe. Quel est cet autre constructeur ?

Il s'agit du constructeur qui sera invoqué lorsque l'on initialise un objet avec la valeur d'un autre objet de la même classe. Par exemple :

```
CString oVoyelle ("a");

CString oArticle = oVoyelle;
```

Exemple 15 : Initialisation membre à membre

Pour effectuer une telle initialisation, le compilateur va invoquer un constructeur par défaut qui va réaliser une initialisation **membre à membre**. L'initialisation de l'objet *oArticle* va se faire en recopiant la valeur de chaque membre de l'objet *oVoyelle*.

Le constructeur par défaut utilisé par le compilateur a la définition suivante :

X :: X (const X&);

Un objet de la classe *X* est initialisé par un autre objet de la classe *X* dont on en reçoit une référence. En outre, l'objet qui sert à réaliser l'initialisation n'est pas modifié.

Dans le cas de la classe *CChaine*, ce constructeur aurait la forme suivante :

```
CChaine::CChaine (const CChaine &oS)
       {
       m_dLen = oS.m_dLen;
       m_pStr = oS.m_pStr;
       }
```

Exemple 16 : Initialisation membre à membre pour la classe CChaine

L'initialisation d'un objet de classe avec un autre objet de classe a lieu dans les trois situations suivantes :

1. L'initialisation explicite d'un objet de classe avec un autre. On peut se référencer à *l'Exemple 15 : Initialisation membre à membre, page 60.*

2. Le passage d'un objet comme argument d'une fonction

3. L'utilisation d'un objet de classe comme valeur de retour d'une fonction. Par exemple :

```
CString CString::Substitute (char , char)
       {
       …
       }

main ()
       {
       CString oRiviere ("meuse");

       cout << oRiviere << oRiviere.Substitute ('m' , 'M');
       …
       }
```

Exemple 17 : Une fonction retourne un objet comme valeur de retour

En règle générale, l'initialisation membre à membre par défaut est insuffisante pour les classes qui contiennent des pointeurs et qui définissent un destructeur.

Ainsi, pour un objet qui possède des pointeurs comme données membres, le destructeur va certainement effectuer une désallocation de la mémoire référencée par l'objet qui est en train d'être détruit.

Cela siginifie que si nous réalisons une initialisation membre à membre entre deux objets qui utilisent des pointeurs, les deux objets vont tous les deux pointer vers les mêmes adresses. Lorsque l'on va détruire un des deux objets, on va aussi désallouer la mémoire à laquelle cet objet faisait référence. Il va en résulter pour le deuxième objet, que ses pointeurs vont pointer vers de la mémoire qui n'est plus allouée !

Pour résoudre ce problème, le concepteur de la classe doit fournir lui même un constructeur qui va permettre de traiter les initialisations membre à membre entre les objets de sa classe ; on l'appelera **constructeur par copie**. Il va devoir définir le constructeur qui suit :

X (const X&)

Dans le cas de la classe *CChaine*, nous allons avoir le constructeur suivant :

```
CChaine :: CChaine (const CChaine &oNew)
       {
       m_dLen = oNew. m_dLen;

       m_pStr = new char [m_dLen + 1];

       assert (m_pStr != 0);
       strcpy (m_pStr , oNew. m_pStr);
       }
```

Exemple 18 : Le constructeur X (const X&) pour la classe CChaine

Construit de cette sorte, tout objet de la classe *CChaine* aura sa propre allocation mémoire qui ne pourra être supprimée **que** lors de sa destruction et **non** par la destruction d'un autre objet.

7.7. Constructeur de conversion

Ce troisième type de constructeur sera abordé dans le chapitre relatif aux conversions : *8.5 Les conversions de type dans les classes, page 76.*

7.8. Construction des tableaux d'objets

Un tableau d'objets déclaré de la manière suivante :

```
CClasseName tNomTableau[taille] ;
```

Peut être initialisé au moyen d'une liste de valeurs qui peuvent être de types différents. Chaque valeur est alors transmise à un constructeur approprié. Il peut s'agir d'expressions. Pour les derniers objets du tableau, on peut ne pas préciser de valeurs. Dans ce cas, c'est le constructeur par défaut qui sera utilisé.

L'initialisation prendra donc la forme suivante :

```
CClasseName tNomTableau[taille] = {expression1, …, expressionn} ;
```

Dans cette liste, il n'est pas autorisé d'omettre des expressions et il n'est pas possible de préciser plusieurs valeurs pour le même constructeur. Les écritures suivantes ne sont donc pas admises :

```
CClasseName tNomTableau[taille] = {expres1, , {expres2, expres3}} ;
```

7.9. Exercices

7.9.1. La classe CPoint

Implémenter une classe *CPoint* pour représenter des points dans un espace à trois dimensions *(x, y ,z)*. Inclure un constructeur par défaut, un constructeur par copie, une fonction *negate ()* qui transforme un point en son opposé, une fonction *norm ()* qui retourne la distance d'un point par rapport à l'origine *(0, 0, 0)*, une fonction *print ()* et le destructeur.

Les données membres doivent être *private* tandis que les fonctions doivent quant à elles être *public.*

7.9.2. La classe Stack

Implémenter une classe *Stack* qui représente des piles d'entiers. Inclure un constructeur par défaut, un destructeur et les opérations classiques sur les piles : *push (), pop()* et *isEmpty ()*. Utiliser une implémentation par pointeur.

7.9.3. La classe CChaine

Exercice PRJ0 4 : La classe CChaine

Implémenter une classe *CChaine*. Chaque objet de cette classe représente une chaîne de caractères. Avec cette classe, l'utilisateur doit être capable de :

- De créer une chaîne de plusieurs manières différentes --> quels constructeurs faut-il prévoir ?
- Définition du constructeur par copie

Exercice PRJ0 5 : La classe CSetInt

En reprenant la classe CSetInt réalisée précédemment, que faut-il lui ajouter afin que l'on puisse transmettre un objet de cette classe en paramètre à une fonction ? On en déduira que l'objet doit être passé par valeur à la fonction.

7.9.4. Quels sont les résultats fournits

Exercice PRJ0 6 : Inscrivez sur papier les résultats fournits par le programme suivant :

```
#include <iostream>

Using namespace std;

class CPoint
{
private :
       int m_dX, m_dY;
public :
       CPoint ()
       {
              m_dX = 0;
              m_dY = 0;
              cout << "*** constructeur 0 argument\n";
       }

       CPoint (int dAbs)
       {
              m_dX = dAbs;
              m_dY = 0;
              cout << "*** constructeur 1 argument\n";
       }

       CPoint (int dAbs, int dOrd)
       {
              m_dX = dAbs;
              m_dY = dOrd;
              cout << "*** constructeur 2 argument\n";
       }

       CPoint (const CPoint &oP)
       {
              m_dX = oP.m_dX;
              m_dY = oP.m_dY;
              cout << "*** constructeur par copie\n";
       }

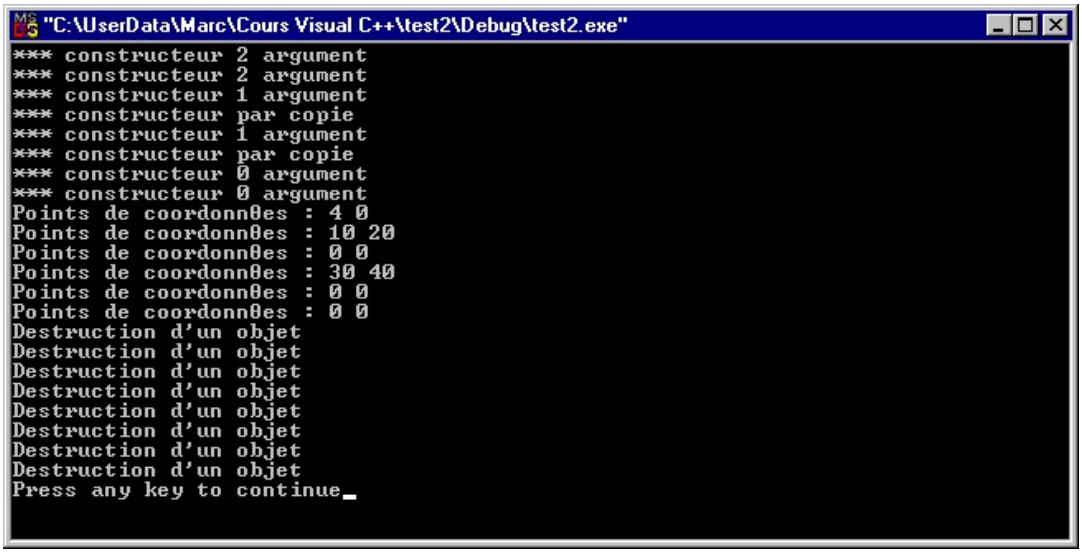
       ~CPoint()
       {
              cout << "Destruction de l'objet \n" ;
       }

       void Affiche ()
       {
              cout << "Points de coordonnées : " << m_dX << " " << m_dY <<
                     "\n";
       }
};

main ()
{
       CPoint oA (10, 20);
       CPoint oB (30, 40);
       CPoint oCourbe [6] = {4, oA, 0, oB};

       for (int dI=0; dI < 6 ; dI++)
              oCourbe [dI].Affiche();
}
```

Voir résultat à la page suivante uniquement lorsque vous aurez mis sur papier votre réponse !



7.9.5. La classe Date

Implémenter une classe *Date* avec des données membres représentant le mois, le jour et l'année. Chaque objet de cette classe représente une date spécifique ; le mois, le jour et l'année étant stockés sous forme d'entiers. Il faut inclure un constructeur par défaut, un constructeur par copie, des fonctions d'accès, une fonction *reset (int y , int m , int d)* pour réinitialiser la date d'un objet, une fonction *advance (int y , int m , int d)* pour avancer une date donnée de *y* années, de *m* mois et de *j* jours. Pour terminer, une fonction *print ()*. Il faut écrire une fonction utilitaire qui va contrôler que la date est correcte. Une fonction qui retourne le nombre de jours dans le mois de la date est certainement une fonction utilitaire très utile.

8. Quelques caractéristiques intéressantes

Dans le chapitre précédent, nous avons abordé la construction d'une classe. Ce chapitre présente maintenant quelques manières d'améliorer l'efficacité des fonctions membres.

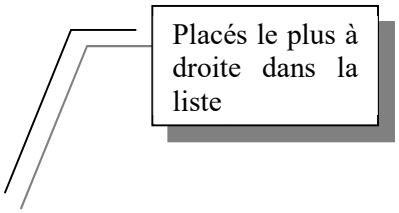
8.1. Les arguments par défaut

Il est possible d'écrire des fonctions membres qui acceptent plusieurs paramètres … Heureusement ! Ce n'est pas nouveau ! Mais, par contre, on peut spécifier que, un ou plusieurs, des paramètres peuvent être omis et que s'ils le sont, alors ils prendront une valeur par défaut…

Les paramètres qui peuvent être omis devront se trouver le plus à droite dans la liste des paramètres. C'est dans la définition du prototype de la fonction que l'on va spécifier quels sont les paramètres qui ont une valeur par défaut. On va utiliser le signe = pour spécifier les paramètres qui ont une valeur par défaut.

Voici les manières que l'on peut utiliser :

- `void fA (int , char, int = 94);`
- `void fB (char cH, int dI = 10);`
- `void fC (int , char, int, char = 'K', int = 7);`



La correspondance entre les paramètres et les arguments est toujours faite sur base de la position qu'ils occupent dans leur liste respective.

Quand le compilateur se retrouve face à une fonction qui possède un paramètre facultatif, il considère qu'il se trouve face à deux signatures de fonction ; une qui contient réellement le paramètre et une qui ne le contient pas. Nous reparlerons des signatures un peu plus loin.

```
// Functions that define default values for selected
// arguments streamline the interface and allow
// class users multiple variations
#include <iostream>

/************ CRectangle Class Declaration *************/
class CRectangle
{
public:
    // This c'tor is equivalent to three c'tors
    CRectangle(int h, int w, int x=0, int y=0);
    ~CRectangle();
    void SetCenter(int, int);
    void SetSize(int, int);
    void Draw();
private:
    int m_dX, m_dY;
    int m_dHeight, m_dWidth;
};

/******* CRectangle Member Function Definitions ********/
CRectangle::CRectangle(int h, int w, int x, int y)
    : m_dHeight(h), m_dWidth (w), m_dX (x), m_dY (y)
{
    cout << "Rect c'tor\n";
}

CRectangle::~CRectangle()
{
    cout << "Rect d'tor\n";
}
void CRectangle::SetCenter(int dX, int dY)
{
    m_dX = dX;
    m_dY = dY;
}
 void CRectangle::SetSize(int dH, int dW)
{
    m_dHeight= dH;
    m_dWidth = dW;
}
 void CRectangle::Draw(void)
{            // Currently just a display function
    cout << "CRectangle at x:" << m_dX << " y:" << m_dY;
    cout << " height:" << m_dHeight
         << " width:" << m_dWidth;
}
 /**************** Small Test Function ****************/
int main()
{
    CRectangle oR1 (1, 2),        // default x and y as 0
               oR2 (5, 6, 8),          // default y as 0
               oR3 (10, 10, 100, 100);     // no defaults

 // CRectangle oR5 (9, 9, , 40);// Error: improper syntax

    cout << "Displaying oR1:\n";
    oR1.Draw();
    cout << endl;
    oR1.SetSize(11, 12);
    oR1.SetCenter(-10, -10);
    cout << "Displaying oR1 after manipulation:\n";
    oR1.Draw();
    cout << endl;
    cout << "Displaying oR2:\n";
    oR2.Draw();
    cout << endl;
    cout << "Displaying oR3:\n";
```

```
    oR3.Draw();
    cout << endl;
    return 0;
}
```

Exemple 19 : Utilisation d'une fonction avec des paramètres par défaut

8.2. La surcharge de fonctions

Avant de définir le concept de *surcharge de fonction*, il est nécessaire de définir le concept de **signature de fonction**. En C++, la signature d'une fonction est formée de trois éléments :

- le nom de la fonction
- le type de ses paramètres
- le nombre de ses paramètres

Attention, le type de retour ne fait pas partie de la signature de la fonction.

Le compilateur C++ génère un message d'erreur lorsqu'il se retrouve confronté à deux fonctions qui possèdent la même signature. Si on compare avec le C, on dirait que dans ce langage, la signature d'une fonction n'est constituée que par le nom de la fonction.

Cette caractéristique va permettre de définir dans un même classe plusieurs fonctions membres qui portent le même nom mais qui possèdent des listes de paramètres qui diffèrent au moins dans le type de l'un de leur paramètre. Une telle fonction sera dite, **fonction surchargée.** Une même fonction (du point de vue logique) est définie plusieurs fois et par conséquent peut s'utiliser de plusieurs manières différentes.

Par exemple, dans la classe *CRectangle*, on se retrouve face à trois constructeurs :

- `CRectangle (int dX);`
- `CRectangle (int dX , int dY);`
- `CRectangle (int dX , float fY);`

On se retrouve face à une opération logique identique (construire un nouvel objet *CRectangle*), mais qui peut s'utiliser de trois manières différentes. Il y a *surcharge* de la fonction *CRectangle*.

Attention aux ambiguïtés ! En écrivant le quatrième constructeur suivant, le compilateur générerait un message d'erreur :

- `CRectangle (int dX, int dY = 0);`

```
// Functions with the same name and different argument
// data-types and/or argument counts are overloaded.
#include <iostream>

/************ CRectangle Class Declaration *************/
class CRectangle
{
public:
    // The following c'tors are overloaded
    CRectangle();
    CRectangle(int dH, int dW, int dX=0, int dY=0);
    ~CRectangle();
    void SetCenter(int, int);
    void SetSize(int,int);
    void Draw(void);
private:
    int m_dX, m_dY;
    int m_dHeight, m_dWidth;
};

/******** CRectangle Member Function Definitions *******/
CRectangle::CRectangle()
    : m_dHeight(0), m_dWidth(0), m_dX(0), m_dY(0)
{
    cout << "Rect default c'tor\n";
}

CRectangle::CRectangle(int dH, int dW, int dX, int dY)
    : m_dHeight(dH), m_dWidth(dW), m_dX(dX), m_dY(dY)
{
    cout << "Rect(int,int,int,int) c'tor\n";
}

CRectangle::~CRectangle()
{
    cout << "Rect d'tor\n";
}

void CRectangle::SetCenter(int dX, int dY)
{
    m_dX = dX;
    m_dY = dY;
}

void CRectangle::SetSize(int dH, int fW)
{
    m_dHeight= dH;
    m_dWidth = dW;
}

// Currently just a display function
void CRectangle::Draw(void)
{
    cout << "CRectangle at x:" << m_dX << " y:" << m_dY;
    cout << " height:" << m_dHeight<< " width:"
         << m_dWidth;
}

/*************** Small Test Function *****************/
                        // function prototypes
void Goodbye(int dX = 1); // Goodbye with default int arg
void Goodbye(CRectangle); // Goodbye with CRectangle arg

int main()       // Note: Cannot overload main function!
{
    CRectangle oR1 (1, 2),
               oR2 (5, 6, 8),
               oR3 (10, 10, 100, 100);
```

```
    CRectangle oR4;        // legal with default c'tor

    cout << "Displaying oR1:\n";
    oR1.Draw();
    cout << "\nDisplaying oR2:\n";
    oR2.Draw();
    cout << "\nDisplaying oR3:\n";
    oR3.Draw();
    cout << "\nDisplaying oR4:\n";
    oR4.Draw();
    cout << endl;
    Goodbye();
    // Note: The destruction of the temporary CRectangle
    Goodbye(oR4); // created when oR4 was passed by value.
    cout << "\nend of main\n";
    return 0;
}

void Goodbye(int dX)
{
    cout << "Hello from Goodbye(int x = "
         << dX << ")\n";
}

void Goodbye(CRectangle oR)
{
    cout << "Hello from Goodbye(CRectangle)\n";
}
```

Exemple 20 : Surcharge de fonction dans la classe CRectangle

8.3. Les fonctions inline

8.3.1. Description

Si on devait comparer une fonction *inline* à un concept connu, on pourrait la comparer à une macro. Une fonction **inline** est une fonction dont le corps va être recopié à l'endroit où elle est appelée. Si une telle fonction doit être utilisée dans plusieurs fichiers sources, alors il faudra définir cette fonction à l'intérieur d'un fichier .

Une fonction de petite taille invoquée souvent est une excellente candidate pour être une fonction *inline*. En effet, une telle fonction sera beaucoup plus rapide puisque le mécanisme du *call* est évité.

Il y a deux manières de définir une fonction *inline* :

- Placer le corps de la fonction à l'intérieur de la classe ; solution à conseiller s'il s'agit d'une fonction publique. En effet, afin de pouvoir être utilisée, une fonction *inline* doit se trouver dans un fichier .

- Placer le mot réservé *inline* devant le nom de la fonction lors de sa déclaration

On peut utiliser le mot réservé *inline* pour des fonctions qui n'appartiennent pas à une classe :

```
inline int max( int a , int b )
{
      if ( a > b )
            return a;
      return b;
}
```

Le compilateur peut refuser de réaliser l'inclusion du corps de la fonction s'il estime que cela serait plus néfaste que bénéfique pour le programme.

```
// Implicitly "inline" functions have the function body
// defined within the class definition.
#include <iostream>

/************** CMoney Class Definition ****************/
class CMoney
{
public:
    CMoney(long lD, int dC)
        : lDollars (lD), dCents (dC)
    { }
    void Display()
        { cout << "$" << lDollars << "." << dCents; }
private:
    long lDollars;
    int dCents;
};

/***************** Small Test Function *****************/
int main()
{
    CMoney PocketChange (1, 50);
    CMoney CMoneyClip (12, 0);
    PocketChange.Display();
    cout << endl;
    CMoneyClip.Display();
    cout << endl;
    return 0;
}
```

Deux fonctions *inline* implicites

Exemple 21 : Fonctions inline implicites

```
// Using the "inline" keyword, functions are suggested
// for inlining regardless of the location of body.
#include <iostream>

/************** CMoney Class Definition ****************/
class CMoney
{
public:
    inline CMoney(long lD, int nC);
    inline void Display();
private:
    signed long m_lDollars;
    int m_dCents;
};

/************ CMoney Class Member Functions *************/
CMoney::CMoney(long lD, int dC)
    : m_lDollars (lD), m_dCents (dC)
{ }

void CMoney::Display()
{
    cout << "$" << m_lDollars << "." << m_dCents;
}

/**************** Small Test Function *****************/
int main()
{
    CMoney PocketChange (1, 50);
    CMoney CMoneyClip (12, 0);
    PocketChange.Display();
    cout << endl;
    CMoneyClip.Display();
    cout << endl;
    return 0;
}
```

Exemple 22 : Fonctions inline explicites

Attention : quand l'usage d'une fonction *inline* se trouve dans un fichier .cpp différent, il est impératif de placer la définition de la fonction à l'intérieur du fichier . C'est normal puisque d'une part, la compilation se fait séparémment, fichier par fichier et ensuite, comme le compilateur doit faire l'inclusion du code à la compilation, cela explique pourquoi la définition de la fonction doit se trouver dans le fichier d'entête.

8.3.2. Les fonctions inline augmentent-elles la performance ?

Il n'y a pas de réponse simple. Les fonctions inline peuvent rendre le code plus rapide, mais elles peuvent aussi le rendre plus lent. Elles peuvent rendre le code plus gros ou le rendre plus petit. Elles peuvent ralentir considérablement le programme ou le rendre plus performant. Elles peuvent aussi n'avoir aucun impact sur la rapidité du code.

Les fonctions inline peuvent rendre le code plus rapide L'intégration du code peut supprimer une poignée d'instructions inutiles (accès à la pile), ce qui peut rendre le code plus rapide.

Les fonctions inline peuvent rendre le code plus lent Un excès de fonctions inline peur rendre le code 'indigeste', ce qui peut provoquer un excès d'accès à la mémoire virtuelle sur certains systèmes. En d'autres mots, si la taille de l'exécutable est trop importante, le système

risque de passer beaucoup de temps à faire de la pagination sur disque pour accéder à la suite du code.

Les fonctions inline peuvent rendre le code plus gros C'est la notion de code 'indigeste', décrite ci-dessus. Par exemple, si un programme a 100 fonctions inline qui augmenteront à chaque fois la taille de l'exécutable de 100 bytes et qui sont appelées 100 fois chacune, l'augmentation de taille de l'exécutable sera proche de 1 MB. Est-ce que ce Mo posera problème ? Qui sait, mais ce Mo risque d'être celui qui fera faire de la pagination au système et donc le ralentir.

Les fonctions inline peuvent rendre le code plus petit Les compilateurs génèrent souvent plus de code pour empiler / désempiler les paramètres que l'inclusion du code ne le ferait. C'est ce qui arrive avec de très petites fonctions, et cela peut même arriver avec de grosses fonctions quand l'optimiseur arrive à supprimer le code redondant via l'inclusion du code - l'optimiseur peut donc transformer de grosses fonctions en petites.

Les fonctions inline peuvent augmenter la pagination Le code généré peut devenir très gros, ce qui risque de causer un ralentissement considérable.

Les fonctions inline peuvent réduire la pagination Le nombre de pages qui doivent se trouver en mémoire au même moment peut se réduire, alors que la taille de l'exécutable augmente. Quand f() appelle g(), le code peut très bien se trouver dans deux pages différentes, alors que lorsque le compilateur inclut le code de g() dans f(), le code a plus de probabilité de se trouver dans la même page.

Les fonctions inline peuvent n'avoir aucune influence sur les performances Les performances ne sont pas liées qu'au CPU. Par contre, les entrées/sorties, les accès aux bases de données, l'accès au réseau peuvent représenter un sérieux goulot d'étranglement. A moins que votre CPU ne soit utilisé à 100% la plupart du temps, l'utilisation des fonctions inline n'améliorera pas les performances générales du système (et même si le goulot d'étranglement se situe au niveau du CPU, les fonctions inline ne seront utiles que dans le code posant problème, ce qui représente souvent de très petits morceaux de code).

Il n'y a pas de réponse simple : vous devez essayer et voir ce qui est le mieux. Ne vous limitez pas à des réponses simplistes telles que "Ne jamais utiliser les fonctions inline" ou "Toujours utiliser les fonctions inline" ou "N'utiliser les fonctions inline que si le code fait moins de ... lignes de code". Ces règles arbitraires sont peut-être faciles à écrire, mais les résultats sont plus que décevants.

8.4. La notion de friend

Comme déjà expliqué, les membres déclarés dans la section *private* d'une classe ne sont accessibles qu'aux seules fonctions membres de la classe. Il existe un mécanisme qui va malgré tout permettre à une ou plusieurs fonctions extérieures à la classe d'accéder aux membres *private* de la classe.

Ce mécanisme est la notion de **friend**. A l'intérieur d'une classe, il va être possible de spécifier qu'une fonction est une fonction *friend* de la classe. L'effet sera que cette fonction *friend* pourra accéder aux membres *private* de la classe.

Par exemple, supposons que l'on ait deux classes : *Enseignant* et *Scientifique.* Chaque classe possède un membre qui indique le salaire d'un enseignant ou d'un scientifique (suivant la classe). On pourrait imaginer une fonction extérieure qui soit chargée de calculer la somme du salaire d'un enseignant et d'un scientifique.

On peut placer les définitions *friend* dans n'importe quelle section car cela n'a aucun effet. Cependant, on placera de préférence les *friend* dans la section *public*.

```
class CEnseignant;
class CScientifique;

double Sal (const CEnseignant *pEns , const CScientifique *pSci)
       {
       return (pEns->m_lfSalaire + pSci ->m_lfSalaire);
       }

class CEnseignant
       {
       private:
       double m_lfSalaire;

       public:
       friend double Sal (const CEnseignant * , const
                          CScientifique *);
       }

class CScientifique
       {
       private:
       double m_lfSalaire;

       public:
       friend double Sal (const CEnseignant * , const
                          CScientifique *);
       }
```

Exemple 23 : Une fonction friend

Il est aussi possible de déclarer qu'une classe entière est une classe *friend* d'une autre classe. Par exemple :

```
class CScreen
       {
       private:
       friend class CShapeCounter;

       }
```

Suivant une telle définition de la classe *CScreen*, toutes les fonctions membres de la classe *CShapeCounter* peuvent accéder aux membres *private* de la classe *CScreen*. L'inverse n'étant pas vrai.

Un *friend* peut être :

- une fonction non membre
- une fonction membre
- une classe

Quelques remarques :

- Au contraire d'une fonction membre classique, une fonction *friend* ne peut pas être invoquée directement par un objet de la classe.
- Attention au pointeur *this*. Il n'est disponible à l'intérieur d'une fonction *friend* qu'à la seule condition que cette fonction soit une fonction membre d'une autre classe. Dans ce cas, le pointeur *this* référence l'objet de la classe d'origine de la fonction.
- Dans la classe, l'indication de la fonction *friend* peut être placée dans n'importe quelle section de la classe

- Le *friendship* n'est pas transitif. Si la classe *A* est amie de *B* alors les fonctions membres de *A* peuvent accéder aux membres privates de *B*. Si *C* est amie de *A*, alors les fonctions membres de *C* peuvent accéder aux membres privates de *A*. **Mais les fonctions membres de *C* ne peuvent pas accéder aux membres privates de *B*** !
- Le *friendship* ne s'hérite pas. Si *D* est une sous-classe de *A*, alors les fonctions membres de *D* ne peuvent pas accéder aux membres privates de *B*.

Faut-il placer le prototype de la fonction avant la déclaration de la classe ? La réponse est non car la compilation de la classe accepte la déclaration de la fonction telle qu'elle. Dans la fonction elle-même, il faut être certain d'avoir déclaré la fonction exactement comme elle l'est dans la classe. Si ce n'est pas le cas, la fonction ne sera pas reconnue comme fonction *amie*.

8.4.1. Exercice

Exercice PRJ0 7 : Ecrire une fonction friend

Soit la classe *CPoint* suivante *:*

```
class CPoint
{
public:
	CPoint (int dAbs = 0 , int dord = 0);
	virtual ~CPoint();

private:
	int dY;
	int dX;
};
```

Ecrire une fonction indépendante *Affiche()*, amie de la classe *CPoint*, permettant d'afficher les coordonnées d'un point.

8.5. Les conversions de type dans les classes

On va aborder dans ce paragraphe le problème de la conversion d'un objet d'une classe donnée vers un autre type.

En C, on peut réaliser une grande série d'opérations implicites comme convertir un *int* en une variable de type *long* ou ajouter une valeur de type *long* à une valeur de type *float*. En C++, on va faire la distinction entre les :

- conversions **implicites**
- conversions **explicites**

Les conversions **implicites** seront réalisées par le compilateur sans que le programmeur n'ait du spécifier quoi que ce soit. Tandis que les conversions **explicites** seront des conversions forcées par le programmeur.

8.5.1. Les conversions implicites

Une conversion implicite a lieu lorsque des types différents sont impliqués dans une instruction : par exemple, lors de l'assignation d'une variable de type *int* à une variable de type *long*. Des conversions implicites sont également réalisées lors du passage d'arguments à une fonction. Les valeurs passées sont éventuellement converties dans le type des paramètres de la fonction.

Par exemple,

```
int main ()
       {
       int dExemp = 3.9587;

       cout << "dExemp = " << dExemp;

       }
```

donnera évidemment comme résultat :

```
dExemp = 3
```

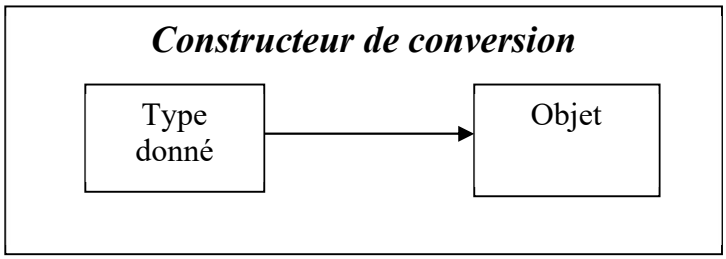
On peut également réaliser des conversions implicites dans les classes au moyen d'un constructeur. Ce constructeur est alors connu comme **constructeur de conversion**. Les arguments passés au constructeur sont convertis dans le type de l'objet. Par exemple, le constructeur suivant converti une valeur de type *int* dans une valeur de type *CComplex* :

```
CComplex::CComplex (int dVal)
       {
       m_lfreel = (double) dval;

       m_lfImag = 0;
       }
```

Avec un tel constructeur, l'appel de la fonction **CComplex (2)** va provoquer la conversion de la valeur *int* 2 en un nombre complexe : **(2.0 , 0.0)**.

En général, tout constructeur ne prenant qu'un seul argument représentera la marche à suivre pour convertir une valeur d'un type donné vers le type de l'objet dont il est le constructeur.



8.5.2. Les conversions explicites

Forcer la conversion d'un type donné vers un autre type est connu sous le terme de **conversion explicite**. Considérons l'exemple suivant. Pour assigner la valeur 2 au pointeur *p*, on ne peut pas utiliser l'instruction suivante :

```
int *p = 2;                              // erreur de conversion
```

Il y a ici une discordance de type qui n'est pas supportée par le compilateur. Cependant, si le programmeur le désire vraiment, il peut forcer la conversion en écrivant :

```
int *p = (int *) 2;
```

Cette instruction place l'adresse *2* dans le pointeur *p* après avoir converti la valeur *2* en une adresse de type *int*.

Les conversions explicites sont aussi possibles pour les classes. Une conversion explicite est définie par un **opérateur de conversion** : aussi connu sous le nome **d'opérateur cast** ou de **fonction de conversion**.

Un opérateur de conversion va utiliser la syntaxe suivante :

```
operator type ();
```

Par exemple, pour convertir un objet vers le type *double*, il va falloir définir l'opérateur de conversion suivant dans la classe :

```
operator double ();
```

La fonction ne spécifie pas de type de retour ; le type de retour est automatiquement le type spécifié après le mot réservé *operator*.

```
#include <iostream>

class CRational
        {
        friend istream& operator >>(istream& , CRational&);
        friend ostream& operator <<(ostream& , const CRational&);

        public:
        CRational (int n = 0 , int d = 1) : m_dNum (n) , m_dDen(d)
                {}

        operator double () const;

        private:
        int m_dNum , m_dDen;
        };

CRational::operator double () const
        {
        return double (m_dNum) / m_dDen;
        }

main ()
        {
        CRational oX (-5 , 8);

        cout << " X = " << oX << ", double (X) = " << double(oX);

        const CRational oP (22 , 7);
        const double lfPI = double (oP);

        cout "P = " << oP << ", PI = " << lfPI;
        }

X = -5/8, double (X) = -0.625
P = 22/7, PI = 3.14286
```

Exemple 24 : Opérateur de conversion dans la classe CRational

```
class CChaine
      {
      public:

      operator char * ();
      };
```

Comme on l'a vu, l'opérateur de conversion est appelé lors de l'utilisation de l'opérateur *cast* sur un objet. Mais, l'opérateur de conversion pourra aussi être appelé par le compilateur de manière implicite. Par exemple :

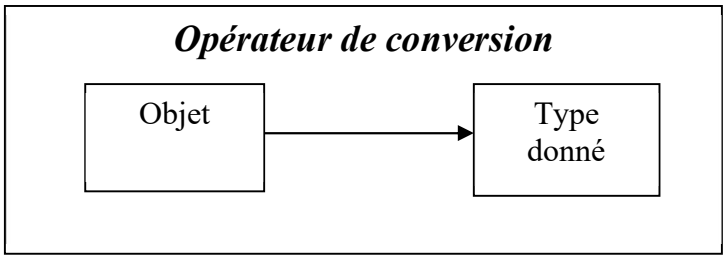
```
void openfile (char *);

main ()
       {
       CChaine oNom;

       openfile (oNom);                 // appel implicite
       openfile ((char *) oNom);        // appel explicite
       }
```

Pour terminer, quelques caractéristiques d'un opérateur de conversion :

- l'opérateur de conversion doit être une fonction membre d'une classe
- l'opérateur de conversion ne peut pas définir de type de retour
- l'opérateur de conversion ne peut pas avoir d'arguments



Ceci est juste pour pour pouvoir utiliser le nom *CCircle* par la suite. Mais, pas possible de déclarer une donnée membre de type *CCircle* car cela ferait appel à un constructeur non connu. La seule chose que l'on puisse faire est la déclaration d'une référence ou d'un pointeur.

```
// Using conversion c'tors and operators.
#include <iostream>

/*************** Class Declarations ******
// Circular forward reference needs declar
// intended). CCircle must be predefined f
class CCircle;

class CSquare
{
public:
    CSquare(int dX=0);              // conversion c'tor
    CSquare(const CSquare&);        // copy c'tor
    CSquare(const CCircle&);        // conversion c'tor
    operator CCircle () const;      //conversion operator
    void Display() const;
private:                            // implementation
    int m_dSide;                    // CSquare's have a side dimension
};

class CCircle
{
public:
    CCircle(int dN)                 //conversion c'tor
        : m_dDia(dN)
        { cout << "CCircle Conversion c'tor (int)\n"; }
    int GetDia(void) const { return m_dDia; }
    void Display(void) const;
private:                             // implementation
    int m_dDia;                      // CCircle's have a diameter dim.
};

/*********** Member Functions Definitions ************/
CSquare::CSquare(int dX)
    : m_dSide(dX)
{
    cout << "CSquare Conversion c'tor (int)\n";
}

CSquare::CSquare(const CSquare& oS)

{
    m_dSide=oS.m_dSide;
       cout << "CSquare Copy c'tor (CSquare&)\n";
}
 CSquare::CSquare(const CCircle& oC)
{
    m_dSide = oC.GetDia();
    cout << "CSquare Conversion c'tor (CCircle&)\n";
}
 CSquare::operator CCircle () const
{
    cout << "CSquare => CCircle operator\n";
    return CCircle(m_dSide);    //Invokes CCircle(int)
}

void CSquare::Display(void) const
{
    cout << "Display CSquare of side "<< m_dSide <<endl;
}

void CCircle::Display(void) const
{
    cout << "Display CCircle of dia. " << m_dDia <<endl;
}

/****************** Test Function ******************/
```

```
int main()
{
    cout << "Construct two CCircle objects:\n";
    CCircle oC1 (33),
            oC2 (66);
 // CCircle cnot;      // error: no default c'tor
    cout << "Construct two CSquare objects:\n";
    CSquare oS1,
           oS2 (25);
    cout << "Construct s3 from s2 (25):\n";
    CSquare oS3 (oS2);   // copy c'tor
    oS3.Display();
    cout << "Construct s4 from c1 (33):\n";
    CSquare oS4 (oC1);   // conv c'tor
    oS4.Display();
    cout << "Construct c3 from s1 (default):\n";
    CCircle oC3 (oS1);   // how does this work?
    oC3.Display();
    cout << "Assign a CCircle to a CSquare, s1 = c2\n";
    oS1 = oC2;          // conv c'tor for temp object
    cout << "Assign a CSquare to a CCircle, c1 = s2\n";
    oC1 = oS2;          // how does this work?
    return 0;
}
```

Exemple 25 : Constructeur de conversion et opérateur de conversion

Explication de certaines instructions de cet exemple :

Instruction	Explication
oS1 = oC2;	Il n'y a pas d'opérateur de conversion d'un cercle vers un carré. Dans ce cas, le compilateur va construire un carré temporaire auquel il va assigner la valeur du cercle C2. Cette opération est possible car il existe un constructeur de conversion d'un cercle vers un carré. Si la classe cercle avait disposé d'un opérateur de conversion le compilateur se trouve face à un choix. Il provoquera une erreur ; pour la corriger, il faudra ajouter l'opérateur d'assignation =.
Ccircle oC3 (oS1) ;	Dans cette déclaration du cercle C3, le compilateur va invoquer l'opérateur de conversion d'un carré en un cercle. De cette manière, il pourra invoquer le constructeur par copie.

Dans l'exemple de la classe CChaine, on peut imaginer la différence qu'il y a entre :

- `operator const char * () ;`
- `operator char * () ;`

La première expression est la plus élégante car elle permettra d'éviter la modification de l'objet au moyen de l'adresse qui a été retournée.

8.5.3. Exercices

Exercice PRJ08 : Les conversions

Quels résultats fournira le programme suivant :

```
#include <iostream>

class CPoint
{
	int m_dX, m_dY;

public:

	CPoint (int dAbs, int dOrd)
	{
		m_dX = dAbs;
		m_dY = dOrd;
	}

	operator int ()
	{
		cout << "** appel int () pour le point " << m_dX << " " << m_dY
		<< endl;
		return m_dX;
	}
};

void fct (double lfV)
{
	cout << "$$ appel fct avec argument : " << lfV << endl;
}

main ()
{
	CPoint oP(1, 4);
	int dN;
	double lfZ1, lfZ2;

	dN = oP + 1.75;
	cout << "dN = " << dN << endl;
	lfZ1 = oP + 1.75;
	cout << "lfZ1 = " << lfZ1 << endl;
	lfZ2 = oP;
	cout << "lfZ2 = " << lfZ2 << endl;
	fct (oP);
}
```

Exercice PRJ09 : Ajouter des opérateurs de conversion

Ajoutez quelques opérateurs de conversion aux classes CChaine et CSetInt.

8.6. La surcharge des opérateurs

On a abordé dans un chapitre précédent (*voir 8.2 La surcharge de fonctions, page 68*) la notion de fonction surchargée. Il s'agissait de pouvoir paramétriser une même opération de plusieurs manières différentes. On va voir dans ce paragraphe, que l'on va pouvoir surcharger les opérateurs prédéfinis du langage C++. Au point précédent, on l'a déjà un peu abordé puisque l'on y a redéfini l'opérateur *cast*.

Si le compilateur C++ sait parfaitement comment il doit compiler l'addition entre deux nombres entiers au moyen de l'opérateur +, comment pourrait-il savoir comment il doit compiler une addition entre deux objets de la classe Complex ? Le programmeur, lors de sa conception d'une classe, va devoir indiquer au compilateur comment il devra compiler l'opérateur + entre deux nombres complexes. Il le fera au moyen de la **surcharge des opérateurs.**

8.6.1. Les opérateurs de type binaire

On verra dans ce paragraphe comment on peut surcharger un opérateur de type binaire ; c'est à dire travaillant sur deux opérandes. Comme on l'a vu pour l'opérateur *cast*, un opérateur est défini de la même manière qu'une fonction membre excepté le fait que son nom est formé par le mot réservé **operator** suivi par le nom de l'opérateur du C++ à surcharger. On arrive ainsi à la syntaxe suivante :

```
operator symbol_opérateur (liste d'arguments)
```

Reprenons comme exemple, notre classe Complex :

```
class CComplex
        {
        private:

        double m_lfReel , m_lfImag;

        public:

        CComplex (double r , double i)
                {
                m_lfreel = r;
                m_lfImag = I;
                }

        CComplex operator = (CComplex);
        CComplex operator + (CComplex);
        };

CComplex CComplex::operator = (CComplex oCplx)
        {
        m_lfReel = oCplx.m_lfReel;
        m_lfImag = oCplx.m_lfImag;

        return *this;
        }
```

```
CComplex CComplex::operator + (CComplex Ocplx)
        {
        CComplex oTemp;

        oTemp.m_lfReel = m_lfReel + oCplx. m_lfReel;
        oTemp.m_lfImag = m_lfImag + oCplx. m_lfImag;

        return oTemp;
        }

int main ()
        {
        CComplex oCplxA (1.0 , 2.0);                // Constructeur
        CComplex oCplxB (2.0 , 3.0);

        CComplex oCplxC = oCplxA + oCplxB;          // Opérateurs + et =
        }
```

Le premier opérande est un objet Complex. C'est sur cet objet que le + porte.

Exemple 26 : La surcharge des opérateurs

L'opérateur + cumule les valeurs de deux nombres complexes oCplxA et oCplxB pour en donner un autre (les deux nombres A et B ne sont pas modifiés). Le nouveau nombre complexe est alors assigné à un dernier nombre oCplxC au moyen de l'opérateur d'assignation qui lui aussi a été surchargé.

Dans l'exemple ci-dessus, on peut dire que l'on invoque la fonction membre + sur l'objet **oCplxA** et en lui passant comme paramètre **oCplxB** :

```
oCplxA.operator +(oCplxB)

oCplxA + oCplxB
```

Il s'agit de deux écritures équivalentes.

Il est très important de remarquer dans notre addition, que le premier opérande est un objet de type *Complex*. Mais, cela ne sera pas toujours le cas. Supposons que l'on soit face à l'expression suivante :

```
oCplxA = 2 + oCplxA;
```

Comme expliqué plus haut, l'opération porte sur le premier opérande et le deuxième opérande est utilisé comme argument. On peut facilement imaginer que cela ne peut pas fonctionner dans cet exemple. Comment pourrait-on imaginer invoquer l'opération d'addition sur l'objet *2* et en lui communiquant comme paramètre l'objet Complex *oCplxA* ?

Pour résoudre ce problème, la déclaration suivante serait-elle correcte ?

```
CComplex operator + (int , CComplex);
```

La réponse est non car l'opération porte toujours sur un objet de la classe et accepte un seul argument ; le deuxième opérande de l'opération.

Pour s'en sortir, la solution va être de considérer la définition ci-dessus comme une fonction *non membre* que l'on va définir comme une fonction *friend* de la classe Complex ; elle pourra ainsi avoir accès aux données privées de la classe *Complex*.

```
class CComplex
```

```
{

friend CComplex operator + (int , CComplex);

}
```

On peut aller plus loin encore et dire que si on dispose d'un constructeur de conversion d'un *int* vers un nombre *Complex*, alors on peut écrire la fonction plus générale suivante :

```
CComplex operator + (CComplex , CComplex);
```

Restrictions

- l'opérateur surchargé doit avoir au moins une opérande qui est du type défini par l'utilisateur. Il paraît en effet difficile de redéfinir l'addition entre deux nombres entiers!

- on ne peut pas utiliser un opérateur d'une manière qui ne respecterait pas les règles d'usage de cet opérateur. Par exemple, on ne pourrait pas redéfinir l'opérateur *% (modulo)* pour accepter une seule opérande ou redéfinir l'opérateur *! (NOT)* pour accepter deux opérandes.

- on ne peut pas modifier la priorité d'évaluation d'un opérateur. Par exemple, si on redéfini l'opération d'addition pour additionner deux nombres complexes, ce nouvel opérateur gardera la même priorité que l'addition classique.

- on ne peut pas utiliser de nouveaux symboles comme opérateur. L'opérateur que l'on surcharge doit déjà exister comme opérateur dans le langage.

- on ne peut pas surcharger les opérateurs suivants :

  - sizeof
  - .
  - * (l'opérateur d'indirection)
  - ::
  - ? :

Ordre d'évaluation

Quand le compilateur rencontre une opération d'addition :

1) Il regarde si l'opérande de gauche est un objet oui ou non. Si oui, il va regarder dans la classe, s'il trouve une surcharge de l'opérateur acceptant un paramètre qui est compatible avec le type de l'opérande de droite de l'opérateur.

2) S'il ne trouve pas d'opérateur dans la classe, il regarde s'il n'y a pas une fonction non membre acceptant deux paramètres dont les types seraient compatibles avec les opérandes de l'opérateur.

3) Si une telle fonction n'existe pas et si l'opérande de droite de l'opérateur d'addition est un objet, il va essayer de convertir cet objet dans le type de l'opérande de gauche.

8.6.2. Les opérateurs de flux

Le C++ permet de surcharger les opérateurs **>>** et **<<** afin de personnaliser les entrées et les sorties. Comme c'était le cas pour les opérateurs arithmétiques, on va faire de ces deux fonctions des fonctions *friend*.

La syntaxe de l'opérateur de sortie pour une classe **T** dotée d'une donnée membre **d** est :

```
ostream& operator << (ostream& ostr , const T& t)
{
       return ostr << t.d;
}
```

ostream est une classe standard définie dans le fichier d'entête *iostream*.

Voici un exemple d'implémentation de l'opérateur d'extraction pour la classe *CRational*.

```
#include <iostream>

ostream& operator << (ostream& ostr , const CRational& r)
{
       return ostr << r.m_dNum << '/' << r.m_dDen;
}

class CRational
{
        friend ostream& operator << (ostream& , const CRational&);

        public:
        CRational (int n=0 , int d=1 ) : m_dNum (n) , m_dDen (d)
        {}

        private:
        int m_dNum , m_dDen;
};

main ()
{
       CRational oX (22, 7) , oY(-3, 8);

       cout << "X = " << oX << " , Y = " << oY << endl;
}

Résultat : X = 22/7 , Y = -3/8
```

Exemple 27: L'opérateur d'extraction pour la classe CRational

Pour surcharger l'opérateur d'entrée d'une classe **T** ayant une donnée membre **d**, on doit écrire :

```
friend istream& operator >> (istream& istr , T& t)
{
            return istr >> t.d;
}
```

Voici son implémentation dans la classe *CRational* :

```
istream& operator >> (istream& istr , CRational& oR)
{
       cout << "\t  Numérateur : " ;
       istr >> oR.m_dNum;
       cout << "\tDénominateur : ";
       istr >> oR.m_dDen;
       oR.reduce ();

       return istr;
}

class CRational
{
        friend ostream& operator << (ostream& , const CRational&);
        friend istream& operator >> (istream& , CRational&);

        public:
        CRational (int n=0 , int d=1 ) : m_dNum (n) , m_dDen (d)
        {}

        private:
        int m_dNum , m_dDen;
        void reduce ();
};

main ()
{
       CRational oX,oY;

       cin >> oX >> oY;
       cout << "X = " << oX << " , Y = " << oY << endl;
}

Résultat :

       Numérateur : -10
       Dénominateur : -24
       Numérateur : 36
       Dénominateur : -20

       X = 5/12, Y = -9/5
```

Exemple 28 : L'opérateur d'insertion de la classe CRational

8.6.3. Surcharge des opérateurs ++ et - -

L'opérateur d'incrémentation ++ et l'opérateur de décrémentation - - se présentent chacun sous deux formes : préfixée et postfixée qui peuvent toutes être surchargées. Dans ce paragraphe, on va étudier l'opérateur d'incrémentation ; il est clair que l'opérateur de décrémentation serait en tout point identique.

L'opérateur ++ est un opérateur unaire : son seul argument est l'objet incrémenté. La surcharge de cet opérateur pour une classe donnée T s'exprime simplement par :

```
T operator ++ ();

CRational operator ++ ();
```

L'exemple ci-dessus ajoute un opérateur de pré-incrémentation ++ à la classe *CRational*. Voici l'effet de cette incrémentation :

$$\frac{22}{7} + 1 = \frac{22 + 7}{7} = \frac{29}{7}$$

Il s'agit donc de simplement ajouter le dénominateur au numérateur et de retourner l'objet. Voici ce que cela donne :

```
class CRational
{
        friend ostream& operator << (ostream& , const CRational&);

        public:
        CRational (int n=0 , int d=1 ) : m_dNum (n) , m_dDen (d)
        {}
        CRational operator ++();

        private:
        int m_dNum , m_dDen;
};

CRational CRational::operator ++ ()
{
        m_dNum += m_dDen;
        return *this;
}

main ()
{
        CRational oX (22 , 7) , oY = ++oX;

        cout << "Y = " << oY << " , X = " << oX << endl;
}

Y = 29/7 , x = 29/7
```

Exemple 29 : Surcharge de l'opérateur d'incrémentation préfixé

Les opérateurs postfixés ont le même nom de fonction que les opérateurs préfixés. Par exemple, le nom des deux opérateurs de pré-incrémentation et de post-incrémentation est :

```
operator ++
```

Pour les distinguer, le C++ indique **que l'opérateur préfixé n'a qu'un seul argument et que l'opérateur postfixé en a deux**. La syntaxe exacte du prototype de l'opérateur surchargé de post-incrémentation est donc :

```
T operator ++ (int);
```

L'argument que l'on attend doit avoir le type *int*. Ceci est un peu étrange parce que lorsque l'on appelle la fonction, on ne lui passe aucun entier. L'argument entier est en fait un *argument factice*, nécessaire pour distinguer les deux opérateurs : préfixé et postfixé.

Pour rester cohérent avec l'opérateur de base de post-incrémentation sur des types entiers, cette version surchargée **ne doit pas** changer la valeur de l'objet X jusqu'à ce que l'on ait affectée à Y. Pour cela, il nous faut un objet temporaire qui stocke la valeur de l'objet propriétaire de l'appel, ce que l'on fait en affectant **this* à *temp*.

```
class CRational

{
        friend ostream& operator << (ostream& , const CRational&);

        public:
        CRational (int dN=0 , int dD=1 ) : m_dNum (dN) , m_dDen (dD)
                {}
        CRational operator ++();
        CRational operator ++(int);

        private:
        int m_dNum , m_dDen;
}

CRational CRational::operator ++ ()
{
        m_dNum += m_dDen;
        return *this;
}

CRational CRational::operator ++ (int)
{
        CRational oTemp = *this;

        m_dNum += m_dDen;
        return oTemp;
}

main ()
{
        CRational oX (22 , 7) , oY = oX++;

        cout << "Y = " << oY << " , X = " << oX << endl;
}

Résultat : Y = 22/7 , x = 29/7
```

L'argument *int* est factice. Sans lui, on serait en présence de deux fonctions ayant la même signature.

Exemple 30 : Surcharge de l'opérateur d'incrémentation postfixé

8.6.4. Surcharge de l'opérateur d'indexation

Le symbole **[ ]** est celui de l'opérateur d'indexation. Sous la forme *a [i]*, l'opérateur d'indexation possède deux opérandes : *a* et *i*. On peut surcharger **[ ]**, comme tout autre opérateur.

```
class CRational
{
        friend ostream& operator << (ostream& , const CRational&);

        public:
        CRational (int dN=0 , int dD=1 ) : m_dNum (dN) , m_dDen (dD)
        {}
        int operator [] (int);

        private:
        int m_dNum , m_dDen;
}

int CRational::operator [](int dI)
{
        if (dI == 1)
                return m_dNum;
        if (dI == 2)
                return m_dDen;
        cerr << "ERREUR : index hors limite)";
        exit (1);
}

main ()
{
        CRational oX (22 , 7);

        cout << " X = " << oX << endl;
        cout << " X[1] = " << oX[1] << " , X[2] = " << oX[2] << endl;
 }

Résultat :

X = 22/7;
X[1] = 22 , X[2] = 7
```

Exemple 31 : Surcharge de l'opérateur d'indexation

8.6.5. La surcharge de l'opérateur d'affectation

De tous les opérateurs, l'**opérateur d'affectation =** est certainement le plus utilisé. Il permet de copier un objet dans un autre. Comme le constructeur par défaut, le constructeur par copie et le destructeur, l'opérateur d'affectation est automatiquement créé lorsqu'on définit une classe. Mais, comme ces trois autres fonctions membres, il peut également être explicitement défini lors de la définition de la classe.

Voici l'exemple de l'opérateur d'affectation dans la classe *CRational*.

```
class CRational
{
        friend ostream& operator << (ostream& , const CRational&);

        public:
        CRational (int n=0 , int d=1 ) : m_dNum (n) , m_dDen (d)
                {}
        int operator [] (int);
        CRational& operator = (const CRational&);

        private:
        int m_dNum , m_dDen;
};

CRational& CRational::operator = (const CRational& oR)
{
       m_dNum = oR.m_dNum;
       m_dDen = oR.m_dDen;
       return *this ;
};
```

Exemple 32 : L'opérateur d'affectation dans la classe CRational

Attention ! La difficulté majeure de la surcharge des opérateurs réside dans la détermination du type de retour. On peut retourner soit une copie d'objet ou l'objet lui-même. Dans le cas de l'opérateur d'assignation, on retournera l'objet lui-même.

8.6.6. Exercices

Exercice PRJ010 : La surcharge des opérateurs

Soit une classe *CVecteur3D* définit comme suit :

```
class CVecteur3D
{
public:
	CVecteur3D(float fC1 = 0.0, float fC2 = 0.0, float fC3 = 0.0);
	virtual ~CVecteur3D();

private:
	float m_fX;
	float m_fY;
	float m_fZ;
};

CVecteur3D::CVecteur3D(float fC1, float fC2, float fC3)
{
	m_fX = fC1;
	m_fY = fC2;
	m_fZ = fC3;
}
```

Définir les opérateurs == et != de manière qu'ils permettent de tester la coïncidence ou la non-coïncidence de deux vecteurs :

a) En utilisant des fonctions amies

Définir l'opérateur binaire + pour qu'il fournisse la somme de deux vecteurs, et l'opérateur binaire * pour qu'il fournisse le produit scalaire de deux vecteurs. On choisira ici des fonctions amies. Les opérateurs devant retourner la valeur appropriée en regard de leur fonctionnement général.

Implémenter le constructeur par copie et afficher un message dans chaque constructeur

Exercice PRJ011 : La surcharge des opérateurs

Reprenons la classe *CSetInt* de l'exercice Prj02. Adapter cette classe en y ajoutant les opérateurs suivants :

a) L'opérateur < permettant d'ajouter un entier à un objet de la class *CSetInt*. Par exemple :

```
OSet < 10 < 20 < 30 ;
```

b) L'opérateur [dValeur] qui retourne la valeur 1 si la valeur référencée par *dValeur* appartient à l'objet *CsetInt* et retourne 0 dans le cas contraire.

c) Implémenter le constructeur par copie

d) Afficher un message dans chaque constructeur

Exemple :

```
main ()
{
	CSetInt oSet(10);
```

```
oSet < 25 < 2 < 25 < 3;

cout << (oSet[25]) << " " << (oSet[5]) << endl;}
```

8.7. Allocation dynamique d'objets et tableau d'objets

Les opérateurs *new* et *delete* peuvent aussi être utilisés pour allouer dynamiquement de nouveaux objets ou en libérer. Par exemple :

```
CRectangle *pRect = new CRectangle (7 , 7 ,0 , 0) ;
…
delete pRect ;
```

Des tableaux d'objets peuvent aussi être alloués dynamiquement :

```
CRectangle *pRect = new CRectangle [7] ;
```

Dans ce cas, le constructeur par défaut sera invoqué pour chaque objet du tableau. Il n'y a aucune possibilité d'invoquer un autre constructeur que celui par défaut. Si on veut invoquer un autre constructeur, il faut utiliser un tableau de pointeurs :

```
CRectangle *tpRect [10] ;

TpRect [0] = new CRectangle (3 , 3 , 5 , 5) ;
```

```
// Dynamically allocates and deallocates objects.
#include <iostream>
#include <stdlib>
#include "rect"
                          // function prototype
void CheckNull(void*);

void main()
{                         // Create a default CRectangle
                          // dynamically in the heap
    CRectangle *pr = new CRectangle;
    pr->Draw();
    delete pr;            // Release the memory
    cout << endl;
                          // Re-use the pointer, pr, to
                          // create another CRectangle
    pr = new CRectangle(4,14,100,-100);
    pr->Draw();
    delete pr;            // Release the memory
    cout << endl;
    unsigned int nNbrRects; // prompt the user for a nbr
    cout << "How many CRectangles would you "
            "like in the array? ";
    cin  >> nNbrRects;
                          // Using pr again, allocate an
                          // array of CRectangles with the
    pr = new CRectangle[nNbrRects]; // user's size
    CheckNull(pr);        // error checking
    for (unsigned int i = 0; i < nNbrRects; i++)
        pr[i].Draw();     // display each CRectangle
    delete [] pr;         // Release the array memory...
                          // Q: Why the [] notation?
    cout << endl;
    pr = new CRectangle;  // Q: When is this destroyed?
    cout << "\nEnding main()" << endl;
}

void CheckNull(void* pv)
{
    if (pv == NULL)
    {
        cerr << "\nERROR: Heap Allocation Failure!"
             << endl;
        exit(1);
    }
}
```

Figure 11 : Allocation dynamique d'objets

8.8. Les membres statiques

8.8.1. Les données membres statiques

Il arrive que la valeur d'une donnée membre s'applique à tous les membres de la classe. Dans ce cas, il serait inefficace de stocker la même valeur dans tous les objets de la classe. De plus, il peut arriver que l'on désire avoir une donnée relative à une classe plutôt qu'à un objet : par exemple, si on désire compter le nombre d'objet existant dans une classe.

On peut alors déclarer la donnée membre comme statique en ajoutant le mot clé **static** au début de la déclaration de la variable. Celle-ci devra cependant être définie globalement. La syntaxe est la suivante :

```
class CX
      {
      public:
      static int dN;
      …
      }

int CX::dN;
```

Ne pas oublier qu'il n'y a à ce niveau aucune réservation de place mémoire ! On ne fait que déclarer le contenu de la classe.

Puisque une variable statique n'est définie dans aucun objet, il va être nécessaire de la définir (donc lui donner de la place mémoire) globalement

Les variables statiques étant automatiquement initialisées à zéro, une initialisation explicite dans la définition est inutile à moins que l'on veuille avoir une valeur initiale non nulle.

Par exemple :

```
class CRectangle
       {
       CRectangle ()
              {
              m_dCount++;
              }
       ~CRectangle ()
              {
              m_dCount--;
              }
       int getCount ()
              {
              return m_dCount;
              }
       private:
       static int m_dCount;
       };

int CRectangle::m_dCount;

main ()
       {
       CRectangle oW, oX;
       cout << "Il y a maintenant " << oW.getCount() << "CRectangles\n";
       }
```

Afin d'éviter de multiples déclarations , on placera cette déclaration dans le fichier source de la classe.

Exemple 33 : Utilisation d'une donnée statique

Une donnée membre *statique* ressemble à une variable globale ordinaire : il n'existe qu'une seule copie de la variable indépendemment du nombre d'objets existant dans la classe.

8.8.2. Les fonctions membres statiques

Comme toute fonction membre, la fonction *getCount* de l'*Exemple 33* doit être invoquée au départ d'un objet quelconque de la classe *CRectangle*. Peu importe l'objet que l'on utilise. Dans l'exemple cité, on a utilisé l'objet *oW* mais on aurait très bien pu utiliser l'objet *oX*.

En aucun cas, cependant, la fonction *getCount* n'aurait pu être appelée avant la création d'un objet de la classe.

A partir du moment où l'action réalisée par la fonction **est indépendante** des objets réels de la fonction, il serait préférable de ne pas devoir effectuer l'appel de la fonction au départ d'un objet. C'est ce que l'on va pouvoir faire en déclarant la fontion comme **statique** (*static*).

```
class CRectangle
	{
	CRectangle ()
		{
		m_dCount++;
		}
	~CRectangle ()
		{
		m_dCount--;
		}
	static int getCount ()
		{
		return m_dCount;
		}
	private:
	static int m_dCount;
	};

int CRectangle::m_dCount;

main ()
	{
	CRectangle oW, oX;
	cout << "Il y a maintenant " << CRectangle::getCount() <<
	"CRectangles\n";
	}
```

Exemple 34 : Utilisation d'une fonction membre statique

8.9. Les Espaces de noms (NameSpace)

8.9.1. Description générale

Les espaces de nom ont été introduits pour permettre de faire des regroupements logiques et résoudre les collisions de noms. Un espace de nom permet de regrouper plusieurs déclarations de variables, fonctions et classes dans un groupe nommé. Une pratique courante du langage C consistant à concaténer des noms entre eux afin de minimiser la probabilité de collisions avec des noms d'autres bibliothèques n'a plus raison d'être en C++. En effet, un même nom pourra être déclaré dans des espaces de noms différents évitant ainsi la collision lorsqu'ils sont inclus en même temps dans une unité.

La déclaration d'un espace de noms se fait à l'aide du mot clé *namespace*.

Tout comme l'accès à un membre d'une classe, l'opérateur de portée :: permet l'accès à l'ensemble des noms importés d'un espace de noms.

Avant l'introduction des espaces de noms, une pratique du C++ consistait à utiliser des classes pour faire des espaces de noms. Le problème est qu'une classe est instanciable et que sa portée est fermée. A contrario, la portée d'un espace de noms est ouvert, c'est-à-dire qu'il est possible de répartir un espace de noms sur plusieurs unités. Un espace de noms est donc extensible.
Enfin, il est possible d'imbriquer un espace de noms dans un autre espace de noms.

Un exemple d'espace de nom est std, défini par la bibliothèque standard, regroupant en autre les flux standards cin, cout et cerr. Ainsi lorsque <iostream> est inclus dans une source, il sera possible de les accéder en les dénommant par leurs noms complets std::cin, std::cout et std::cerr.

Pour simplifier le code, et les désigner sans spécifier l'espace de noms, il faut utiliser l'instruction suivante :

```
using namespace std;
```

Utiliser différents espaces de noms permet, lors d'un projet en équipe ou de l'utilisation de bibliothèques externes, de créer plusieurs entités portant le même nom.

Pour déclarer vos fonctions, variables et classes dans votre espace de noms, placez-les dans un bloc de ce type :

```
namespace identifiant
{
  // Déclarations ici ...
}
```

Exemple :

```
namespace Exemple
{
    int Suivant(int dN)
    {
        return dN + 1;
     }
}
```

Recommandation : Il est préférable de ne pas utiliser les directives using dans un fichier entête, excepté dans les fonctions inline. En effet, la directive ayant pour effet d'ouvrir l'espace de noms à l'espace global, toute unité qui inclurait ce fichier entête se verrait automatiquement ouvrir l'espace en question pouvant provoquer rapidement des collisions si cette même unité incluait d'autres fichiers entêtes utilisant aussi la directive using sur d'autres espaces de noms.

Autre exemple :

```
namespace a
{
    void toto() // première fonction toto
    {
    }
}
namespace b
{
    void toto() // seconde fonction toto()
    {
    }
}
a::toto(); // appelle la première
b::toto(); // appelle la seconde
```

8.9.2. Espace de nom anonyme

Un namespace anonyme est un espace de nom… sans nom.

```
namespace
{
    int a; // variable déclarée dans un namespace anonyme
}
```

Un namespace anonyme a la particularité de n'être visible que par l'unité de compilation dans laquelle il se trouve (c'est-à-dire le fichier compilable). Son utilité est de permettre la déclaration d'une variable / fonction / type dont la portée doit être celle du fichier.
Parce qu'un namespace traduit mieux la notion de portée restreinte à un fichier, il est recommandé de privilégier son usage en C++ par-rapport à celui du mot-clé static utilisé dans le même but mais en C.

8.9.3. Comment implémenter une fonction définie dans un Namespace ?

Imaginons que nous ayons défini la classe suivante ainsi que deux fonctions non-membres dans un namespace :

```
namespace ns
{
    class Test
    {
    public:
        void F1();
        void F2();
```

```
    };

    void Fonction1();
    void Fonction2();
}
```

Il suffit de procéder de la même manière pour les implémenter, ou bien de préfixer par le nom du namespace :

```
namespace ns
{
    void Test::F1()
    {
    }

    void Fonction1()
    {
    }
}

// autre possibilité

void ns::Test::F2()
{
}

void ns::Fonction2()
{
}
```

8.9.4. À quoi sert le namespace std ?

Toute la bibliothèque standard C++ est définie dans son propre espace de nom, le namespace **std**. Ainsi, il faut systématiquement utiliser le préfixe **std::** devant tous les éléments qui en sont issus.

```
#include <iostream>

int main()
    std::cout << "cout" << std::endl;
}
```

Pour alléger l'écriture, on utilise l'expression using namespace std; ce qui permet de s'affranchir de l'obligation de préfixer par std::. Le code précédent peut alors s'écrire ainsi :

```
#include <iostream>
using namespace std;

int main()
{
    cout << "coucou" << endl;
}
```

On peut aussi utiliser la clause *using namespace* dans un bloc et cela en vue de réduire le risque de collision de noms :

```
include <iostream>

void affiche_coucou()
{
    using namespace std;

    cout << "coucou" << endl;
}

int main()
{
    affiche_coucou();
    std::cout << "coucou" << std::endl;
}
```

8.9.5. Quelques remarques additionnelles

On peut rouvrir un Namspace existant afin d'y ajouter de nouvelles définitions.

```
namespace Alpha
{
    void alpha1();
    void alpha2();
}

namespace Beta
{
    void beta1();
}

namespace Alpha
{
    void alpha3();
}
```

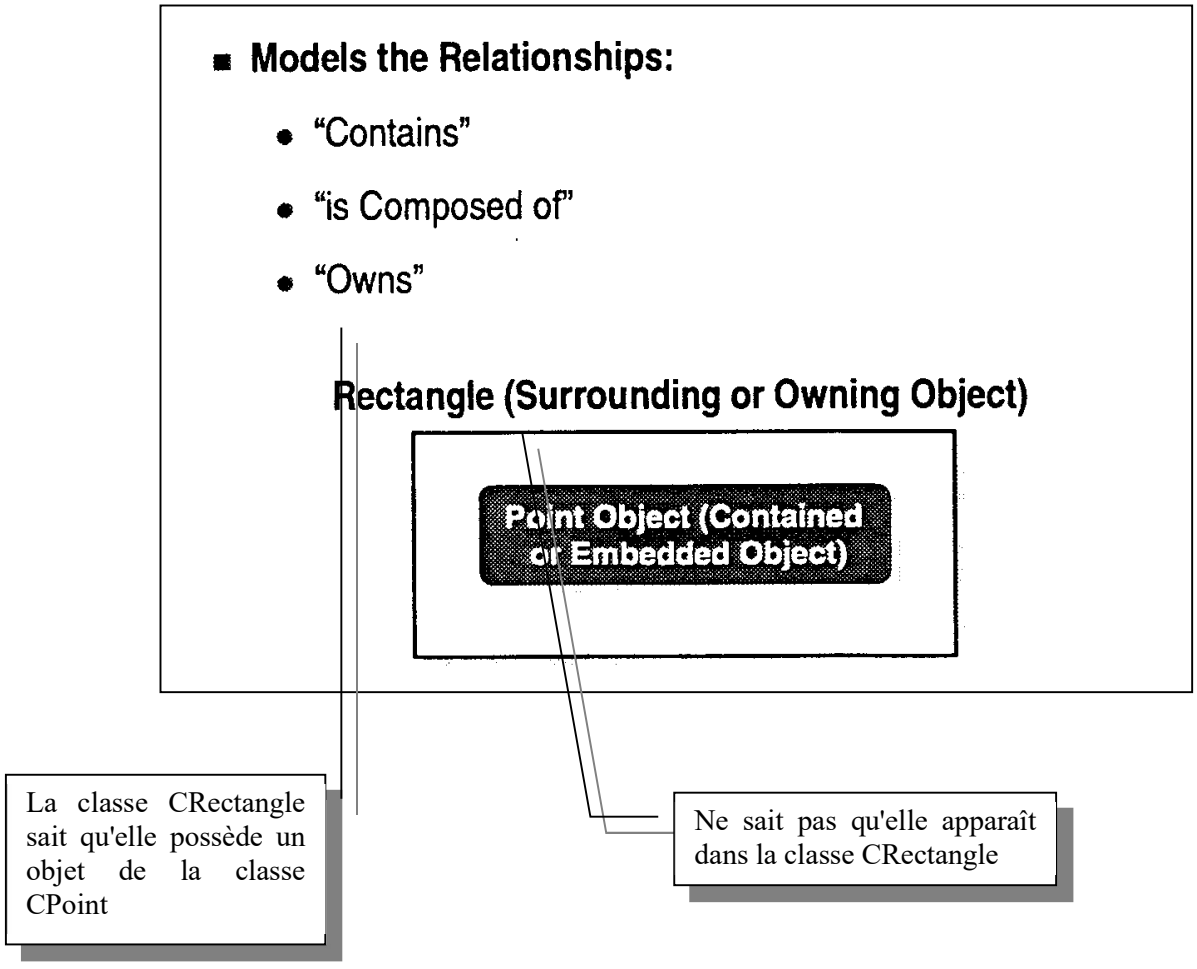
9. L'agrégation

9.1. Définition de la relation Has-A

L'agrégation représente un type de relation **partie-de** (Has a) entre deux classes d'objets. L'agrégation est un concept important car elle va permettre de réutiliser le code écrit par d'autres programmeurs.

Une agrégation se réalise très simplement en déclarant un objet comme membre d'une autre classe. Pour illustrer ce mécanisme, on utilisera dans ce module l'exemple de la classe *CRectangle* qui utilisera un objet de la classe *CPoint* pour matérialiser le centre du CRectangle.

Comme un objet *qui est inclus* l'est comme un membre donnée, il sera déclaré dans la section *private* de la classe. Dans l'exemple du CRectangle, seule cette classe sait qu'elle contient un objet de la classe *CPoint*. Par contre, dans la conception de la classe *CPoint*, on ne doit absolument pas tenir compte du fait que la classe *CPoint* sera utilisée à l'intérieur de la classe *CRectangle*.



9.2. L'ordre d'exécution des constructeurs et destructeurs

Lorsque l'on va déclarer un objet de la classe *CRectangle*, on va commencer par construire l'objet de la classe *CPoint* ; le constructeur de cette classe sera donc exécuté en premier.

Par contre, lors de la suppression d'un objet de la classe *CRectangle*, on commencera d'abord par exécuter le destructeur de la classe *CRectangle* avant d'exécuter le destructeur de la classe *CPoint*.

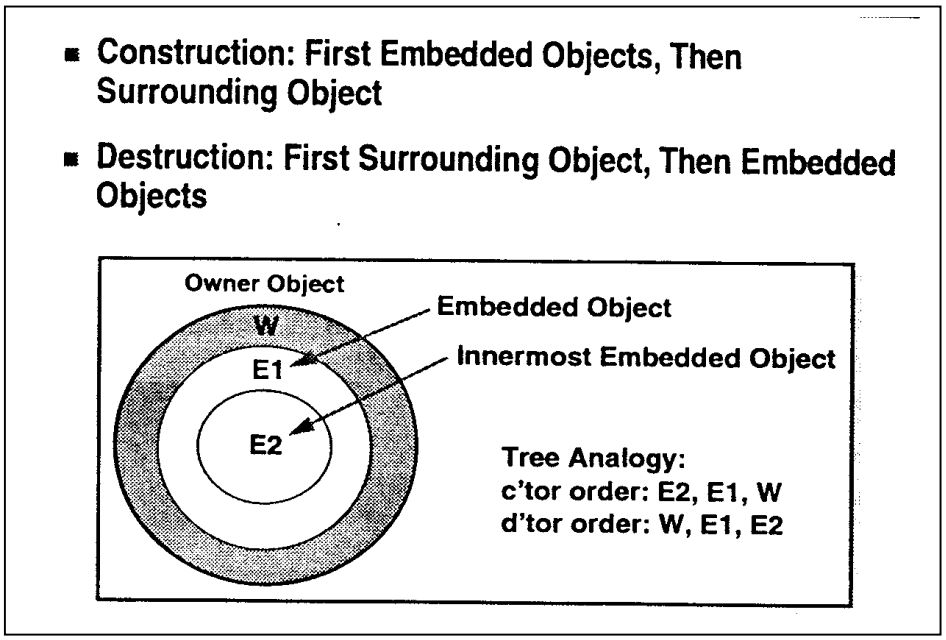


Figure 13 : Ordre d'exécution des constructeurs dans une agrégation

9.3. Exemple

```
// Classes that contain classes use embedding.
#include <iostream>

/****************** CPoint Class ********************
 Declaration and definition since the CPoint class has
 only implicitly inline member functions.
*****************************************************/
class CPoint
{
public:                                    // construction
    CPoint(int x=0, int y=0)
        : m_dX(x), m_dY(y)
        { cout << "CPoint c'tor\n"; }
    ~CPoint()
        { cout << "CPoint d'tor" << endl; }
                                           // attributes
    int Getx(void) { return m_dX;}
    int Gety(void) { return m_dY;}
    void Setx(int x) { m_dX = x; }
    void Sety(int y) { m_dY = y; }
private:                                   // implementation
    int m_dX, m_dY;
};

/************ CRectangle Class Declaration *************/
class CRectangle
{
public:                    // construction
        // Default c'tor creates "point" rect's at 0,0
    CRectangle();
        // 3-arg c'tor (default arg) may invoke CPoint
        // c'tor (and its default copy c'tor) to build
        // a CPoint object at 50,50
    CRectangle(int h, int w, CPoint oP=CPoint(50,50));
    CRectangle(int h, int w, int x, int y);
    ~CRectangle();
                           // attributes
    void SetCenter(CPoint oP);
    CPoint GetCenter(void);
                           // implementation
    void Size(int nh, int nw);
    void Draw(void);
private:
    CPoint m_oCenter;
    int m_dHeight, m_dWidth;
};

/******* CRectangle Member Function Definitions *******/
inline CRectangle::CRectangle()
    : m_dHeight(0), m_dWidth(0), m_oCenter(0,0)
{
    cout << "CRectangle default c'tor\n";
}

inline CRectangle::CRectangle(int h, int w, CPoint oP)
    : m_dHeight(h), m_dWidth(w), m_oCenter(oP)
{
    cout << "CRectangle c'tor: 3 args (int, int, pt)\n";
}

inline CRectangle::CRectangle(int h, int w, int x, int y)
    : m_dHeight(h), m_dWidth(w), m_oCenter(x,y)
{
```

```
    cout << "CRectangle c'tor: 4 args (all int's)\n";
}

inline CRectangle::~CRectangle()
{
    cout << "CRectangle d'tor" << endl;
}

inline void CRectangle::SetCenter(CPoint oP)
{
    m_oCenter = oP;
}

inline CPoint CRectangle::GetCenter(void)
{
    return m_oCenter;
}

void CRectangle::Size(int nh, int nw)
{
    m_dHeight= nh;
    m_dWidth = nw;
}

// Currently just a display function
void CRectangle::Draw(void)
{
    cout << "CRectangle at x:" << m_oCenter.Getx()
         << " y:" << m_oCenter.Gety();
    cout << " height:" << m_dHeight
         << " width:" << m_dWidth;
}

/*************** Simple Test Function ***************/
int main()
{
    cout << "Create oP1:";  // Create a CPoint, oP1, at
    CPoint oP1 (25, 35);     // coordinates 25,35
    cout << endl;
    cout << "Create oR1:";  // Creating oR1 creates a
    CRectangle oR1;          // CPoint default center 0,0
    cout << endl;
    cout << "Create oR2:";    // Create oR2 using oP1 obj
    CRectangle oR2 (1, 2, oP1); // for center at 25,35
    cout << endl;
    cout << "Create oR3:";      // Create oR3 passing
    CRectangle oR3 (8, 8, 9, 9); // c'tor 9, 9 for CPoint
    cout <<"\nNow leaving main()\n";
    Note: The destruction order of non-embedded objects
    with respect to each other is not guaranteed.
    return 0;
}
```

Exemple 35 : La classe CRectangle avec un member de type Cpoint

Dans cet exemple, on peut remarquer une liste d'initialisation dans chacun des deux constructeurs :

```
inline CRectangle::CRectangle(int h, int w, CPoint oP)
    : m_dHeight(h), m_dWidth(w), m_oCenter(oP)
{
    cout << "CRectangle c'tor: 3 args (int, int, pt)\n";
}

inline CRectangle::CRectangle(int h, int w, int x, int y)
```

```
: m_dHeight(h), m_dWidth(w), m_oCenter(x,y)
```

La présence de ces listes d'initialisation est fort intéressante car elles informent le compilateur sur la manière de construire l'objet *m_oCenter*. Sans la présence de ces listes d'initialisation, le compilateur aurait invoqué le constructeur par défaut de la classe CPoint.

9.4. Exercice

Définir et tester une classe *Personne* dont les membres sont les suivants :

- L'identification de la personne : cela devra être un objet de la classe *Nom* (voir sa description juste après)
- La date de naissance et de décès : objets de la classe *Date* (*voir 7.9.5 La classe Date, page 65)*
- Le sexe de la personne de type *int* (0 = féminin et 1 = masculin).
- La nationalité de la personne : objet de la classe *CChaine* (*voir 7.9.3 La classe , page 63*).
- Une adresse : objet de la classe *Adresse* (voir sa description juste après)
- Le téléphone : objet de la classe Telephone (voir sa description juste après)
- Un ou plusieurs constructeurs
- Une fonction d'affichage des données de la personne
- D'autres fonctions qui pourraient être nécessaires

La classe *Nom* est composée des membres suivants :

- Le nom, le prénom, titre, surnom : objets de la classe *CChaine* (*voir 7.9.3 La classe , page 63*).
- Un ou plusieurs constructeurs
- La surcharge des opérateurs d'insertion et d'extraction
- D'autres fonctions éventuelles qui pourraient être nécessaires.

La classe *Adresse* est composée des membres suivants :

- L'adresse, le code postal, la ville, le pays : objets de la classe *CChaine* (*voir 7.9.3 La classe , page 63*).
- Le constructeur
- La surcharge des opérateurs d'insertion et d'extraction
- Toute autre fonction qui pourrait être utile.

La classe *Telephone* est composée des membres suivants :

- Le préfixe et le numéro : objets de la classe *CChaine* (*voir 7.9.3 La classe , page 63*).
- Le constructeur
- La surcharge des opérateurs d'insertion et d'extraction
- Toute autre fonction qui pourrait être utile

10. L'héritage

10.1. Introduction

Autre type de relation, la relation *IS A* va permettre de créer une hiérarchie entre les classes. L'héritage va être la possibilité de définir une nouvelle classe par extension d'une classe existante. La classe ainsi construite (**la classe dérivée**) va hériter de toutes les propriétés de la classe dont elle est dérivée ; elle hérite de tous les membres de la classe parent **ou classe de base**.

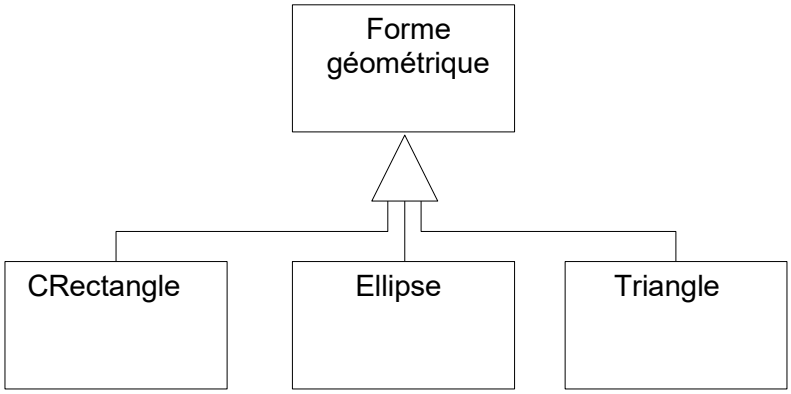


Figure 14 : L'héritage simple

A la figure précédente, on est en présence :

- d'une classe de base, la classe *Forme Géométrique*
- de trois classes dérivées : les classes *CRectangle, Ellipse et Triangle*
- d'une relation de type IS-A : un triangle est une forme géométrique, un CRectangle est une forme géométrique et idem pour l'éllipse.

D'un point de vue vocabulaire :

- une classe de base porte parfois les noms de : classe parent ou de superclasse.
- une classe dérivée porte parfois les noms de : classe enfant ou de sousclasse.

Une classe dérivée hérite de tous les members de la classe de base. Cependant, la classe dérivée aura des caractéristiques supplémentaires par rapport à sa classe de base où elle pourra modifier certaines caractéristiques de la classe de base. Une classe dérivée pourra donc :

- se définir de nouveaux members
- modifier la signification de certaines fonctions membres de la classe de base, en les **substituant**.

10.2. La syntaxe

La définition d'une classe de base se réalise comme vu jusqu'à présent. La définition d'une classe dérivée s'effectue très simplement comme suit :

`class` **`ClasseDérivée : public ClasseDeBase`** → le plus courant

```
ou

class ClasseDérivée : private ClasseDeBase
```

Par exemple :

```
class CRectangle : public CFormGeo
      {
      …
      }
```

Dans un héritage **public**, les membres *public* et *protected* de la classe de base restent *public* et *protected* dans la classe dérivée.

Dans un héritage **private**, les membres *public* et *protected* de la classe de base deviennent des membres *private* de la classe dérivée.

Les membres *private* d'une classe ne font jamais partie d'un héritage.

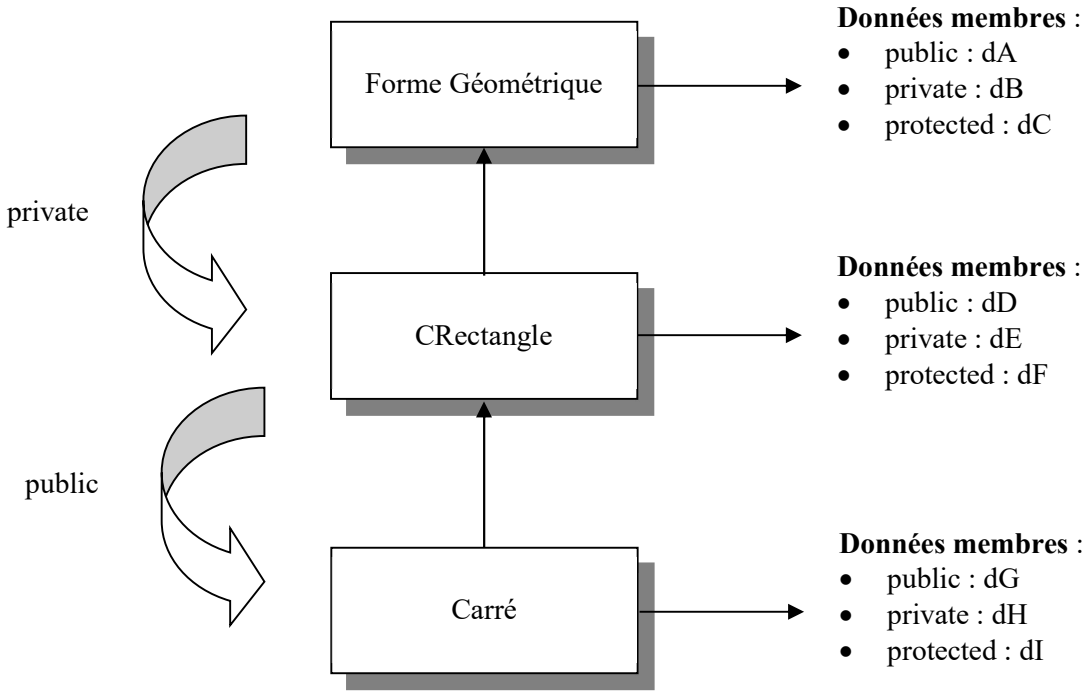


Figure 15 : Héritage public et héritage private

Etant donné la construction des trois classes ci-dessus, quelles sont les données membres accessibles à chacune des classes ?

Classe	Données membres accessibles
Classe Forme Géométrique	La classe peut accéder à toutes ses données membres : dA, dB et dC
Classe CRectangle	La classe peut accéder à toutes ses données membres : dD,dE et dF Par héritage, elle acquiert les données membres dA et dC. Elle ne peut pas accéder à dB car cette donnée est déclarée dans la section *private* de la classe de base. Comme on utilise l'héritage *private*, les données héritées deviennent *private*.
Classe CCarre	La classe peut accéder à toutes ses données membres : dG, dH et dI. Par héritage, elle acquiert les données membres dD et dF. Elle ne peut pas accéder à dE car il s'agit d'une donnée déclarée dans la section *private* de la classe de base. Mais, en aucun cas, elle ne peut accéder aux données membres de la classe Forme Géométrique car suite à l'héritage *private* entre les deux classes FG et Rectangle, elles sont considérées comme *private*

10.3. Substitution et dominance des membres hérités

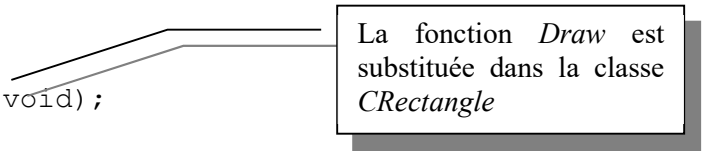
Lors d'un héritage, il est fort possible qu'une fonction membre de la classe de base ne satisfasse pas la classe dérivée. De par la nature de la classe de base, cette fonction membre est peut-être trop générale et a peut-être besoin d'être spécialisée au niveau de la classe dérivée. Comme l'opération qui doit être ainsi spécialisée est logiquement identique à la fonction de la classe de base, il serait bon de pouvoir donner à la fonction de la classe dérivée le même nom que celui de la classe de base. C'est ce qui va être fait … On va **substituer** la fonction de la classe de base par celle de la classe dérivée.

```
class CFormGeo
       {
       public:

       void draw (void);
       …
       };

class CRectangle : public CFormGeo
       {
       public:

       void draw (void);
       …
       };
```



En écrivant,

```
oRect.draw ();
```

On accèdera à la version *draw ()* de la fonction dérivée. Par contre, en écrivant :

```
oRect.CFormGeo::draw()
```

On accèdera à la fonction *draw ()* de la classe de base.

De même, dans certains cas, on souhaite peut-être définir une version locale d'un membre hérité. Par exemple, si *a* est une donnée membre de *X* et si *Y* est une sous-classe de *X*, on pourrait aussi définir une donnée membre de *Y* appelée *a*. Dans ce cas, on dit que la donnée dans ***Y*** **domine** la donnée *a* définie dans *X*.

Par conséquent, si *y* est un objet de la classe *Y*, alors

```
y.a
```

fera référence au member *a* de la classe *Y*. Par contre :

```
y.X::a
```

fera référence à l'objet *a* de la classe *X*.

Attention ! Il ne faut pas confondre **surcharge** (*overloading*) et **substitution** (*overriding*). La *surcharge* a lieu à l'intérieur d'une même classe et le compilateur différentie les fonctions de par leur signature. Tandis que la *substitution* a lieu entre classe de base et classe dérivée.

Dans la fonction substituée, il sera aussi parfois utile d'invoquer la fonction de la classe de base. Dans ce cas, on écrira ce qui suit :

```
CRectangle::Draw()
{
        … écriture du code spécifique à la fonction Draw()
        de la classe CRectangle

        CFormGeo::Draw();
}
```

10.4. L'ordre d'invocation des constructeurs et destructeurs

On commence par construire l'objet de la classe de base et ensuite celui de la classe dérivée. Si l'un de ces objets contient un autre objet, ce dernier sera construit en premier. En résumé :

1. On commence par construire l'objet de la classe de base :
   - On commence par construire les objets qu'il contient
   - Ensuite, on construit l'objet en lui-même
2. On continue par la construction de l'objet de la classe dérivée :
   - On commence par construire les objets qu'il contient
   - Ensuite, on construit l'objet en lui-même

Pour forcer l'usage d'un constructeur de la classe de base, on peut utiliser la liste d'initialisation en y plaçant le nom de la classe de base suivi par les paramètres d'initialisation.

Reprenons l'exemple de la classe *CRectangle* et supposons que l'on déclare un objet comme suit :

```
CRectangle oR1 (2, 4, 150, 150) ;
```

On va invoquer le constructeur de la classe *CRectangle* qui correspond aux paramètres communiqués. Dans le cas de notre classe, on invoquera le constructeur suivant :

```
CRectangle (int h, int w, int x, int y) : CGeoShape (x,
y) , m_dHeight (h) , m_dWidth (w)
      {
      …
      }
```

La partie *CGeoShape* de l'objet *oR1* doit être construite avant la construction de l'objet *CRectangle* ; aussi, on va faire appel au constructeur de la classe *CGeoShape* en lui communiquant les deux paramètres *x* et *y*. Le constructeur invoqué est le suivant :

```
CGeoShape (int x , int y) : m_oCenter (x , y)
      {
      …
      }
```

A son tour, avant de construire l'objet de la classe *CGeoShape*, il faut construire l'objet *CPoint* qui est inclus dans cette classe. Le constructeur de la classe *CPoint* utilisé est le suivant :

```
CPoint (int x , int y) : m_dX(x) , m_dY (y)
        {
        …
        }
```

Ensuite, il reste a exécuter :

- Le corps du constructeur CPoint
- Le corps du constructeur CGeoShape
- Le corps du constructeur CRectangle

En ce qui concerne l'ordre d'invocation des **destructeurs**, c'est beaucoup plus simple puisqu'il n'y a pas de paramètres communiqués. L'ordre d'invocation des **destructeurs** est exactement l'ordre inverse de celui des constructeurs. On commence par détruire les objets de la classe dérivée avant de détruire l'objet de la classe de base. Pour rappel, s'il y a des objets contenus dans d'autres, on commence par détruire l'objet contenant et puis l'objet qui est inclus.

On détruira donc :

- L'objet CRectangle car il appartient à une classe dérivée.
- L'objet CGeoShape car il appartient à une classe de base ; de plus, comme il contient un objet de la classe CPoint, on commencera par détruire l'objet contenant … donc, l'objet CGeoShape.
- Enfin, on détruira l'objet CPoint car il est contenu dans l'objet CGeoShape.

10.5. Exemple

```
// CGeoShape has an embedded CPoint.  CRectangle inherits
// from CGeoShape and calls base member functions.
#include <iostream>

/***** Declaration and Definition of CPoint Class *****/
class CPoint
{
public:                              // construction
    CPoint(int dX=0, int dY=0)
        : m_dX(dX), m_dY(dY)
        { cout << "CPoint c'tor\n"; }
    ~CPoint()
        { cout << "CPoint d'tor" << endl; }
    int Getx(void) { return m_dX;} // attributes
    int Gety(void) { return m_dY;}
    void Setx(int dX) { m_dX = dX; }
    void Sety(int dY) { m_dY = dY; }
private:                             // implementation
    int m_dX, m_dY;
};

/********** CGeoShape Class Declaration ***************
 Base class for the 2-D geometrical classes CRectangle,
 Ellipse, and Triangle.  Dimensions do not make sense
 for a generic shape, but a center point does.
*****************************************************/
class CGeoShape
{
public:                             // construction
    CGeoShape(CPoint oP=CPoint(0,0));
    CGeoShape(int dX, int dY);
    ~CGeoShape();
    void SetCenter(CPoint oP);      // attributes
    CPoint GetCenter(void);
    void Draw(void);                // operations
private:                            // implementation
    CPoint m_oCenter; // CPoint is "embedded" in CGeoShape
};

/************ CRectangle Class Declaration *************/
class CRectangle : public CGeoShape  // Inheritance!
{
public:                             // construction
    CRectangle();
    CRectangle(int dH, int dW, CPoint oP=CPoint(50,50));
    CRectangle(int dH, int dW, int dX, int dY);
    ~CRectangle();
    void Size(int dH, int dW);    // operations
    void Draw(void);
private:                            // implementation
    int m_dHeight, m_dWidth;
};

/******* CGeoShape Member Function Definitions ********/
inline CGeoShape::CGeoShape(CPoint oP)
    : m_oCenter(oP)
{
    cout << "CGeoShape c'tor: 1 arg\n";
}

inline CGeoShape::CGeoShape(int dX, int dY)
    : m_oCenter(dX,dY)
{
    cout << "CGeoShape c'tor: 2 arg\n";
```

```
}

inline CGeoShape::~CGeoShape()
{
    cout << "CGeoShape d'tor" << endl;
}

inline void CGeoShape::SetCenter(CPoint p)
{
    m_oCenter = p;
}

inline CPoint CGeoShape::GetCenter(void)
{
    return m_oCenter;
}

/* Currently just a display function */
void CGeoShape::Draw(void)
{
    cout << "Center at x:" << m_oCenter.Getx()
         << " y:" << m_oCenter.Gety() << endl;
}

/******* CRectangle Member Function Definitions *******/
inline CRectangle::CRectangle()
    : m_dHeight(0), m_dWidth(0), CGeoShape(0,0)
{
    cout << "CRectangle default c'tor\n";
}

inline CRectangle::CRectangle(int dH, int dW, CPoint oP)
    : m_dHeight(dH), m_dWidth(dW), CGeoShape(oP)
{
    cout << "CRectangle c'tor: 3 arg (int, int, Pt)\n";
}

inline CRectangle::CRectangle(int dH, int dW, int dX, int dY)
    : m_dHeight(dH), m_dWidth(dW), CGeoShape(dX,dY)
{
    cout << "CRectangle c'tor: 4 arg (all int's)\n";
}

inline CRectangle::~CRectangle()
{
    cout << "CRectangle d'tor" << endl;
}

void CRectangle::Size(int dH, int dW)
{
    m_dHeight= dH;
    m_dWidth = dW;
}

/* Currently just a display function */
void CRectangle::Draw(void)
{
    CGeoShape::Draw();    // :: used for qualification
    cout << " height:" << m_dHeight
         << " width:" << m_dWidth;
}

/*************** Small Test Program ****************/
void main()
{
    cout << "Create p:";
    CPoint oP (55, -55);
    cout << "Creating two generic objects:\n";
```

```
    CGeoShape oG1, oG2 (12, -12);
    cout << "Creating three CRectangles:\n";
    CRectangle oR1 (2, 4, 150, 150),
               oR2 (10, 10, oP),
               oR3 (55, 55);
    cout<<"\n\"Draw\" two objects:\n";
    cout <<"g1 draws as: \n";
    oG1.Draw();
    cout <<"oR2 draws as: \n";
    oR2.Draw();
    cout << "\nEnding main()" << endl;
}
```

Exemple 36 : Héritage publique

Exercices au départ de cet exemple :

1) Précisez la liste des messages qui seront affichés suite à la déclaration suivante :

```
CGeoShape oG1 ;
```

2) Précisez la liste des messages qui seront affichés suite à la déclaration suivante :

```
CRectangle oR2(10, 10, oP) ;
```

10.6. Exercice

Exercice PRJ012 : L'héritage

On dispose d'un fichier nommé *Point* contenant la déclaration suivante de la classe *CPoint* :

```
class CPoint
{
	int m_dX, m_dY;

public:

	CPoint (int dAbs, int dOrd)
	{
		m_dX = dAbs;
		m_dY = dOrd;
	}

	void Affiche ()
	{
		cout << "Coordonnées : " << m_dX << " " << m_dY << endl;
	}

	void Deplace (float fX, float fY)
	{
		m_dX += fX;
		m_dY += fY;
	}
};
```

A. Créer une class *CPointCol* , dérivée de *CPoint*, comportant :

- Une donné membre supplémentaire, couleur de type int, contenant la couleur d'un point.
- Les fonctions membres suivantes :

  Affiche (redéfinie), qui affiche les coordonnées et la couleur d'un objet de type *CPointCol*.
  Colore (int couleur), qui permet de définir la couleur d'un objet de type *CPointCol*
  Un constructeur permettant de définir la couleur et les coordonnées d'un objet de la classe *CPointCol*.

B. Que fera alors précisément l'instruction suivante :

```
CPointCol (2, 3, 5) ;
```

10.7. Le polymorphisme

Une des caractéristiques les plus puissantes du langage C++ est de permettre à des objets de classes différentes de répondre différemment à un même appel de fonction. C'est ce qu'on appelle le **polymorphisme** que l'on réalise grâce aux fonctions **virtuelles**. Le polymorphisme est possible parce qu'un pointeur d'une classe de base peut recevoir l'adresse d'un objet appartenant à une classe dérivée.

Afin de bien comprendre cette notion de *polymorphisme*, il est utile de repartir de l'exemple précédent. Nous y avions défini les deux classes *CRectangle* et *CGeoShape*.

```
class CGeoShape
{
public:                          // construction
    CGeoShape(CPoint oP=CPoint(0,0));
    CGeoShape(int dX, int dY);
    ~CGeoShape();
    void SetCenter(CPoint oP);     // attributes
    CPoint GetCenter(void);
    void Draw(void);             // operations
private:                         // implementation
    CPoint m_oCenter; // CPoint is "embedded" in CGeoShape
};

/*********** CRectangle Class Declaration *************/
class CRectangle : public CGeoShape  // Inheritance!
{
public:                          // construction
    CRectangle();
    CRectangle(int dH, int dW, CPoint oP=CPoint(50,50));
    CRectangle(int dH, int dW, int dX, int dY);
    ~CRectangle();
    void Size(int dH, int dW);   // operations
    void Draw(void);
private:                         // implementation
    int m_dHeight, m_dWidth;};
```

Par rapport à cette définition, on peut constater que la fonction *draw* de la classe *CGeoShape* est substituée dans la classe *CRectangle*. Prenons en bonne note pour la suite.

Supposons maintenant que nous nous définissions un pointeur comme suit :

```
CGeoShape *p;
```

Une caractéristique importante de ce pointeur est qu'il va pouvoir recevoir l'adresse d'un objet de la classe *CGeoShape* mais aussi l'adresse d'un objet de la classe *CRectangle*. On peut en déduire le principe suivant :

> Un pointeur définit comme pointant sur une classe de base va pouvoir recevoir une adresse des objets de cette classe mais aussi **de tous** les objets appartenants aux classes dérivées.

Définissons effectivement un objet dans chacune des classes :

```
CGeoShape *p;
CGeoShape oGeo;
CRectangle oRect;
```

Nous pouvons donc écrire :

```
p = &oGeo;
```

Dans ce cas, si on écrit l'expression suivante, que va t-on dessiner ?

```
p->draw ();
```

Nous allons bien sûr dessiner **une forme géométrique** puisqu'il s'agira bien de la fonction *draw* de la classe *CGeoShape* qui sera invoquée. Maintenant, avec les expressions qui suivent, on peut se poser la même question et constater que la réponse est déjà moins évidente.

```
p = &oRect;
p->draw ();
```

Dans cette situation, ce qui sera dessiné à l'écran sera … toujours une forme géométrique ! Techniquement, on peut comprendre puisque le pointeur est de type *CGeoShape* * mais logiquement, on aurait certainement aimé que ce soit la fonction *draw* de la classe *CRectangle* qui soit invoquée.

La logique va pouvoir rejoindre la technique tout simplement grâce à la notion de **polymorphisme**. On va définir que la fonction *draw* est une fonction *virtuelle* grâce au mot réservé **virtual**. Ce mot réservé placé devant le prototype de la fonction dans la classe de base indique au compilateur qu'il a à faire à une fonction virtuelle qui est suceptible d'être substituée dans une ou plusieurs des classes dérivées. Dans les classes dérivées, l'emploi du mot *virtual* n'est plus obligatoire mais il sera conseillé en guise de documentation.

```
class CGeoShape
{
public:                             // construction
    CGeoShape(CPoint oP=CPoint(0,0));
    CGeoShape(int dX, int dY);
    ~CGeoShape();
    void SetCenter(CPoint oP);      // attributes
    CPoint GetCenter(void);
    virtual void Draw(void);        // operations
private:                            // implementation
    CPoint m_oCenter; // CPoint is "embedded" in CGeoShape
};

/*********** CRectangle Class Declaration *************/
class CRectangle : public CGeoShape  // Inheritance!
{
public:                             // construction
    CRectangle();
    CRectangle(int dH, int dW, CPoint oP=CPoint(50,50));
    CRectangle(int dH, int dW, int dX, int dY);
    ~CRectangle();
    void Size(int nh, int dW);      // operations
    virtual void Draw(void);
private:                            // implementation
    int m_dHeight, m_dWidth;
};
```

Le réemploi du mot *virtual* dans la classe dérivée n'est plus obligatoire mais est conseillé.

Maintenant que la fonction *draw* est virtuelle, l'utilisation de l'expression suivante :

```
p = &oRect;

p->draw ();
```

invoquera automatiquement la bonne fonction *draw* suivant l'adresse que l'on aura placé dans le pointeur. Cette réalisation est faisable grâce à un *lien dynamique* parce que ce n'est qu'à l'exécution que l'on connaît quelle fonction il faut appeler.

Suppossons, que l'on définisse d'autres classes dérivées de la classe *CGeoShape* :

```
class CCircle : public CGeoShape
      {
      …
      virtual void draw ();
      …
      }

class CSquare : public CGeoShape
      {
      …
      virtual void draw ();
      …
      }
```

La grande utilité des fonctions virtuelles va apparaître encore plus clairement dans la petite opération qui suit :

```
int main (int)
       {
       CGeoShape *tpGeo [25];
       int dInd;

       /* On suppose que le programme va initialiser les */
       /* 25 pointeurs avec des adresses d'objets carre, rectangle et cercle
*/
       /* appartenant aux différentes classes énumérées. */

       for (dInd = 0 ; dInd < 25 ; dInd++)

              tpGeo [dInd] -> draw ();

       …

       }
```

On peut imaginer toute la puissance de cette boucle puisqu'elle va être capable d'invoquer la bonne fonction *draw* pour chaque adresse se trouvant dans le tableau.

Comme exemple plus complet, voici une version simplifiée d'un *SpreadCheet* (feuille de calcul). On va utiliser une classe de base pour rassembler les caractéristiques communes de chaque cellule. On va lui en dériver deux autres :

- une classe dérivée pour préciser les cellules de type texte
- une classe dérivée pour préciser les cellules de type numérique

La fonction *display ()* sera virtuelle car on désire utiliser cette fonction pour n'importe quel type de cellule. Bien entendu, le corps de la fonction *display ()* sera différent selon le type des cellules traité.

Quelques propriétés :

- si on s'attend qu'une fonction soit substituée dans une classe dérivée, alors il faut la déclarer comme virtuelle dans la classe de base.
- il est souvent utile de déclarer le destructeur de la classe de base comme fonction virtuelle (à ce sujet, voir le paragraphe suivant).
- on peut déclarer n'importe quelle fonction membre d'une classe comme virtuelle et ce en nombre quelconque.
- les fonctions virtuelles sont des fonctions qui peuvent être substituées et surchargées.

Enfin, les fonctions virtuelles peuvent aussi être invoquées au départ de référence. Donc :

- Une référence vers une classe de base peut recevoir les coordonnées d'un objet dérivé de cette classe.
- Au départ d'une référence vers une classe de base, on peut invoquer des fonctions virtuelles définies dans cette classe de base.

```
class CCell
	{
	int m_dX , m_dY;

	public:
	CCell (int , int);
	virtual ~CCell ();
		{
		}
	virtual void display();
	};

CCell::CCell (int dCoordX , int dCoordY) : m_dX (dCoordX), m_dY(dCoordY)
	{
		M_dX = dCoordX ;
	}

void CCell::display ()
	{
	cout << "Dessiner une cellule";
	}

class CTextCell : public CCell
	{
	char *m_pTexte;

	public:
	CTextCCell (const char * , int , int);
	~CTextCCell ();
	virtual void display ();
	}

CTextCCell::CTextCell (const char *pContent , int dX , int dY) : CCell (dX,
	dY)
	{
	m_pTexte = new char [(strlen (pContent) + 1)];
	strcpy (m_pTexte , pContent);
	}

CTextCell::~CTextCell ()
	{
	delete [ ] m_pTexte;
	}

void CTextCell::display()
	{
	CCell::display ();
	cout << m_pTexte << endl;
	}

class CNumCell : public CCell
	{
	float m_fNum;

	public:
	CNumCell (float , int , int);
	~CNumCell ()
		{
		}
	virtual void display ();
	};


CNumCell::CNumCell (float fVal, int dX , int dY) : CCell (dX , dY)
	{
	m_fNum = fVal;
	}
```

```
CNumCell::display ()
       {
       CCell::display ();
       cout << m_fNum << endl;
       }

int main ()
       {
       const int dNbrCells = 5;

       CCell *tpSpreadS [dNbrCells];

       tpSpreadS [0] = new CNumCell (7.00 , 1 , 0);
       tpSpreadS [1] = new CNumCell (4.59 , 3 , 4);
       tpSpreadS [2] = new CNumCell (3.98 , 2 , 2);
       tpSpreadS [3] = new CTextCell ("First" , 4 , 5);
       tpSpreadS [4] = new CTextCell ("Second" , 1 , 1);

       for (dInd = 0 ; dInd < dNbrCells ; dInd++)
              {
              tpSpreadS [dInd]->display ();
              delete tpSpreadS [dInd];
              }
       }
```

Exemple 37 : Les fonctions polymorphiques

10.7.1. Exercices

Exercice PRJ013 :Le polymorphisme

Quel est le résultat du code ci-dessous ?

```
#include <iostream>

class CPoint
{
protected:
	int m_dX, m_dY;

public:
	CPoint (int dAbs = 0, int dOrd = 0)
	{
		m_dX = dAbs;
		m_dY = dOrd;
	}

	virtual void Affiche()
	{
		cout << "Je suis un point \n";
		cout << " mes coordonnées sont : " << m_dX << " " << m_dY <<
			endl;
	}
};

class CPointCol : public CPoint
{
	short m_dCouleur;
public:
	CPointCol (int dAbs = 0, int dOrd = 0, short dC = 1) : CPoint(dAbs, dOrd)
	{
		m_dCouleur = dC;
	}

	void Affiche ()
	{
		cout << "Je suis un point coloré \n";
		cout << " mes coordonnées sont : " << m_dX << " " << m_dY <<
			endl;
		cout << " ma couleur est :	" << m_dCouleur << endl;
	}
};

main ()
{
	CPoint oP(3, 5);
	CPoint *pP = &oP;

	CPointCol oPC (8, 6, 2);
	CPointCol *pPC = &oPC;

	pP->Affiche();
	pPC->Affiche();

	cout << "-----------------------------" << endl;

	pP = pPC;

	pP->Affiche();
	pPC->Affiche();
}
```

Exercice PRJ014 : Le polymorphisme
Quels résultats produira ce programme :

```
#include <iostream>

class CPoint
{
protected:
       int m_dX, m_dY;

public:
       CPoint (int dAbs = 0, int dOrd = 0)
       {
           m_dX = dAbs;
           m_dY = dOrd;
       }

       virtual void Affiche()
       {
           Identifie();
           cout << " mes coordonnées sont : " << m_dX << " " << m_dY << endl;
       }

       virtual void Identifie()
       {
           cout << "Je suis un point \n";
       }

};

class CPointCol : public CPoint
{
       short m_dCouleur;
public:
       CPointCol (int dAbs = 0, int dOrd = 0, short dC = 1) : CPoint(dAbs,
dOrd)
       {
           m_dCouleur = dC;
       }

       void Identifie ()
       {
           cout << "Je suis un point coloré de couleur : " << m_dCouleur <<
endl;
       }
};

main ()
{
       CPoint oP(3, 4);
       CPointCol oPC (5, 9, 5);

       oP.Affiche();
       oPC.Affiche();
       cout << "--------------------" << endl;

       CPoint *pP = &oP;
       CPointCol *pPC = &oPC;

       pP->Affiche();
       pPC->Affiche();
       cout << "------------------------------" << endl;

       pP = pPC;
       pP->Affiche();
       pPC->Affiche();}
```

Exercice PRJ015 : Le polymorphisme

On souhaite créer une classe nommée *CSetType* permettant de manipuler des ensembles dont le type des éléments est non seulement inconnu de la classe, mais également susceptible de varier d'un élément à un autre. Pour que la chose soit possible, on imposera simplement la contrainte suivante : tous les types concernés devront dériver d'un même type de base nommé *CInterfaceSetType*. Le type *CInterfaceSetType* sera supposé connu au moment où l'on définit la classe *CSetType*.

La classe *CInterfaceSetType* disposera d'une fonction virtuelle pure nommée *Affiche()* ; cette fonction devra être redéfinie dans les classes dérivées pour afficher les caractéristiques de l'objet concerné.

La classe *CSetType* disposera des fonctions membres suivantes :

- *Ajoute(CInterfaceSetType *)* pour ajouter un nouvel élément à l'ensemble (elle devra s'assurer qu'il n'existe pas déjà).
- *Appartient (CInterfaceSetType *)* pour tester l'appartenance d'un élément à l'ensemble
- *Cardinal()* qui fournira le nombre d'éléments de l'ensemble.
- *Display()* qui affiche le contenu de l'objet. Si l'object contient des éléments de types différents alors, il serait intéressant de prévoir un mécanisme permettant d'afficher uniquement les objets souhaités. On pourrait par exemple ajouter un paramètre à la fonction *Display()* qui est tout simplement un objet dont on souhaite afficher les objets du même type contenus dans la liste. On utilisera la fonction *Type()* en vue de retrouver le type de l'objet passé en paramètre.

De plus, la classe *CSetType* devra être munie d'un *itérateur*, c'est à dire d'un mécanisme permettant de parcourir les différents éléments de l'ensemble. On prévoira 3 fonctions :

- *Init()* pour initialiser le mécanisme d'itération.
- *CInterfaceSetType * Suivant()* qui fournira en retour le prochain élément (objet d'un type dérivé de *CInterfaceSetType*)
- *bool Existe()* pour préciser s'il existe encore un élément non examiné.

Il faudra réaliser un petit programme d'essai de la classe *CSetType*, en créant un ensemble comportant des objets *CPoint* (deux coordonnées entières) et *CComplexe* (une partie réelle et une partie imaginaire de type *float*). Ces deux classes devront être dérivées de *CInterfaceSetType*.

En outre, la fonction *Ajoute(CInterfaceSetType *)* ne pourra pas mémoriser tel quel l'adresse reçue en paramètre. Elle devra faire appel à la fonction virtuelle pure *Allocate()* de la classe *CInterfaceSetType*.

Pour cet exercice, plusieurs niveaux de difficulté sont envisageables (les voici de la plus simple à la plus complexe) :

1) L'objet CSetType mémorise toujours un même type d'objet et l'adresse *CInterfaceSetType* * reçue en paramètre dans la fonction *Ajoute()* est mémorisée dans l'objet lui-même. On utilise un tableau de taille fixe pour mémoriser les objets.
2) L'objet CSetType mémorise toujours un même type d'objet et l'adresse *CInterfaceSetType* * reçue en paramètre dans la fonction *Ajoute()* n'est pas mémorisée dans l'objet lui-même. Dans la classe *CInterfaceSetType*, il faut donc ajouter une fonction virtuelle pure d'allocation d'un objet.

3) L'objet CSetType peut mémoriser des objets de type différent et l'adresse *CInterfaceSetType* * reçue en paramètre dans la fonction *Ajoute()* est mémorisée dans l'objet lui-même.
4) L'objet CSetType peut mémoriser des objets de type différent et l'adresse *CInterfaceSetType* * reçue en paramètre dans la fonction *Ajoute()* n'est pas mémorisée dans l'objet lui-même. Dans la classe *CInterfaceSetType*, il faut donc ajouter une fonction virtuelle pure d'allocation d'un objet.
5) L'objet CSetType implémente une liste pour mémoriser les différents objets.
6) L'object CSetType mémorise les objets d'un même type en séquence de telle sorte que si on affiche l'objet dans son entièreté, les objets de types différents sont affichés en séquences.

Pour permettre l'allocation dynamique des objets à insérer dans l'objet CSetType, il s'agit de définir une fonction virtuelle pure à l'intérieur de la classe *CInterfaceSetType* avec le profil suivant :

```
CInterfaceSetType * Allocate() ;
```

Dans la fonction *Ajoute()* de la classe CSetType, on va trouver le code suivant :

```
CSetType::Ajoute(CInterfaceSetType *pNew)
{
 CInterfaceSetType *pTemp ;

 pTemp = pNew->Allocate() ;

 …
}
```

Dans la chacune des classes dérivées de *CInterfaceSetType*, on va trouver une implémentation de la fonction *Allocate()* :

```
CInterfaceSetType * CDerived::Allocate()
{

 CDerived *pTemp ;

 pTemp = new CDerived (*this) ;

 return pTemp ;
}
```

Pour gérer des objets de type différent à l'intérieur de l'objet CSetType, on va se trouver confronté à un problème de taille. En effet, prenons par exemple la fonction *Ajoute()* chargée d'ajouter un nouvel objet à l'intérieur de l'objet CSetType. Cette insertion ne peut se faire qu'à la condition que l'objet à insérer n'existe pas déjà dans l'objet CSetType.

Pour faire cette vérification, il faut comparer l'objet à insérer avec chacun des objets se trouvant dans l'objet CSetType. Cependant, cette comparaison n'a de sens que si on compare des objets de même type.

Dans la classe *CInterfaceSetType*, on va ajouter une fonction virtuelle pure *Type()* qui va permettre de retourner le type de l'objet. Dans chaque classe dérivée de *CInterfaceSetType*, on va trouver l'implémentation de cette fonction comme suit :

```
int CDerived::Type()
{
      return numeric value ;
}
```

La valeur numérique à retourner doit être une valeur unique parmi toutes les classes qui peuvent se rattacher à *CInterfaceSetType*. Le plus simple est de calculer une valeur sur base du nom de la classe puisqu'il ne peut pas y avoir deux classes identiques dans la solution. On peut aussi décider de retourner une chaîne de caractères plutôt qu'une valeur numérique.

L'objectif pour la fonction d'insertion d'un nouvel objet va être de comparer ce qui est comparable. Pour faire cela, il y a deux solutions possibles :

1) Dans la fonction d'insertion, on teste le type de l'élément courant avec le type de l'élément à insérer.
2) Dans la fonction *Compare()* de chaque classe dérivée, on implémente un test afin de savoir si le type de l'élément à insérer est bien égal au type de la classe dérivée où on se trouve.

La solution 1 paraît être la plus simple.

10.7.2. Les destructeurs virtuels

Comme vu dans le paragraphe précédent, une fonction virtuelle doit être une fonction qui possède la même signature au travers des différentes classes où on la déclare. En ce qui concerne les *constructeurs* et les *destructeurs*, il serait logique de penser que l'on ne peut jamais en faire des fonctions virtuelles. C'est effectivement le cas des *constructeurs* mais les *destructeurs,* eux, font exception à cette règle de par leur nom.

Toute classe possède un destructeur unique, défini soit explicitement dans la définition de la classe, soit implicitement par le compilateur. Un destructeur explicite peut être défini comme *virtuel*. Regardons l'exemple suivant.

```
class CX
       {
       public :
       CX()
           {
           m_pPtr = new int [2] ;
           cout << "X().   ";
           }
       ~CX()
           {
           delete [] m_pPtr;
           cout << "~X().\n";
           }
       private:
       int *m_pPtr;
       };

class CY : public CX
       {
       public:
       CY()
           {
           m_pQ = new int [1023];
           cout << "Y() : Y::q = " << m_pQ << ".  ";
           }
       ~CY()
           {
           delete [] m_pQ;
           cout << "~Y().   ";
           }
       private:
       int *m_pQ;
       };

int main ()
       {
       for (int dI = 0; dI < 8 ; dI++)
           {
           CX *pR = new CY;
           delete pR;
           }
       }
```

Le résultat du programme sera :

```
X().    Y():  Y::q = xxxxxxx.    ~X().
X().    Y():  Y::q = xxxxxxx.    ~X().
X().    Y():  Y::q = xxxxxxx.    ~X().
X().    Y():  Y::q = xxxxxxx.    ~X().
X().    Y():  Y::q = xxxxxxx.    ~X().
X().    Y():  Y::q = xxxxxxx.    ~X().
X().    Y():  Y::q = xxxxxxx.    ~X().
X().    Y():  Y::q = xxxxxxx.    ~X().
```

Dans la boucle *for*, on crée à chaque itération un nouvel objet dynamiquement. Pour rappel, on commence toujours par construire l'objet de la classe de base et ensuite l'objet de la classe dérivée. On commence donc par construire X et ensuite Y.

Pour la destruction de l'objet *pR*, comme *pR* est un pointeur vers un objet de la classe X, seul le destructeur de cette classe sera invoqué. Il y a donc un gaspillage énorme de mémoire puisque seul l'objet de la classe X est supprimé.

Pour résoudre ce problème, on va déclarer le destructeur *~X()* comme une fonction **virtuelle**.

```
class CX
       {
       public :
       CX()
           {
           m_pPtr = new int [2] ;
           cout << "X().   ";
           }
       virtual ~CX()
           {
           delete []m_pPtr;
           cout << "~X().\n";
           }
       private:
       int * m_pPtr;
       };
```

Que va-t-il se passer ? Le pointeur *pR* pointe vers un objet de la classe X( ). Comme on demande de détruire cet objet et que le destructeur de la classe de base a été déclaré *virtual*, on va invoquer le destructeur de la classe Y et ensuite le destructeur de la classe X : on commence à supprimer l'objet de la classe Dérivée et ensuite l'objet de la classe de Base. On obtiendra donc le résultat suivant :

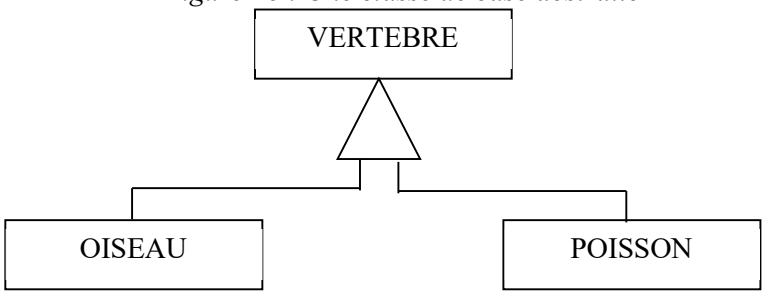
```
X().    Y():  Y::q = xxxxxxx.   ~Y()  ~X().
X().    Y():  Y::q = xxxxxxx.   ~Y()  ~X().
X().    Y():  Y::q = xxxxxxx.   ~Y()  ~X().
X().    Y():  Y::q = xxxxxxx.   ~Y()  ~X().
X().    Y():  Y::q = xxxxxxx.   ~Y()  ~X().
X().    Y():  Y::q = xxxxxxx.   ~Y()  ~X().
X().    Y():  Y::q = xxxxxxx.   ~Y()  ~X().
X().    Y():  Y::q = xxxxxxx.   ~Y()  ~X().
```

En conclusion, il est conseillé de déclarer comme *virtual* le destructeur d'une classe de base afin d'éviter les gaspillages de place mémoire.

Exemple 38 : Un destructeur virtuel

10.8. Les classes de base abstraites

Figure 16 : Une classe de base abstraite



Dans la hiérarchie ci-dessus, on peut imaginer que la fonction membre *manger* de la classe de base *vertébré* va être remplacée dans chacune de ses deux sous-classes, *oiseau* et *poisson*. Il s'agira donc de déclarer cette fonction comme virtuelle dans la classe de base. Dans cet exemple, on est certain que la fonction sera modifiée dans chaque classe dérivée. Dans ce cas, il n'est pas utile de lui donner une implémentation dans la classe de base *vertébré*.

Dans ce cas, on dit que la fonction virtuelle est *pure*. Une *fonction virtuelle pure* est une fonction virtuelle qui n'a pas d'implémentation dans sa classe de base. Pour spécifier une fonction membre virtuelle pure, on insère l'initalisateur *=0* à la place du corps de la fonction :

```
virtual int f() = 0;
```

Une classe de base **abstraite** est une classe qui possède une ou plusieurs fonctions membres virtuelles pures. Une classe **concrète dérivée** est une classe qui n'a *aucune* fonction membre virtuelle pure. Les classes de base abstraites ne peuvent pas être instanciées.

Dans une hiérarchie de classes, on peut créer des classes abstraites à n'importe quel niveau. On peut même avoir des hiérarchies de classes abstraites. Cependant, il faut garder à l'esprit que l'on ne peut pas instancier une classe de base abstraite. Par conséquent, il faut absolument définir au moins une classe concrète dérivée dans la descendance de toute classe de base abstraite.

Les fonctions virtuelles pures doivent absolument être implémentées dans chaque classe concrète dérivée de la classe de base où se trouvent les fonctions virtuelles pures.

Une *classe de base abstraite* est définie durant les premières étapes de conception d'une hiérarchie de classes. Elle donne la structure à partir de laquelle on dérive les détails dans les sous-classes. Ses fonctions virtuelles pures imposent une certaine homogénéité dans la hiérarchie.

Si une classe concrète dérivée peut ne pas implémenter toutes les fonctions virtuelles pures de la classe de base abstraite. Dans ce cas, il sera toujours interdit de déclarer un objet dans cette classe. Donc, pour déclarer un objet dans une classe concrète dérivée, il est absolument nécessaire que toutes les fonctions virtuelles pures aient été implémentées. Dans le cas où une fonction abstraite virtuelle pure n'est pas substituée, alors on recevra le message d'erreur :

```
C2259 : 'classname' cannot instantiate abstract class
```

Pourquoi les classes de base bastraites ?

Supposons que vous possédiez trois téléviseurs, chacun équipé de son propre magnétoscope. Comme la plupart des magnétoscopes, les vôtres disposent de nombreuses fonctions mais leurs manuels d'utilisation sont confus. Vos trois magnétoscopes sont donc différents et difficiles à

utiliser. Un jour, vous trouvez chez votre revendeur une télécommande qui peut commander n'importe quel magnétoscope. Elle dispose, par exemple, d'une touche *Record* qui déclenche sur chaque magnétoscope l'enregistrement du programme télévisé en cours.

Cette merveille de télécommande symbolise l'essence même des classes abstraites : une **simplification** conceptuelle des différentes implémentations grâce à une **interface unique**. Dans l'exemple des magnétoscopes, l'interface est représentée par la télécommande et les fonctions sont représentées par les fonctions de chaque magnétoscope. L'interface est la classe de base abstraite suivante :

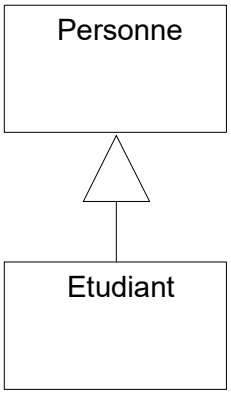
```
class CVCR
      {
      virtual void on () = 0;
      virtual void off () = 0;
      virtual void record () = 0;
      virtual void stop () = 0;
      virtual void play () = 0;
      };

class CPanasonic : public CVCR
      {
      virtual void on ();
      virtual void off ();
      virtual void record ();
      virtual void stop ();
      virtual void play ();
      };

class CSony : public CVCR
      {
      virtual void on ();
      virtual void off ();
      virtual void record ();
      virtual void stop ();
      virtual void play ();
      };

class CNokia : public CVCR
      {
      virtual void on ();
      virtual void off ();
      virtual void record ();
      virtual void stop ();
      virtual void play ();
      };
```

10.9. Exercice



Définir la classe *Etudiant* qui est une sous-classe de la classe *Personne*. La classe étudiant se verra ajouter les membres suivants :

- L'adresse sur le campus : objet de la classe *Adresse*
- L'adresse e-mail : objet de la classe *String*
- L'université : objet de la classe *String*
- La section : objet de la classe *String*
- Un constructeur
- Une fonction virtuelle Affichage
- Toute autre fonction qui pourrait être utile

10.10. L'héritage multiple

L'héritage multiple est totalement mis en oeuvre par le compilateur Visual C++ mais constitue certainement une des caractéristiques de la POO des plus controversées. Son usage impose beaucoup de contraintes.

La syntaxe à utiliser est la suivante :

```
class CClassedeDérivée : {public,private} CClassedeBase,
                         {public,private} CClassedeBase2, …
      {
      …
      }
```

Par exemple :

```
class CListWnd : public CFrameWnd, public CObList { ... };


CListWnd myListWnd;
```

Comme toutes les autres classes MFC, les deux classes *CFrameWnd* et *COblist* sont dérivées directement ou indirectement de la classe *CObject*. La classe *CObject* est une classe abstraite.

Dans l'exemple précédent, l'objet *CObject* est donc hérité deux fois par la classe *CListWnd*. Cela aboutit aux deux difficultés suivantes :

1. Toute référence à une fonction membre de la classe *CObject* sera jugée ambigüe par le compilateur.

   ```
   myListWnd.Dump(afxDump);

   // compile time error, CFrameWnd::Dump or CObList::Dump ?
   ```

2. Les fonctions membres statiques de la classe *CObject*, y compris les fonctions *operator new* et *operator delete* seront dans le même cas d'ambiguïté si on y fait référence dans la classe *CListWnd*.

Dans le cadre de cet exemple, voici comment y remédier :

```
class CListWnd : public CFrameWnd, public CObList
{
public:
    void* operator new(size_t nSize)
        { return CFrameWnd::operator new(nSize); }
    void operator delete(void* p)
        { CFrameWnd::operator delete(p); }

    void Dump(CDumpContent& dc)
        { CFrameWnd::Dump(dc);
          CObList::Dump(dc); }
     ...
};
```

11. Les templates

11.1. Les fonctions template

Considérons une fonction *max (x , y)* qui retourne la plus grande valeur de ses deux arguments. Les paramètres, *x* et *y,* peuvent être de n'importe quel type pour autant que les éléments de ce type puissent être ordonnés. Le type des deux paramètres doit être déclaré au moment de la compilation. De ce fait, on devra écrire une fonction *max (x , y)* pour chaque type pour lequel on désire faire des comparaisons. Par exemple :

```
int max (int dX, int dY)
      {
      return (dX > dY) ? dX : dY;
      }

int max (float fX, float fY)
      {
      return (fX > fY) ? fX : fY;
      }
```

et encore d'autres versions possibles.

Il s'agit là d'une lourdeur évidente qui va pouvoir être éliminée grâce à la notion de **fonction template**. Une *fonction template* est une fonction qui reçoit **un ou plusieurs types** comme paramètre. D'une manière générale, voici comment définir une telle fonction :

```
template<class T, class U, …> déclaration de fonction
```

Comme on le voit, on doit faire précéder chaque paramètre de type par le mot arbitraire *class ou typename* (en effet, le mot *class* prête à confusion).

Dans le cas de la fonction *max*, voici comment on pourrait écrire la fonction :

```
template<typename T>
T max(T tyX , T tyY)
      {
      return (tyX > tyY) ? tyX : tyY;
      }
```

Le symbole **T** est le *paramètre type*. C'est un symbole qu'on remplace à l'appel de la fonction par un type ou une classe réelle. Ce symbole peut être utilisé à l'intérieur de la fonction à tous les endroits où on aurait pu utiliser un type ordinaire.

Les fonctions *templates* sont appelées de la manière suivante :

```
int dM = 22 , dN = 66;
max<int> (dN , dM);    → Précision explicite du type
max (dN , dM);         → Précision implicite du type

String oS1 = "chaîne 1" , oS2 = "chaîne 2";
max<char *> (oS1 , oS2); ou
max(oS1 , oS2);
```

Pour chaque appel, le compilateur engendre la fonction complète en remplaçant le paramètre de type par le type ou la classe de l'argument. Pour ce faire, il cherche à réaliser une **correspondance absolue** des types : aucune conversion, qu'il s'agisse de promotion numérique ou de conversion standard n'est permise.

La définition d'une fonction Template ne peut pas être compilée seule : elle doit être connue du compilateur pour qu'il puisse instancier la bonne fonction. En conséquence, on placera généralement le code des fonctions Template dans le fichier les décrivant → tout dans le fichier .h.

Il est nécessaire que chaque paramètre de type apparaisse au moins une fois dans l'entête de la fonction template.

Encore quelques exemples :

```
template <typename T, typename U>
void fct (T a, T *b, U c)
{
       T x ;
       U *adr ;
       …
       adr = new U [10] ;
       …
       N = sizeof (T) ;
       …
}

int dN, dP ;
float fX ;
char c ;
int *pAdi ;
float *pAdf ;
CPoint oP, *pP ;

fct (dN, pAdi, fX) ;  // instancie la fonction void fct (int, int *, float)
fct (dN, pAdi, pP) ;  // instancie la fonction void fct (int, int *, CPoint*)
fct (fX, pAdf, pP) ;// instancie la fonction void fct (float, float *, CPoint*)
fct (c,  pAdi, fX) ;  // erreur car char et int * ne correspondent pas à T et
                      // T *
fct (&dN, &pAdi, fX) ;// instancie la fonction void fct (int*, int**, float)
fct (*pP, pP, dN) ;   // instancie la fonction void fct (CPoint,CPoint*, int)
```

Exemple 39 : Instanciation d'une fonction Template

Les fonctions *templates* sont une généralisation directe de la notion de *surcharge de fonctions*. On pourrait avoir écrit plusieurs versions surchargées de la fonction *max*, une pour chaque type que l'on envisage d'utiliser. La fonction *template* les remplace toutes à elle seule. Il s'agit d'une amélioration des plus importantes car on ne doit plus écrire le code qu'une seule fois et aussi parce que l'on ne doit pas connaître à l'avance la liste de tous les types à utiliser.

Remarque :

Une instruction telle que (T désignant un type quelconque) :

```
T x(3);
```

Est légale même si T n'est pas un type classe ; dans ce dernier cas, elle est simplement équivallente à :

```
T x=3;
```

Remarque :

Dans une fonction template, on peut avoir des paramètres dont le type est fixe. Dans ce cas, les règles en vigueur pour les paramètres classiques sont en vigueur et donc les conversions sont possibles.

```
template <typename T>
int Compte (T *pTab, int dN)
```

Remarque :

On peut définir plusieurs fonctions Template de même nom possédant des paramètres de types différents. La seule règle à respecter dans ce cas est que l'appel d'une fonction de ce nom ne doit pas conduire à une ambiguïté : une seule fonction Template peut être utilisable à la fois.

Par ailleurs, il est possible de fournir la définition d'une ou plusieurs fonctions particulières qui seront utilisées en lieu et place de celle instanciée par un Template. On parlera de spécialisation. Par exemple :

```
template <class T> T Min (T tyA, T tyB)
{
…
}

char * Min (char *pA, char *pB) → Précision implicite du
                                  type
{
…
}

float * Min<float> (float *pA, float *pB) → Précision
                                  explicite du type
{
…
}

Main ()
{
int dN, dP;
char *p1, *p2;

Min (dN, dP);  // Invoque la fonction Template avec le
               // type int
Min (p1, p2);  // appelle la fonction spécialisée
```

Remarque :

Lors d'un appel de fonction, afin de trouver la fonction qui doit être appelée, le compilateur regarde d'abord à essayer de trouver une fonction ordinaire avec une correspondance exacte des types. S'il n'en trouve pas, alors, il essaiera de trouver une fonction Template. Enfin, si aucune fonction template n'est trouvée, il essaiera de trouver une fonction ordinaire pour laquelle une ou plusieurs conversions pourraient assurer une correspondance.

À n'importe laquelle de ces étapes, si le compilateur trouve plus d'une fonction correspondante à l'appel de fonction, alors il y a ambiguïté et dans ce cas une erreur est provoquée.

11.2. Exercice

Exercice PRJ016 : Les fonctions templates

Créer une fonction Template permettant de calculer le carré d'une valeur de type quelconque (le résultat possèdera le même type).

Exercice PRJ017 : Les fonctions templates

Créer une fonction Template permettant de calculer la somme d'un tableau d'éléments de type quelconque : le nombre d'éléments du tableau étant fourni en paramètre. Attention, dans le cas où le type est une classe, alors on doit y trouver la surcharge de certains opérateurs comme l'addition et l'assignation. Voici le prototype de la fonction:

```
template <typename T> T
Som (T* pTab, int dNbrElem)
```

Exercice PRJ018 : Les fonctions templates

Soit cette définition de template de fonctions :

```
template <typename T, typename U>
T fct (T a, U b, T c)
{
…
}
```

Avec les déclarations suivantes :

```
int dN, dP, dQ ;
float fX ;
char s[20] ;
char c ;
```

Quels sont les appels corrects et, dans ce cas, quels sont les prototypes des fonctions instanciées ?

```
a. fct (dN, dP, dQ) ;
b. fct (dN, fX, dQ) ;
c. fct (fX, dN, dQ) ;
d. fct (s, dN, &c) ;
```

Exercice PRJ019 : Les fonctions templates

Soit cette définition de template de fonctions :

```
template <class T, class U> void fct (T a, U b) {…}   // template I
template <class T, class U> void fct (T *a, U b) {…}  // template II
template <class T>          void fct (T a, T b, T c) {…}
                                                      // template III
Void fct (int dA, float fB) {…}                       // template IV
```

Avec ces déclarations :

```
int dN, dP, dQ;
float fX, fY;
double lfZ;
```

Quels sont les appels corrects et, dans ce cas, quells sont les templates utilizes et avec quells types :

```
a. fct (dN, dP)
b. fct (fX, fY);
c. fct (dN, fX);
d. fct (dN, lfZ);
e. fct (&dN, dP);
f. fct (&dN, fX);
g. fct (&dN, &dP, &dQ);
```

Les bonnes réponses sont :

a. Template I : `void fct (int, int)`

b. Template I : `void fct (float, float)`

c. Function IV : `void fct (int, float)`

d. Template I : `void fct (int, double)`

e. Erreur : ambiguïté entre `fct (T, U)` et `fct (T *, U)`

f. Erreur : ambiguïté entre `fct (T, U)` et `fct (T *, U)`

g. Template III : `void fct (int *, int *, int *)`

La fonction Template II ne peut jamais être utilisée. En effet, chaque fois qu'elle pourrait l'être, la fonction template I peut l'être également, de sorte qu'il y a ambiguïté. La fonction Template II est donc, ici, parfaitement inutile.

Une ambiguïté ne peut être détectée que lorsque le compilateur doit instancier une fonction et non simplement au vu des définitions de template eux-mêmes : les templates sont donc acceptés par le compilateur tant que l'ambiguïté n'est pas mise en évidence par un appel la révalant.

11.3. Les classes Templates

Un *template* de classe fonctionne de la même façon qu'un *template* de fonction ; au lieu de générer une fonction, le compilateur va ici générer une nouvelle classe. La syntaxe générale est la suivante :

```
Template <typename T, typename U, int n>
class CX {…};
```

T et *U* désignent des types quelconques.
n désigne une valeur quelconque.

Par exemple :

```
template <typename T>
class CPair
{
      private :
      T m_tdValues [2];
      public:
      CPair (T tyFirst, T tySecond)
      {
          m_tdValues[0] = first;
          m_tdValues[1] = second;
      }
      CPair(const CPair<T>& oCP);
};
```

Pour se déclarer des objets dans cette classe :

```
CPair<int> oMyObject (115, 36);
CPair<float> oMyfloats (3.0, 2.18);
```

Pour définir la fonction membre à l'extérieur de la classe, il faut utiliser la synthaxe suivante :

```
template <typename T>
class CPair
{
    T m_tyA, m_tyB;
    public:
    CPair (T tyFirst, T tySecond)
    {
         m_tyA = tyFirst;
         m_tyB = tySecond;
    }
    T getmax ();
};

template <typename T>
T CPair<T>::getmax ()
{
  T tyRetval;
  tyRetval = m_tyA > m_tyB ? m_tyA : m_tyB;
  return tyRetval;
}

int main ()
{
```

```
    CPair<int> tyMyObject (100, 75);
    cout << tyMyObject.getmax();
    return 0;
}
```

On déclare une classe template en fournissant un nombre de paramètres effectifs (noms de types ou **expressions constantes**) correspondant aux paramètres figurant dans la liste.

Voici quelques exemples de la classe CX précédente :

```
CX <int, float, 5> oC1 ;          // T = int, U = float, n = 5
CX <int, int , 12> oC2 ;          // T = int, U= int,    n = 12
const int cdN = 100 ;

CX <Cpoint, double, cdN> oC3;     // T = Cpoint, U = double, n = 100
```

Un paramètre effectif peut lui-même être une classe Template. Par exemple, voici maintenant la déclaration de la classe CY :

```
template<class T> class CY {…};
```

Voici quelques instances possibles pour CX :

```
CX <CY<int>, float, 10> oC4 ;     // T = CY<int>, U = float, n = 10
```

11.3.1. Spécialisation d'un template de classe

Attention! Pour arriver à compiler et éditer les liens correctement une classe template, il est nécessaire de placer le contenu du fichier .CPP dans le fichier .H de la classe. Le code source doit être disponible dans tous les fichiers .cpp qui utilisent la classe ou la fonction template.

Because templates are compiled when required, this forces a restriction for multi-file projects: the implementation (definition) of a template class or function must be in the same file as its declaration. That means that we cannot separate the interface in a separate header file, and that we must include both interface and implementation in any file that uses the templates.

Since no code is generated until a template is instantiated when required, compilers are prepared to allow the inclusion more than once of the same template file with both declarations and definitions in a project without generating linkage errors.

Lorsque l'on dispose d'un template de classe, il est toujours possible de spécialiser une fonction membre de la classe.

Par exemple, supposons que l'on dispose de la classe template suivante :

```
template <class T, int dN> class CArray {…} ;
```

On peut écrire une version spécialisée de n'importe quelle fonction de la classe. La spécialisation signifie que la fonction sera invoquée à la condition que les paramètres de la classe reçoivent une certaine valeur. Par exemple, on pourrait imaginer écrire une version spécialisée du constructeur :

```
CArray<CPoint, 10> : :CArray(…)
```

On peut aussi spécialiser une classe. Par exemple, supposons que nous disposions du template suivant :

```
template <class T> class CPoint {…};
```

On peut fournir une version spécialisée pour le cas où T est le type *char* en procédant ainsi :

```
template <> Class CPoint<char> { … };
```

11.3.2. Paramètre constant pour une classe

Dans la déclaration d'une classe template, on peut avoir des paramètres pour les types mais aussi des valeurs en paramètres. Dans ce cas, il faut utiliser des expressions constantes lors de la déclaration d'un objet :

```
template <class T, int N> class CSequence
{
    T m_tyMemBlock [N];
  public:
    void SetMember (int dX, T tyValue);
    T GetMember (int dX);
};

template <class T, int N>
```

```
void CSequence<T,N>::SetMember (int dX, T tyValue)
{
  m_tyMemBlock[dX] = tyValue;
}

template <class T, int N>
T sequence<T,N>::GetMember (int dX)
{
  return m_tyMemBlock[dX];
}

int main ()
{
  CSequence <int,5> oMyints;
  CSequence <double,5> oMyfloats;

  oMyints.SetMember (0,100);
  oMyfloats.SetMember (3,3.1416);

  cout << oMyints.GetMember(0) << '\n';
  cout << oMyfloats.GetMember(3) << '\n';

  return 0;
}
```

On peut avoir des valeurs par défaut pour les paramètres d'une classe template :

```
template <class T=char, int N=10> class CSequence {..};
```

Pour déclarer un objet de cette classe en utilisant les valeurs par défaut, on écrira :

```
CSequence<> oMyseq;
```

Est équivallent à :

```
CSequence<char,10> oMyseq;
```

11.3.3. Assignation d'objets

On ne peut assigner entre eux que des objets de même type. Dans le cas d'objets d'une classe template, on considère qu'il y a identité de type lorsque leurs paramètres de types sont identiques et que les paramètres expression ont les mêmes valeurs.

11.3.4. Classes template et héritage

On peut combiner de plusieurs manières différentes l'héritage avec les classes template.

a) **Classe ordinaire dérivée d'une classe template**

Par exemple, si CA est une classe template définie par :

```
template <class T> class CA

class CB : public CA <int>
```

CB se dérive de la classe template CA <int>. On obtient une seule classe CB.

b) **classe template dérivée d'une classe ordinaire**

Par exemple, si CA est une classe ordinaire :

```
template <class T> class CB : public CA
```

On obtient une famille de classes de paramètre de type T.

c) **classe template dérivée d'une classe template**

Si CA est une classe template définie par :

```
template <class T> class CA
```

On peut :

- Définir une nouvelle famille de fonctions dérivées par :

  ```
  template <class T> class CB : public CA<T>
  ```

  Dans ce cas, il existe autant de classes dérivées possibles que de classes de base possibles.

- Définir une nouvelle famille de fonctions dériviées par :

  ```
  template <class T, class U> class CB : public CA <T>
  ```

  Dans ce cas, on peut dire que chaque classe de base possible peut engendrer une famille de classes dérivées de paramètres de type U.

11.3.5. Classes template et friend

Imaginons qu'une classe Template contienne une classe et une fonction *friend*. Alors, chaque instance de cette classe acceptera ces deux membres comme *friend*. Par exemple :

```
class C {};
void f (int);

template <typename T> class A
{
public:
  //...
  friend class C;
  friend void f (int);
};

A <int> oInt;
A<std::string> oStr;
```

11.3.6. Restriction des classes Template

L'usage des classes Template est une solution très souple et flexible puisque cela permet de réutiliser les mêmes classes pour n'importe quel type. Cependant, la solution consistant à utiliser le polymorphisme reste une solution encore plus puissante.

La différence essentielle entre les classes Template et les classes utilisant la solution du polymorphisme est :

- Un objet définit dans une instance d'une classe Template, ne peut contenir que des éléments du même type.
- Un objet définit dans une classe utilisant la solution du polymorphisme peut contenir des éléments de n'importe quel type.

Dans les deux solutions, l'usage d'une interface est conseillé. En effet, quand on veut utiliser une classe dont le type des éléments peut être quelconque, il est nécessaire de définir les fonctions outils grâce à une interface.

11.3.7. Exemple : une Classe Liste

Cette section montre comment on peut implementer la classe *Liste* en utilisant les *Templates*. La solution fournit les fichiers suivants :

- Le fichier *Liste* : contient la définition de la classe *Liste* ainsi que l'implémentation des fonctions membres.
- Le fichier *Cell* : contient la définition de la classe *CCell* ainsi que l'implémentation de chaque fonction membre.
- Le fichier *InterfaceListe* : définition d'une classe abstraite qui définit les fonctions dont doivent être équipées les classes que l'on veut gérer dans les objets de la classe *Liste*.
- Le fichier *Rectangle* : contient la description très partielle de la classe *CRectangle*. Il faut spécialement y remarquer la substitution de toutes les fonctions virtuelles pures définies dans l'interface de la liste.
- Le fichier *Rectangle.cpp* : contient l'implémentation des fonctions membres.

- Le fichier *Liste-Template.cpp* : contient la fonction *main()*.

11.3.7.A. Fichier Liste

```
namespace Liste
{
	#pragma once
	#include "Cell"
	using namespace Cell;

	enum etRC {OK, EXISTE, ERROR};

	template<class Data, bool bType>class CListe
	{
	private:
		CCell<Data> *m_pFirst;
		CCell<Data> *m_pLast;
		CCell<Data> *m_pCurrent;
	public:
		CListe(void);
		~CListe(void);

		void Ajoute(Data &tyData, etRC *pRC);
		CCell<Data> * Existe(Data &tyData, etRC *pRC);
		void Display();
		void StartIter();
		bool End();
		Data& Next();
	};

	template<class Data, bool bType> CListe<Data, bType>::CListe(void)
	{
		m_pFirst = new CCell<Data>();
		m_pLast = new CCell<Data>();

		m_pFirst->SetPrev(NULL);
		m_pFirst->SetNext(m_pLast);

		m_pLast->SetPrev(m_pFirst);
		m_pLast->SetNext(NULL);
	}

	template<class Data, bool bType> CListe<Data, bType>::~CListe(void)
	{
		CCell<Data> *pTemp;

		pTemp = m_pFirst->GetNext();

		while (pTemp->GetNext())
		{
			pTemp = pTemp->GetNext();
			delete pTemp->GetPrev();
		}

		delete m_pFirst;
		delete m_pLast;
	}

	template<class Data, bool bType> void CListe<Data, bType>::Ajoute(Data &tyData,
etRC *pRC)
	{
		CCell<Data> *pTemp;

		pTemp = Existe(tyData, pRC);
		if (*pRC == EXISTE)
			return;

		CCell<Data> *pNew;

		pNew = new CCell<Data> ();

		pNew->SetData(tyData);
		pNew->SetNext(pTemp);
		pNew->SetPrev(pTemp->GetPrev());
```

```
        pTemp->GetPrev()->SetNext(pNew);
        pTemp->SetPrev(pNew);
}

template<class Data, bool bType> void CListe<Data, bType>::StartIter()
{
        if (bType)
                m_pCurrent = m_pFirst->GetNext();
        else m_pCurrent = m_pLast->GetPrev();
}

template<class Data, bool bType> bool CListe<Data, bType>::End()
{
        if (bType)
                if (m_pCurrent == m_pLast)
                        return true;
                else return false;
        else if (m_pCurrent == m_pFirst)
                return true;
        else return false;
}

template<class Data, bool bType> Data& CListe<Data, bType>::Next()
{
        if (bType)
                m_pCurrent = m_pCurrent->GetNext();
        else m_pCurrent = m_pCurrent->GetPrev();

        if (bType)
                return m_pCurrent->GetPrev()->GetData();
        else return m_pCurrent->GetNext()->GetData();
}

// ************************************************************************
// La fonction Existe recherche l'existence de l'élèment à ajouter.
// Si l'élèment n'existe pas, elle retourne l'adresse de la première cellule
// dont le contenu est plus grand que la donnée à ajouter.
// ************************************************************************

template<class Data, bool bType> CCell<Data> * CListe<Data, bType>::Existe(Data
                                &tyData, etRC *pRC)
{
        CCell<Data> *pTemp;
        bool bExiste = false;
        bool bFinLecture = false;

        pTemp = m_pFirst->GetNext();

        while (pTemp->GetNext() && !bExiste && !bFinLecture)
        {
                int dResult;

                dResult = pTemp->GetData().Compare(&tyData);
                if (dResult == 0)
                        bExiste = true;
                else if (dResult > 0)
                        bFinLecture = true;
                else pTemp = pTemp->GetNext();
        }

        if (bExiste)
        {
                *pRC = EXISTE;
        }
        else
        {
                *pRC = OK;
        }
        return pTemp;
}

template<class Data, bool bType> void CListe<Data, bType>::Display()
{
        CCell<Data> *pTemp;

        pTemp = m_pFirst->GetNext();
```

```
                while (pTemp->GetNext())
                {
                        pTemp->GetData().Display();
                        pTemp = pTemp->GetNext();
                }
        }
}
```

11.3.7.B. Fichier Cell

```
namespace Cell
{
        #pragma once

        template<class Data> class CCell
        {
        private:
                CCell *m_pNext;
                CCell *m_pPrev;
                Data m_tyData;
        public:
                CCell(void);
                ~CCell(void);
                CCell *GetNext();
                CCell *GetPrev();
                Data &GetData();
                void SetData(Data tyData);
                void SetNext(CCell *pCell);
                void SetPrev(CCell *pCell);
        };

        template<class Data> CCell<Data>::CCell(void) : m_tyData()
        {
                m_pNext = NULL;
                m_pPrev = NULL;
        }

        template<class Data> CCell<Data>::~CCell(void)
        {
        }

        template<class Data> CCell<Data> *CCell<Data>::GetPrev()
        {
                return m_pPrev;
        }

        template<class Data> CCell<Data> *CCell<Data>::GetNext()
        {
                return m_pNext;
        }

        template<class Data> void CCell<Data>::SetNext(CCell<Data> *pData)
        {
                m_pNext = pData;
        }

        template<class Data> void CCell<Data>::SetPrev(CCell<Data> *pData)
        {
                m_pPrev = pData;
        }

        template<class Data> Data &CCell<Data>::GetData()
        {
                return m_tyData;
        }

        template<class Data> void CCell<Data>::SetData(Data tyData)
        {
                m_tyData = tyData;
        }
}
```

11.3.7.C. Fichier InterfaceListe

Cette classe abstraite oblige les classes Childs a implémenter certaines fonctions qui seront appelées en direct par les fonctions de la classe Liste. Il n'y a pas de polymorphisme dans ce cas-ci.

```
#pragma once

class CInterfaceListe
{
public:
	CInterfaceListe(void);
	~CInterfaceListe(void);

	virtual void Raz() = 0;
	virtual int Compare(CInterfaceListe *pData) = 0;
	virtual void Display() = 0;
};
```

11.3.7.D. Fichier Rectangle

```
#pragma once
#include "InterfaceListe"
#include <iostream>

using namespace std;

class CRectangle : public CInterfaceListe
{
private:
	int m_dLong;
	int m_dLarg;

public:
	CRectangle(void);
	CRectangle(int dLarg, int dLong);
	~CRectangle(void);
	virtual void Raz();
	virtual int Compare(CInterfaceListe *pRec);
	virtual void Display();
};
```

11.3.7.E. Fichier Rectangle.cpp

```
#include "StdAfx"
#include "Rectangle"

CRectangle::CRectangle(void)
{
	Raz();
}

CRectangle::~CRectangle(void)
{
}

void CRectangle::Raz()
{
	m_dLarg = -1;
	m_dLong = -1;
}

CRectangle::CRectangle(int dLarg, int dLong) : m_dLarg(dLarg), m_dLong(dLong)
{
}

int CRectangle::Compare(CInterfaceListe *pRec)
{
	return (m_dLarg * m_dLong) - (((CRectangle *)pRec)->m_dLarg *
		((CRectangle *)pRec)->m_dLong);
}
```

```
void CRectangle::Display()
{
	cout << "Largeur : " << m_dLarg << ", Longueur : " << m_dLong << endl;
}
```

11.3.7.F. Fichier Liste-Template.cpp

```
// Liste-Template.cpp : Defines the entry point for the console application.
//

#include "stdafx"
#include <conio>
#include <iostream>
#include "Rectangle"
#include "Liste"

using namespace Liste;
using namespace std;

int _tmain(int argc, _TCHAR* argv[])
{
	CRectangle oR1 (10, 20);
	CRectangle oR2 (20, 30);
	CRectangle oR3 (30, 40);
	CRectangle oR;
	etRC enRC;

	CListe<CRectangle, false> oListe;

	oListe.Ajoute(oR1, &enRC);
	oListe.Ajoute(oR2, &enRC);
	oListe.Ajoute(oR3, &enRC);

	oListe.Display();

	cout << "Itération sur les éléments de la liste" << endl;

	oListe.StartIter();

	while (!oListe.End())
	{
		oR = oListe.Next();
		oR.Display();
	}

	getch();

	return 0;
}
```

11.4. Exercice

Exercice PRJ020 : Les Classes Template

Soit la définition suivante d'une classe Template :

```
template <class T, int dN> class Essai
{
     T tyTab [dN];
     public:
     Essai (T tyParam);
};
```

Questions :

- Écrire le constructeur de la classe *Essai* en supposant :
  - Que la fonction doit être écrite en dehors de la classe,
  - Que le constructeur recopie la valeur reçue en argument dans chacun des éléments du tableau *tyTab*.
- Disposant de la définition précédente de la classe *Essai* et des déclarations ci-dessous:
  - `const int dN = 3;`
  - `int dP = 5;`

  Quelles sont les déclarations ci-dessous qui sont correctes ?
  Fournir dans chaque cas, une définition équivalente sous la forme d'une classe ordinaire, c'est-à-dire dans laquelle la notion de paramètre a disparu.

```
Essai <int, 10>     oA (3);
Essai <float, dN>   oB (0.0);
Essai <double, dP>  oC (2.5);
```

Exercice PRJ021 : Les Classes Template

Créer une classe Template permettant de représenter des vecteurs dynamiques capables de gérer des types quelconques. On y trouvera :

- Le ou les constructeurs,
- L'opérateur [ ],
- Le destructeur.

Pour définir la taille du vecteur, on peut utiliser un constructeur ou alors un paramètre d'initialisation. Quelle est la différence majeure entre les deux solutions ? La première doit utiliser l'allocation dynamique tandis que pour l'autre solution, on peut utiliser un tableau de taille fixe mais dont la valeur vient d'un paramètre de la classe.

Exercice PRJ022 : Les Classes Template

Serait-il possible d'écrire la classe CSetType avec une classe Template ?

12. La gestion des Erreurs

12.1. Introduction

La gestion des erreurs à l'intérieur d'une application représente sans aucun doute possible une partie essentielle et malheureusement for négligée car il s'agit d'une opération qui n'est pas très amusante. En plus, la gestion des erreurs prend du temps et donne bien souvent l'impression au programmeur de ne pas progresser. L'envie est donc très fort de négliger cette partie.

Pourtant, lorsqu'une application devient volumineuse, il va devenir essentiel de pouvoir détecter la moindre erreur qui surviendrait. En effet, détecter l'erreur au moment de son apparition va grandement aider la mise au point du problème. Si l'erreur n'est pas détectée, cela signifie que l'erreur risque de s'amplifier et provoquer des effets de bord indésirables. Dans ce cas de figure, les symptômes d'un problème apparaissent bien longtemps après l'erreur elle-même. Dans cette situation, la recherche du problème devien vraiment fort difficile.

Nous avons donc intérêt à détecter les erreurs quand elles se produisent. Quand on détecte une erreur, plusieurs comportements sont possibles :

- Soit, le programme trouve l'erreur normale et décide de continuer son exécution. Dans ce cas, la détection de l'erreur a permi d'éviter un arrêt inopiné du programme.
- Soit, le programme ne trouve pas l'erreur normale et décide de prendre actions. Ces actions peuvent être (liste non exhaustive) :

    - Je réessaie l'action qui a provoqué l'erreur : utile dans le cas où on interagit avec l'extérieur. Une information pouvait ne pas encore être disponible.
    - Je change le court normal du programme.
    - J'interagis avec l'utilisateur afin de lui signaler qu'il y a un problème.
    - J'essaie de résoudre l'erreur.
    - …

Dans ce chapitre, on présente trois techniques permettant de détecter et de gérer les erreurs au sein d'un programme orienté objet. Ces techniques sont :

- La gestion des Return Codes. Technique intéranssante mais ancienne et plus dédicacées à des programmes non orientés objets.
- La gestion des exceptions. Technique plus moderne que les return codes et qui est utilisée par la plupart des classes disponibles. D'un point de vue logique, cette technique se révèle fort semblable à celle des return codes.
- Les assertions. Cette technique est plus radicale et provoque l'arrêt immédiat du programme dés qu'une erreur est détectée. Particulièrement utile lors de la mise au point d'un programme.

Ces trois techniques sont étudiées dans les paragraphes suivants.

12.2. Les return codes

La technique des return codes est assez simple car elle consiste à munir chaque fonction d'un paramètre supplémentaire qui permet d'indiquer comment la fonction s'est exécutée. Si l'exécution est correcte alors le return code recevra une valeur positive. Dans le cas contraire, le return code contiendra une valeur permettant de comprendre le pourquoi du problème.

La fonction appelante, par le test des valeurs retournées par un return code pourra prendre les actions adéquates.

Il s'agit d'une technique fort simple et qui s'adpte bien dans les situations où de multipples fonctions sont imbiquées : il est assez simple de faire remonter un code d'erreur à la fonction de niveau supérieur.

Mais, la technique est ancienne et dans le contexte de l'orienté objet, elle a été remplacée par les exceptions.

12.3. Les Exceptions

12.3.1. Le mécanisme en général

Le C++ dispose d'un mécanisme de gestion des exceptions. Une exception est une rupture de séquence déclenchée ou levée par une fonction au moyen de l'instruction *throw*. L'instruction *throw* est suivie d'une expression. La levée d'une exception n'est pas un appel de fonction !

Lorsqu'une Exception est levée, il y a branchement vers un ensemble d'instructions dit *gestionnaire d'exception* choisi en fonction du type de l'expression indiquée dans le *throw*.

Pour qu'une portion de programme puisse intercepter une exception, il est nécessaire qu'elle figure à l'intérieur d'un bloc précédé du mot réservé *try*. Ce dernier doit ensuite être suivi d'une ou plusieurs instructions *catch* représentant les différents gestionnaires correspondants, comme dans le schéma suivant :

```
try
{
     …     // instructions susceptibles de lever une
           // exception soit directement par throw soit
           // par le biais d'une fonction appelée
}
catch (type_a)
{
     …     // traitement de l'exception correspondante au
           // type type_a
}
catch (type_b)
{
     …     // traitement de l'exception correspondante au
           // type type_b
}
…
catch (type_n)
{
     …     // traitement de l'exception correspondante au
           // type type_n
}
```

Un gestionnaire d'exceptions peut contenir des instructions *exit* ou *abort* qui mettront fin directement à l'exécution du programme. La présence de l'instruction *return* fera sortir de la fonction ayant levé l'exception. Sans présence de l'une de ces instructions, on passera aux instructions suivant le bloc *try* concerné.

Algorithme de choix d'un gestionnaire d'exceptions

Lorsqu'une exception est levée par un *throw* avec le type *T*, on recherche dans cet ordre un gestionnaire correspondant à l'un des types suivants :

1. Le type *T*,
2. Un type de base de *T*,
3. Un pointeur sur une classe dérivée (si *T* est d'un type pointeur de classe),
4. Un type indéterminé dans le gestionnaire indiqué par *catch(...)*.

Cheminement des exceptions

Lorsqu'une exception est levée par une fonction, on cherche tout d'abord un gestionnaire dans l'éventuel bloc *try* associé à cette fonction. Si le bloc *try* existe, les gestionnaires d'exception sont examinés en séquentiel et on prend le premier qui correspond.

Si on n'en trouve pas, on poursuit la recherche dans un éventuel bloc *try* associé à une fonction appelante et ainsi de suite.

Si aucun gestionnaire d'exception n'est trouvé, on appelera la fonction *terminate* qui par défaut utilise l'instruction *abort* et qui fournit un message du genre : *Abnormal Program Termination*.

Spécification d'interface

Une fonction (y compris le *main*) peut spécifier les exceptions qu'elle est susceptible de provoquer elle-même ou via une fonction qu'elle appelle.

On donne cette liste au moyen du mot réservé *throw* suivi, entre parenthèses, de la liste des exceptions concernées. Dans ce cas, toute exception non prévue et levée à l'intérieur de la fonction entraîne l'appel d'une fonction particulière nommée *unexpected*. Par défaut, cette fonction appelle la fonction *terminate*.

Comment lancer une exception

```
throw expression;
```

Par exemple :

```
throw 123;   // On lance l'entier 123, par exemple si l'erreur 123 est
survenue

throw strString("Erreur fatale. Contactez un administrateur"); // On peut
lancer un string.

throw Personnage; // On peut tout à fait lancer une instance d'une
classe.

throw 3.14 * 5.12; // Ou même le résultat d'un calcul
```

throw peut se trouver n'importe où dans le code mais, s'il n'est pas dans un bloc *try*, l'erreur ne pourra pas être rattrapée et le programme plantera.

Comment rattraper une exception

Avec le mot-clé *catch*, on peut créer un bloc de gestion d'une exception survenue. Il faut créer un bloc *catch* par type d'objet lancé. Chaque bloc *try* doit obligatoirement être suivi d'un bloc *catch*. Inversement, tout bloc *catch* doit être précédé d'un bloc *try* ou d'un autre bloc *catch*.

La syntaxe est la suivante : **catch (type const& e){...}**

On attrape les exceptions par référence constante (d'où la présence du &) et pas par valeur, ceci afin d'éviter une copie et de préserver le polymorphisme de l'objet reçu. Souvenez-vous des ingrédients du polymorphisme : une référence ou un pointeur sont nécessaires. Comme l'objet lancé pourrait avoir des fonctions virtuelles, on l'attrape via une référence, de sorte que les deux ingrédients soient réunis.

```
try
{
	// Le bloc sensible aux erreurs.
}
catch (int dE) // On rattrape les entiers lancés (pour les entiers, une
               // référencen'a pas de sens)
{
	// On gère l'erreur
}
catch (string const& strE) //On rattrape les strings lancés
{
	// On gère l'erreur
}
catch (Personnage const& oE) //On rattrape les personnages
{
	//On gère l'erreur
}
```

12.3.2. Un exemple

Imaginons une fonction réalisée de diviser un entier par un autre. Or, tout le monde sait qu'une erreur se produira si on divise par zéro. Voici une première manière de réaliser cette fonction. On peut voir que l'exception est lancée par la fonction elle-même. Cependant, le *catch* n'est pas placé dans la fonction mais bien dans la fonction appelante :

```
int division (int dA, int dB)
{
	If (dB == 0)
		throw string("ERREUR : Division par zéro !");
	else
		return dA / dB;
}
```

L'exception qui est lancées dans la fonction n'est pas rattrappée par la fonction elle-même. Pour éviter un arrêt du programme, il va être nécessaire de placer un « try » dans la fonction appelante. On arrivera ainsi à la fonction appelante suivante :

```
int main()
{
    int dA, dB;

    cout << "Valeur pour a : ";
    cin >> dA;

    cout << "Valeur pour b : ";
    cin >> dB;
```

```
        try
        {
            cout << dA << " / " << dB << " = " << division(dA, dB) << endl;
        }
        catch(string const& strChaine)
        {
            cerr << strChaine << endl;
        }

        return 0;
    }
```

12.3.3. La classe Exception

Maintenant que l'on sait gérer les exceptions, la question principale est de savoir quel type d'objet lancer.

On a présenté auparavant la possibilité de lancer des exceptions de type entier ou string. On peut aussi, par exemple, lancer un objet qui contiendrait plusieurs attributs comme :

- une phrase décrivant l'erreur ;
- le numéro de l'erreur ;
- le niveau de l'erreur (erreur fatale, erreur mineure...) ;
- l'heure à laquelle l'erreur est survenue ;
- etc.

Un bon moyen de réaliser ceci est de dériver la classe exception de la bibliothèque standard du C++. On parle d'exception et pas d'erreur puisque, si on la traite, ce n'est plus une erreur.

La classe exception

La classe exception est la classe de base de toutes les exceptions lancées par la bibliothèque standard. Elle est aussi spécialement pensée pour qu'on puisse la dériver afin de réaliser notre propre type d'exception. La définition de cette classe est :

```
    class exception
    {
           public:
           exception() throw(){ } //Constructeur.
           virtual ~exception() throw(); //Destructeur.

           virtual const char* what() const throw();
                  //Renvoie une chaîne "à la C" contenant des infos sur
                  //l'erreur.
};
```

Pour l'utiliser, il faut inclure le fichier d'en-tête correspondant soit, ici, le fichier *exception*.
On peut remarquer que la classe possède des fonctions virtuelles et donc également un destructeur virtuel. C'est un bon exemple de polymorphisme.

Les méthodes de la classe sont suivies du mot-clé *throw*. Cela sert à indiquer que ces méthodes ne vont pas lancer d'exceptions... ce qui est plutôt judicieux parce que, si la classe *exception* commence à lancer des exceptions, on n'est pas sorti de l'auberge.

Indiquer qu'une méthode ne lance pas d'exception est un mécanisme du C++ très rarement utilisé. En fait, cette classe est à peu près le seul endroit où on verra cela.

On peut alors créer sa propre classe d'exception en la dérivant grâce à un héritage. Cela donnerait par exemple :

```
#include <exception>
using namespace std;

class Erreur: public exception
{
    public:
    Erreur(int dNumero=0, string const& strPhrase="", int dNiveau=0)
       throw()
       : m_dNumero(dNumero), m_strPhrase(strPhrase), m_dNiveau(dNiveau)
    {}

    virtual const char* what() const throw()
    {
         return m_strPhrase;
    }

    int getNiveau() const throw()
    {
          return m_dNiveau;
    }

    virtual  Erreur() throw()
    {}

private:
    int m_dNumero;              //Numéro de l'erreur
    string m_strPhrase;         //Description de l'erreur
    int m_dNiveau;              //Niveau de l'erreur
};
```

à la lumière de cette nouvelle classe, on pourrait réécrire la fonction *divise* de la manière suivante :

```
using namespace std ;

int division(int dA, int dB) // Calcule a divisé par b.
{
   if(dB == 0)
      throw Erreur(1, "Division par zéro", 2);
   else
      return dA / dB;
}

int main()
{
   int dA, dB;

   cout << "Valeur pour a : ";
   cin >> dA;

   cout << "Valeur pour b : ";
   cin >> dB;

   try
   {
       cout << dA << " / " << dB << " = " << division(dA, dB) << endl;
   }
   catch(exception const& oE)
```

```
        {
            cerr << "ERREUR : " << oE.what() << endl;
        }

        return 0;
    }
```

Les méthodes de la classe sont suivies du mot-clé throw. Cela sert à indiquer que ces méthodes ne vont pas lancer d'exceptions... ce qui est plutôt judicieux parce que, si la classe exception commence à lancer des exceptions, on n'est pas sorti de l'auberge. Indiquer qu'une méthode ne lance pas d'exception est un mécanisme du C++ très rarement utilisé. En fait, cette classe est à peu près le seul endroit où vous verrez cela.

Certaines fonctions standard lancent elles aussi des exceptions. Toutes les exceptions lancées par les fonctions standard dérivent de la classe exceptionce qui permet, avec un code générique, de rattraper toutes les erreurs qui pourraient arriver. Ce code générique est le suivant :

```
catch(std::exception const& e)

{

   cerr << "ERREUR : " << e.what() << endl;

}
```

Cette possibilité résulte du polymorphisme. On attrape un objet de type exception mais, grâce aux fonctions virtuelles et à la, c'est la méthode what()de la classe Child qui sera appelée, ce qui est justement ce que l'on souhaite.

La bibliothèque standard peut lancer 5 types d'exceptions différents résumés dans le tableau suivant :

Nom de la classe	Description
`bad_alloc`	Lancée s'il se produit une erreur en mémoire.
`bad_cast`	Lancée s'il se produit une erreur lors d'un *dynamic_cast*.
`bad_exception`	Lancée si aucun`catch`ne correspond à un objet lancé.
`bad_typeid`	Lancée s'il se produit une erreur lors d'un`typeid`.
`ios_base::failure`	Lancée s'il se produit une erreur avec un flux.

On peut par exemple observer un exemple debad_allocavec le code suivant :

```
#include <iostream>

#include <vector>

using namespace std;
```

```
int main()
{
    try
    {
        vector<int> a(1000000000,1); //Un tableau bien trop grand
    }
    catch(exception const& e) //On rattrape les exceptions standard de tous types
    {
        cerr << "ERREUR : " << e.what() << endl; //On affiche la description de
l'erreur
    }
    return 0;
}
```

Cela donne le résultat suivant dans la console :

```
ERREUR : std::bad_alloc
```

12.3.4. Le Polymorphisme

Certaines fonctions standards lancent elles aussi des exceptions. Toutes les exceptions lancées par les fonctions standards dérivent de la classe *exception* ce qui permet, avec un code générique, de rattraper toutes les erreurs qui pourraient arriver. Ce code générique est le suivant :

```
catch (exception const& oE)
{
   cerr << "ERREUR : " << oE.what() << endl;
}
```

Cette possibilité résulte du **polymorphisme**. On attrape un objet de type *exception* mais, grâce aux fonctions virtuelles et à la référence (les deux ingrédients !), c'est la méthode *what()* de la **classe fille** qui sera appelée, ce qui est justement ce que l'on souhaite.

12.3.5. Les classes d'exception standards

Si, vous êtes des fainéants et que vous n'avez pas envie de créer votre propre classe d'exception, sachez qu'il existe un fichier standard qui contient des classes d'exception pour les cas les plus courants.

Le fichier **stdexcept** contient 9 classes d'exceptions séparées en 2 catégories, les exceptions « logiques » (logic errors en anglais) et les exceptions « d'exécution » (runtime errors en anglais).

Toutes les exceptions présentées dérivent de la classe *exception* et possèdent un constructeur prenant en argument une chaîne de caractères qui décrit le problème.

Nom de la classe	Catégorie	Description
domain_error	logique	Erreur de domaine mathématique.
invalid_argument	Logique	Argument invalide passé à une fonction.
length_error	Logique	Taille invalide.
out_of_range	Logique	Erreur d'indice de tableau.
logic_error	Logique	Autre problème de logique.
range_error	Exécution	Erreur de domaine.
overflow_error	Exécution	Erreur d'overflow.
underflow_error	Exécution	Erreur d'underflow.

```
int division(int dA, int dB)
{
   if(dB == 0)
      throw domain_error("Division par zéro");
   else
      return dA / dB;
}
```

12.3.6. Exercices

Exercice PRJ023 : Le Exceptions

Quels résultats fournira le programme ci-dessous avec comme données : 1 et 0 ?

```
main()
{
     int dN;
     cout << "donnez un entier : ";
     cin >> dN;

     try
     {
          cout << "début premier bloc try\n";
          if (dN)
               throw dN;
          cout << "Fin premier bloc try\n";
     }
     catch (int dN)
     {
          Cout << "catch 1 avec dN = " << dN << endl;
     }
```

```
        cout << "Suite du programme";

        try
        {
              cout << "début seconf bloc try\n";
              throw dN;
              cout << "Fin second bloc try\n";
        }
        catch (int dN)
        {
              cout << "catch 2 avec dN = " << dN << endl;
        }
        cout << "Fin Programme";
}
```

Dans le premier bloc *try*, l'exception de type *int* provoque un branchement au bloc *catch(int)* correspondant. Comme ce dernier ne comporte pas d'instruction d'interruption de programme ou de fonction, on passe aux instructions suivant le dernier gestionnaire associé, c'est-à-dire ici au bloc *try* suivant.

Exercice PRJ024 : Les Exceptions

Quels résultats fournira le programme ci-dessous avec les données : 1, 2, 3 et 4 ?

```
main()
{
     int dN;
     float fX;
     double lfZ;

     cout << "Introduisez un entier : ";
     cin >> dN;

     try
     {
           switch (dN)
           {
                      case 1: throw dN;
                              break;
                      case 2: fX = dN;
                              throw fX;
                              break;
                      case 3: lfZ = dN;
                              throw lfZ;
                              break;
           }
     }
     catch (int dN)
     {
           cout << "catch entier";
     }
     catch (float fX)
     {
           cout << "catch flottant";
           exit (-1);
```

```
     }
     cout << "suite et fin du programme";
}
```

Exercice PRJ025 : Les Exceptions

Exemple extrait du MSDN:

```
// exceptions_trycatchandthrowstatements2.cpp
// compile with: /EHsc
#include <iostream>
using namespace std;
void MyFunc( void );

class CTest
{
public:
   CTest(){};
   ~CTest(){};
   const char *ShowReason() const
   {
          return "Exception in
          CTest class.";
   }
};

class CDtorDemo
{
public:
   CDtorDemo();
   ~CDtorDemo();
};

CDtorDemo::CDtorDemo()
{
   cout << "Constructing CDtorDemo.\n";
}

CDtorDemo::~CDtorDemo()
{
   cout << "Destructing CDtorDemo.\n";
}

void MyFunc()
{
   CDtorDemo D;
   cout<< "In MyFunc(). Throwing CTest exception.\n";
   throw CTest();
}

int main()
{
   cout << "In main.\n";
```

```
    try
    {
        cout << "In try block, calling MyFunc().\n";
        MyFunc();
    }
    catch( CTest E )
    {
        cout << "In catch handler.\n";
        cout << "Caught CTest exception type: ";
        cout << E.ShowReason() << "\n";
    }
    catch( char *str )
    {
        cout << "Caught some other exception: " << str <<
                "\n";
    }
    cout << "Back in main. Execution resumes here.\n";
    return 0;
}
```

La solution sera :

```
In main.
In try block, calling MyFunc().
Constructing CDtorDemo.
In MyFunc(). Throwing CTest exception.
Destructing CDtorDemo.
In catch handler.
Caught CTest exception type: Exception in CTest class.
Back in main. Execution resumes here.
```

Exercice PRJ026 : Les Exceptions

Quels resultats fournira le programme ci-dessous ?

```
class CErreur
{
 public:
 int dNum;
};

class CErreurD: public CErreur
{
 public:
 int dCode;
};

class CA
{
 public:
 CA (int dN)
 {
     if (dN == 1)
     {
          CErreurD oE;
```

```
            oE.dCode = 12;
            oE.dNum  = 999;
            throw oE;
      }
};

main ()
{
 try
 {
      CA oA(1);

      cout << "apres creation oA(1)\n";
 }
 catch (Cerreur oE)
 {
      cout << "Exception erreur : " << oE.dNum << "\n";
 }
 catch (CerreurD oE)
 {
      cout << "Exception erreurD : " << oE.dNum <<  " "
           << oE.dCode << "\n";
 }

 cout << "Suite\n";

 try
 {
      CA oB(1);

      cout << "apres creation oB(1)\n";
 }
 catch (CerreurD oE)
 {
      cout << "Exception erreurD : " << oE.dNum <<  " "
           << oE.dCode << "\n";
 }
 catch (Cerreur oE)
 {
      cout << "Exception erreur : " << oE.dNum << "\n";
 }
}
```

Les messages générés seront :

```
Exception erreur : 999
Suite
Exception erreurD: 999 12
```

Dans le premier bloc *try*, la construction de l'objet *oA(1)* lève une exception de type *CErreurD* en transmettant une expression de type *CErreurD*. Le premier gestionnaire trouvé *(catch (CErreur oE))* puisque la classe *CErreur* est une classe de base de *CErreurD*.

Dans le second bloc *try*, en revanche, les gestionnaires ont été disposés dans l'ordre inverse.

Exercice PRJ027 : Les Exceptions

Écrire une classe *CPoint* avec deux coordonnées entières qui dispose d'un constructeur à deux arguments levant une exception lorsque les deux composantes sont égales. De plus, l'appel d'un constructeur sans argument ou à un seul argument devra également lever un autre type d'exception.

Écrire un programme d'utilisation de la classe *CPoint*, interceptant convenablement les exceptions prévues, en en mentionnant la cause.

12.4. Les Assertions

Les exceptions c'est bien mais, il y a des cas où mettre en place tous ces blocs *try* / *catch* est fastidieux. Il existe un autre mécanisme de détection et de gestion qui vient du langage C : les **assertions**.

Une assertion permet de tester si une expression est vraie ou non. Si c'est vrai, rien ne se passe et le programme continue. Par contre, si le test est négatif, le programme s'arrête brutalement et un message d'erreur s'affiche dans le terminal.

Il faut inclure le fichier `<cassert>`.

Désactivations des assertions

Un autre point fort des assertions est la possibilité de les désactiver totalement. Dans ce cas, le compilateur ignore simplement les lignes assert(...) et n'effectue pas le test qui se trouve entre

les parenthèses. Ainsi, le code sera (légèrement) plus rapide, mais aucun test ne sera effectué. Il faut donc choisir.

Pour désactiver les assertions, il faut ajouter l'option -DNDEBUG à la ligne de compilation. Cela se fait via le menu project > build options. Dans la fenêtre qui s'ouvre, sélectionnez l'onglet Compiler settings puis, dans le champ Other options, ajoutez simplement -DNDEBUG comme à la figure suivante.

13. Les Composants standards

13.1. Introduction

Les exercices précédents ont été volontairement résolus sans recourir aux composants standards introduits dans la norme C++. Manifestement, l'existence de ces composants influe sur certains exercices, soit en rendant leur solution quasiment triviale, soit en la simplifiant notablement.

13.2. Exercice

Exercice PRJ028 : Créer une classe manipulant des entiers

Réaliser une classe permettant de manipuler des ensembles d'entiers. On devra pouvoir réaliser sur un tel ensemble les opérations classiques suivantes :

- Lui ajouter un nouvel élément,
- Connaître son cardinal (nombre d'éléments),
- Savoir si un entier donné lui appartient ou non.

Ici, on n'effectuera aucune allocation dynamique d'emplacements mémoire. Écrire un programme *main* utilisant la classe *CSetInt* pour déterminer le nombre d'entiers différents contenus dans une liste d'entiers lue à l'écran.

Solution

La réalisation de cette classe peut se faire facilement au moyen de la classe template *set* à laquelle on fourni le type *int* en paramètre. La gestion dynamique sera prise en charge directement par cette classe. Extrait du MSDN:

```
template <
   class Key,
   class Traits=less<Key>,
   class Allocator=allocator<Key>
>
class set
```

Parameters

Key
: The element data type to be stored in the set.

Traits
: The type that provides a function object that can compare two element values as sort keys to determine their relative order in the set. This argument is optional, and the binary predicate **less** <*Key*> is the default value.

Allocator
: The type that represents the stored allocator object that encapsulates details about the set's allocation and deallocation of memory. This argument is optional, and the default value is **allocator**<*Key*>.

Exactement comme on l'a fait en C, il est possible de fournir à cette classe des functions lui permettant de réaliser de l'allocation dynamique et une comparaison entre les elements.

Exercice PRJ029 : Ajouter un itérateur à classe permettant de manipuler des ensembles d'entiers

Modifier la classe précédente de manière à disposer de ce que l'on nomme un *itérateur* sur les différents éléments de l'ensemble. Il s'agit d'un mécanisme permettant d'accéder séquentiellement aux différents éléments.

On devra prévoir trois nouvelles fonctions membres :

- La fonction *init* qui initialise la lecture séquentielle,
- La fonction *next* qui fournit l'élément suivant lorsque celui-ci existe,
- La fonction *exist* qui vérifie s'il existe encore un élément non exploré.

On pourra compléter le programme de test *main* afin qu'il puisse afficher le contenu complet de l'objet de la classe.

Solution

De nouveau, on va ici utiliser la classe *set* qui dispose d'un itérateur intégré : *set<int>::iterator*. Les fonctions membres *begin* et *end* fournissent les valeurs initiales et finales à utiliser pour explorer l'ensemble à l'aide d'un tel itérateur. Attention que *end* ne pointe pas sur le dernier élément de l'objet mais bien sur la position après. L'avancement de l'itérateur s'obtient par l'opérateur ++.

Implémentation d'un itérateur dans une classe :

```
template<class T, std::size_t thesize>
class carray {
  private:
    T v[thesize]; // fixedsize array of elements of type T

  public:
    // type definitions
    typedef T        value_type;
    typedef T*       iterator;
    typedef const T* const_iterator;
    typedef T&       reference;
    typedef const T& const_reference;
    typedef std::size_t    size_type;
    typedef std::ptrdiff_t difference_type;

    // iterator support

    iterator begin()
    {
      return v;
    }
    const_iterator begin() const
    {
      return v;
    }
    iterator end()
    {
     return v + thesize;
```

```
        }
        const_iterator end() const
        {
         return v + thesize;
        }

        // direct element access

        reference operator[](std::size_t i)
        {
         return v[i];
        }

        const_reference operator[](std::size_t i) const
        {
         return v[i];
        }

        // size is constant

        size_type size() const
        {
         return thesize;
        }

        size_type max_size() const
        {
         return thesize;
        }

        // conversion to ordinary array

        T* as_array()
        {
         return v;
        }
    };
```

Exercice PRJ030 : Utiliser la classe Vecteur

Utiliser une classe vecteur permettant de représenter des *vecteurs dynamiques*, càd dont la taille n'est pas connue à la compilation. Lors de la construction d'un objet de cette classe, on devra pouvoir donner une approximation sur le nombre d'éléments qui seront placés dans l'objet que l'on construit.

On devra pouvoir utiliser l'opérateur [ ] afin de pouvoir accéder à un élément spécique de la collection.

Solution

La classe *vect<int>* pourra être utilisée pour réaliser ce qui est demandé. Elle dispose d'un itérateur et d'un opérateur [ ] permettant l'accès directà un élément.

Cependant, l'opérateur [ ] n'est pas protégé contre les index qui dépasseraient la valeur maximale. À la place de cet opérateur, il y a possibilité d'utilise rla fonction membre *at* qui déclenche une exception dans le cas où on dépasse la valeur maximale de l'index.

Exercice PRJ031 : Utiliser la classe Vecteur en sécurisant le [ ]

Reprendre l'exercice précédent et faire en sorte de disposer d'un opérateur [ ] résistant à toute erreur.

Solution

Il faut créer une nouvelle classe héritée de la classe *vector<int>* et dans laquelle on va écrire une substitution de l'opérateur [ ].

Exercice PRJ032 : Tableaux dynamiques à deux indices

En s'inspirant de l'exercice précédent, il s'agit de réaliser une classe permettant de représenter des tableaux dynamiques d'entiers à deux indices.

Solution

On peut réutiliser la classe *vector* utilisée à l'exercice précédent mais en indiquant cette fois que le type de chaque élément du tableau est lui-même un *vecteur*. Cette approche ne diffère pas de celle du C qui considère que chaque élément d'un tableau à plusieurs dimensions est lui-même un tableau.

14. Solutions pour les exercices du dernier chapitre

Solution de l'exercice 28:

```
#include "stdafx"
#include <set>
#include <iostream>

using namespace std;

int _tmain(int argc, _TCHAR* argv[])
{
	set<int> colInt;
	int dNum;

	for (int dI = 0; dI < 20; dI++)
	{
		cout << "Introduisez votre nombre" << endl;
		cin >> dNum;

		colInt.insert(dNum);
	}

	cout << "Il contient " << colInt.size() << " entiers
	différents" << endl;

	return 0;
}
```

On peut aussi écrire (attention à bien mettre un espace entre les deux '>' afin de ne pas confondre avec l'opérateur shift >>):

```
#include "stdafx"
#include <set>
#include <iostream>

using namespace std;

int _tmain(int argc, _TCHAR* argv[])
{
	set<int, less<int> > colInt;
	int dNum;

	for (int dI = 0; dI < 20; dI++)
	{
		cout << "Introduisez votre nombre" << endl;
		cin >> dNum;

		colInt.insert(dNum);
	}

	cout << "Il contient " << colInt.size() << " entiers
	différents" << endl;

	return 0;
}
```

Solution de l'exercice 29 :

```
#include "stdafx"
```

```
#include <set>
#include <iostream>

using namespace std;

int _tmain(int argc, _TCHAR* argv[])
{
	set<int, less<int> > colInt;
	int dNum;

	for (int dI = 0; dI < 5; dI++)
	{
		cout << "Introduisez votre nombre" << endl;
		cin >> dNum;

		colInt.insert(dNum);
	}

	cout << "Il contient " << colInt.size() << " entiers
	différents"
	<< endl;

	set<int>::iterator oItem;

	for (oItem = colInt.begin(); oItem != colInt.end();
	oItem++)
		cout << *oItem << " ";

	return 0;
}
```

Solution de l'exercice 30 (avec en prime une gestion d'exception) :

```
#include "stdafx"
#include <vector>
#include <iostream>

using namespace std;

int _tmain(int argc, _TCHAR* argv[])
{
	try
	{
		vector<int> colVect(6);
		int dI;

		for (dI = 0; dI < 6; dI++)
			colVect.at(dI) = dI;

		for (dI = 0; dI < 8; dI++)
			cout << colVect.at(dI) << " ";
	}
	catch(out_of_range e)
	{
		cout << "Exception out of range\n";
		exit (-1);
	}
}
```

Solution de l'exercice 31 :

```
class NewVect : public vector<int>
{
public:
	NewVect (int dTaille) : vector<int> (dTaille)
	{
	}

	int & operator [] (int dI)
	{
		if (dI < 0 || dI >= size())
			dI = 0;

		return at(dI);
	}
};

int _tmain(int argc, _TCHAR* argv[])
{
	try
	{
		NewVect colVect(6);
		int dI;

		for (dI = 0; dI < 6; dI++)
			colVect[dI] = dI;

		for (dI = 0; dI < 8; dI++)
			cout << colVect[dI] << " ";
	}
	catch(out_of_range e)
	{
		cout << "Exception out of range\n";
		exit (-1);
	}
}
```

Solution de l'exercice 32 :

```
#include "stdafx"
#include <vector>
#include <iostream>

using namespace std;

int _tmain(int argc, _TCHAR* argv[])
{
	vector<vector<int> > tTab(4);
	vector<int> tInit(3);

	int dI, dJ;

	for (dI = 0; dI < 4; dI++)
		tTab[dI] = tInit;

	for (dI = 0; dI < 4; dI++)
		for (dJ = 0; dJ < 3; dJ++)
			tTab[dI][dJ] = dI + dJ;

	for (dI = 0; dI < 4; dI++)
```

```
        {
                for (dJ = 0; dJ < 3; dJ++)
                        cout << tTab[dI][dJ] << " ";

                cout << "\n";
        }
}
```

L'usage de l'objet temporaire *tInit* est obligatoire.

Même solution mais en utilisant cette fois la méthode *at()* :

```
int _tmain(int argc, _TCHAR* argv[])
{
        vector<vector<int> > tTab(4);
        vector<int> tInit(3);

        int dI, dJ;

        for (dI = 0; dI < 4; dI++)
                tTab.at(dI) = tInit;

        for (dI = 0; dI < 4; dI++)
                for (dJ = 0; dJ < 3; dJ++)
                        (tTab.at(dI)).at(dJ) = dI + dJ;

        for (dI = 0; dI < 4; dI++)
        {
                for (dJ = 0; dJ < 3; dJ++)
                        cout << (tTab.at(dI)).at(dJ) << " ";

                cout << "\n";
        }
}
```

Index

A

Abstraction
    Définition, 12
Allocation dynamique d'objets, 96
Argument
    Valeur par défaut, 66

C

Cardinalité, 13
Classe
    Définir une fonction membre, 46
    Définition, 43
    fstream, 35
    Où placer sa définition, 46
Classe de base abstraite, 134
Classe template, 144
Commentaire, 20
Const, 22
Constructeur
    de conversion, 77
    Définition, 52
    invoquer un constructeur, 53
    Ordre d'exécution, 105
    par copie, 61
    par défaut, 52
Conversion
    d'un objet vers un autre type, 77
    vers un objet, 76

D

Destructeur, 54
    Ordre d'exécution, 106
Destructeur virtuel, 131

F

Fichier en mode texte, 38
Fonction
    main (), 20
    open (), 38
    put (), 33
    seekp (), 33
    tellp (), 33
    write (), 33
Fonction const, 23
Fonction inline, 70
Fonction template, 139
Fonction virtuelle pure, 134
Fonctions virtuelles, 121
Friend
    Définition, 73

H

Héritage
    de type public et private, 112
    Définition, 15
    Dominance des données membres, 114

Ne pas confondre surcharge et substitution, 114
Ordre d'invocation des constructeurs et destructeurs, 115
Substitution des fonctions membres, 111
Utilisation, 111

L

Liaison dynamique
Définition, 18
Lien dynamique, 123
Liste d'initialisation, 59

M

Modes d'ouverture d'un fichier, 38
Mots Réservés
Les nouveaux, 21

N

Null, 31

O

Object-Oriented Programming
Définition, 8
Objet
Définition, 45
Invoquer une fonction membre, 46
Membre d'une autre classe, 105
Objet cerr, 32
Objet cin, 32
Objet clog, 32
Objet cout, 32
Opérateur cast, 30
Opérateur de conversion, 78
Opérateur de portée, 21
Opérateur delete, 24
aussi pour des objets, 96
Opérateur d'extraction >>, 32
Opérateur d'insertion <<, 32
Opérateur new, 24
aussi pour des objets, 96
Operator, 85

P

Passage par référence, 27
Pointeur this, 49
Pointeur vers une classe de base, 121
Polymorphisme, 121
Définition, 18
Private, 44
Protected, 45
Public, 44

R

Référence, 27
Relation
Agrégation, 14, 105
Association, 13
Généralisation, 13
Spécialisation, 13

S

Signature de fonction, 68
Spécificateurs d'accès, 44
Static
    données membres, 98
    fonctions membres, 99
Struct, 21
Substitution
    Définition, 17
Surcharge de fonction, 68
Surcharge de l'opérateur d'affectation, 94
Surcharge de l'opérateur d'indexation, 93
Surcharge des opérateurs, 84
Surcharge des opérateurs ++ et --, 89
Surcharge des opérateurs de flux, 88

T

Type de donnée abstrait, 45

V

Virtual, 122